



SQL from Scratch: SQL for Data Engineering

From your very first SELECT to idempotent, incremental,
tested pipelines

14

modules + a capstone

127

executed queries

```
1  -- where this tutorial takes you
2  WITH latest AS (
3      SELECT *,
4          ROW_NUMBER() OVER (
5              PARTITION BY order_id
6              ORDER BY updated_at DESC
7          ) AS rn
8      FROM raw_orders
9  )
10 SELECT
11     order_date,
12     COUNT(*)      AS n_orders,
13     SUM(amount)   AS revenue
14 FROM latest
15 WHERE rn = 1
16     AND status = 'completed'
17 GROUP BY order_date
18 ORDER BY order_date;
```

How to read this deck

Every slide teaches one idea. Coloured tags tell you what kind of help a text box is giving you, so you can skim or dig in.

MENTAL MODEL

A picture for your head

The intuition behind a feature, before any syntax.

TRAP

Where people get burned

Bugs that run without errors but return wrong data.

DATA ENGINEERING LENS

Why pipelines care

How the idea shows up in real data engineering work.

DIALECT NOTE

Same idea, different spelling

How Snowflake, BigQuery, SQL Server and others differ.

PRO TIP

Habits of senior engineers

Small practices that prevent large incidents.

TRY IT

Stop and type

Exercises with the expected output shown, solution next.

1 **Numbered markers** link a line of code to the note that explains it.

RESULT · 3 rows

Result tables are real output: every query was executed on the sample data.

orders

Dark tags name the table shown.

Your route from zero to pipeline-ready

Five stages. Each module ends with a recap, and most include an exercise with the expected output.

STAGE 1 · READ

Query one table

- 01 Foundations
- 02 SELECT basics
- 03 Types and functions
- 04 Aggregation

STAGE 2 · COMBINE

Query many tables

- 05 Joins
- 06 Set operations, subqueries and CTEs
- 07 Window functions

STAGE 3 · BUILD

Create and change

- 08 DDL: defining tables
- 09 DML and transactions
- 10 Data modelling

STAGE 4 · ENGINEER

Run it every day

- 11 Pipeline patterns
- 12 Data quality
- 13 Performance
- 14 Advanced toolkit

STAGE 5 · SHIP

Prove it

Capstone: raw files to a tested daily revenue mart

Wrap-up: patterns, interview questions, next steps

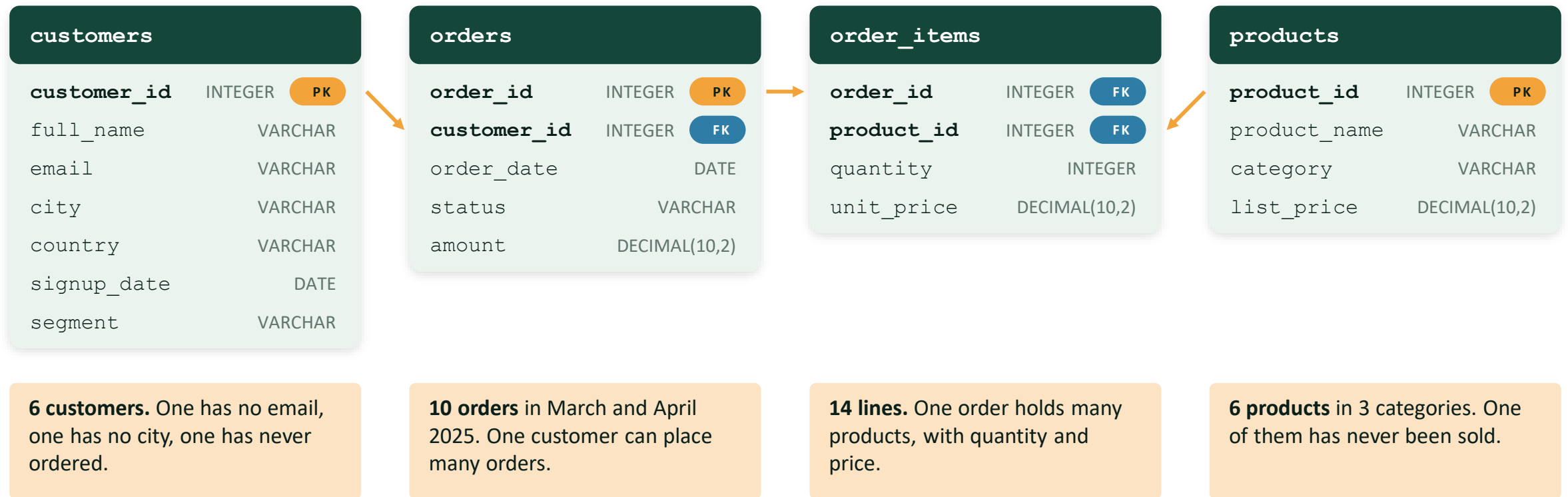
★ PRO TIP

Type, do not just read

SQL is a motor skill. Run every example yourself, then change one thing and predict the output before you press Enter.

Meet Brightcart, our running example

A small online electronics shop: four core tables linked by keys. Its awkward rows (missing values, unmatched keys) are deliberate, because real pipelines are made of them.



The data: customers and products

Keep this slide handy. When a result surprises you, come back and trace it by hand.

customers

customer_id	full_name	email	city	country	signup_date	segment
1	Ada Moreno	ada@mail.com	Madrid	ES	2025-01-15	consumer
2	Ben Carter	ben@mail.com	Leeds	UK	2025-02-03	business
3	Chloe Dubois	chloe@mail.com	Lyon	FR	2025-02-20	consumer
4	Dev Patel	dev@mail.com	NULL	UK	2025-03-11	consumer
5	Elena Rossi	NULL	Milan	IT	2025-04-02	business
6	Farid Khan	farid@mail.com	Leeds	UK	2025-04-18	consumer

products

product_id	product_name	category	list_price
101	Laptop Stand	accessories	45.00
102	USB-C Hub	accessories	30.00
103	Mech Keyboard	peripherals	90.00
104	Wireless Mouse	peripherals	25.00
105	Monitor 27in	displays	220.00
106	Webcam HD	peripherals	60.00

MENTAL MODEL

Spot the gaps

Dev has no city and Elena has no email (highlighted). Farid has never ordered, and no one has bought the Webcam HD. These rows power the NULL and join lessons.

The data: orders and order_items

orders

order_id	customer_id	order_date	status	amount
1001	1	2025-03-01	completed	135.00
1002	2	2025-03-01	completed	220.00
1003	1	2025-03-05	completed	55.00
1004	3	2025-03-07	cancelled	90.00
1005	2	2025-03-12	completed	310.00
1006	4	2025-03-15	completed	25.00
1007	5	2025-04-03	completed	265.00
1008	2	2025-04-10	shipped	60.00
1009	1	2025-04-12	completed	90.00
1010	3	2025-04-20	completed	75.00

order_items

order_id	product_id	quantity	unit_price
1001	101	1	45.00
1001	103	1	90.00
1002	105	1	220.00
1003	102	1	30.00
1003	104	1	25.00
1004	103	1	90.00
1005	103	1	90.00
1005	105	1	220.00
1006	104	1	25.00
1007	101	1	45.00
1007	105	1	220.00
1008	102	2	30.00
1009	101	2	45.00
1010	104	3	25.00

orders.amount equals the sum of that order's lines.
Order 1001: 45 + 90 = 135.

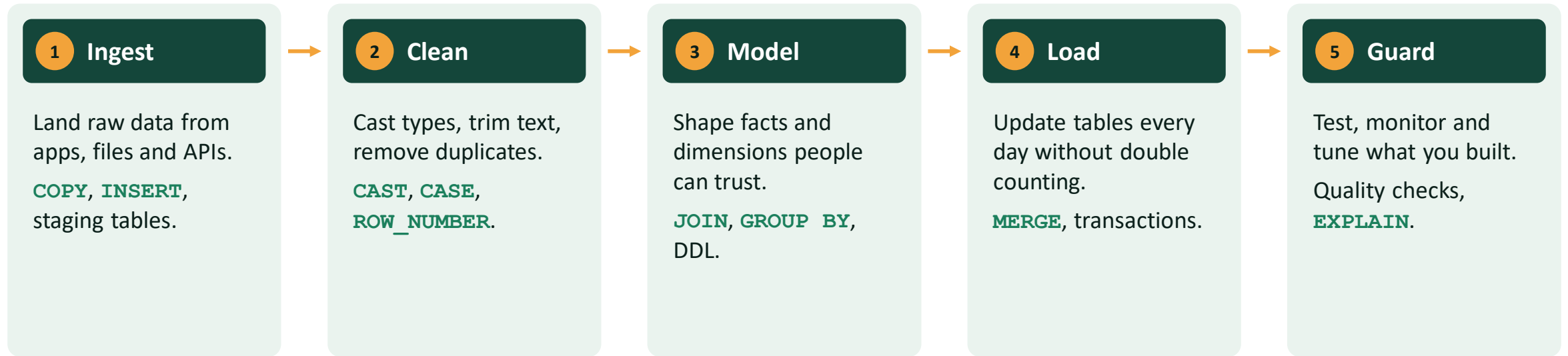
Statuses: eight completed, one cancelled (1004), one shipped (1008).

Two orders share a date (1 March). That tie matters in Module 7.

Totals: 1,325.00 for all orders; 1,175.00 completed.

What a data engineer does with SQL

Analysts use SQL to ask questions. Data engineers use it to build the tables those questions are asked of.



WHY IT MATTERS

The difference in mindset

A one-off query only has to be right once. A pipeline query has to be right every night, on data you have not seen yet, even when it is re-run.



MODULE 01

Foundations

What a database is, what SQL is, and where both sit in a modern data platform.

BY THE END YOU CAN

- 1 Explain tables, rows, columns and keys in plain words
- 2 Say what "declarative" means and why it matters
- 3 Tell OLTP systems from analytical warehouses
- 4 Name the five families of SQL statements
- 5 Set up a practice database and run a first query

A relational database stores data in tables

Like a spreadsheet, but strict: every column has a name and a type, and every row has the same shape.

1	customers	4			
	customer_id	full_name	city	country	segment
1	1	Ada Moreno	Madrid	ES	consumer
2	2	Ben Carter	Leeds	UK	business
3	3	Chloe Dubois	Lyon	FR	consumer
4	4	Dev Patel	NULL	UK	consumer
5	5	Elena Rossi	Milan	IT	business
6	6	Farid Khan	Leeds	UK	consumer

1 **Primary key** (the shaded first column). Unique in every row: it points at exactly one customer.

2 **Row (record)**. One real-world thing: here, one customer.

3 **Cell**. A single value, or **NULL** when unknown, as with Dev's city.

4 **Column (field)**. One attribute, such as country, with one data type for the whole table.

MENTAL MODEL

Tables describe things, keys describe relationships

"Relational" means tables are linked by shared values: `orders.customer_id` holds a value that also exists in `customers.customer_id`.

SQL is declarative



You describe the result you want. The database decides how to get it.

In Python you write the loop. In SQL you state the rule ("orders over 100, newest first") and a query optimizer chooses the algorithm, the index and the join order.

YOU WRITE

What

Which columns, which rows, grouped and sorted how.

THE ENGINE PICKS

How

Scan or index, hash join or merge join, one machine or one hundred.

SO THAT

The same SQL scales

A query that works on 10 rows still works on 10 billion; only the plan changes.

Five families of SQL statements

SQL is one language with five jobs. You will use the first three every day.

DQL

Query

Read data.

SELECT

Modules 2 to 7.

DDL

Define

Create and change structures.

CREATE ALTER DROP

Module 8.

DML

Manipulate

Change rows.

**INSERT UPDATE
DELETE MERGE**

Module 9.

TCL

Transact

Group changes so they succeed or fail together.

**BEGIN COMMIT
ROLLBACK**

DCL

Control

Decide who may do what.

GRANT REVOKE

Module 14.

Two kinds of database: OLTP and OLAP

The same SQL runs on both, but they are built for opposite workloads. Data engineers move data from the first to the second.

OLTP · RUNS THE BUSINESS

- **Job:** record transactions as they happen: an order, a payment, a login.
- **Queries:** tiny and frequent. Read or write a handful of rows by key.
- **Storage:** row-oriented, heavily indexed, highly normalised.
- **Examples:** PostgreSQL, MySQL, SQL Server, Oracle.
- **Users:** applications and their customers.

OLAP · ANALYSES THE BUSINESS

- **Job:** answer questions across history: revenue by month, churn by cohort.
- **Queries:** few and huge. Scan millions of rows, aggregate, join.
- **Storage:** column-oriented, compressed, partitioned.
- **Examples:** Snowflake, BigQuery, Redshift, Databricks, DuckDB.
- **Users:** analysts, dashboards, ML pipelines.

⚙️ DATA ENGINEERING LENS

Your job lives on the arrow between them

Pipelines copy data out of OLTP systems, reshape it, and load it into an OLAP warehouse where heavy questions cannot slow down the live application.

Where SQL sits in the modern data stack



WHY IT MATTERS

SQL is the one skill used at every stage

Orchestrators, languages and vendors change every few years. SQL has been the interface to data since the 1970s and still is in every tool above.

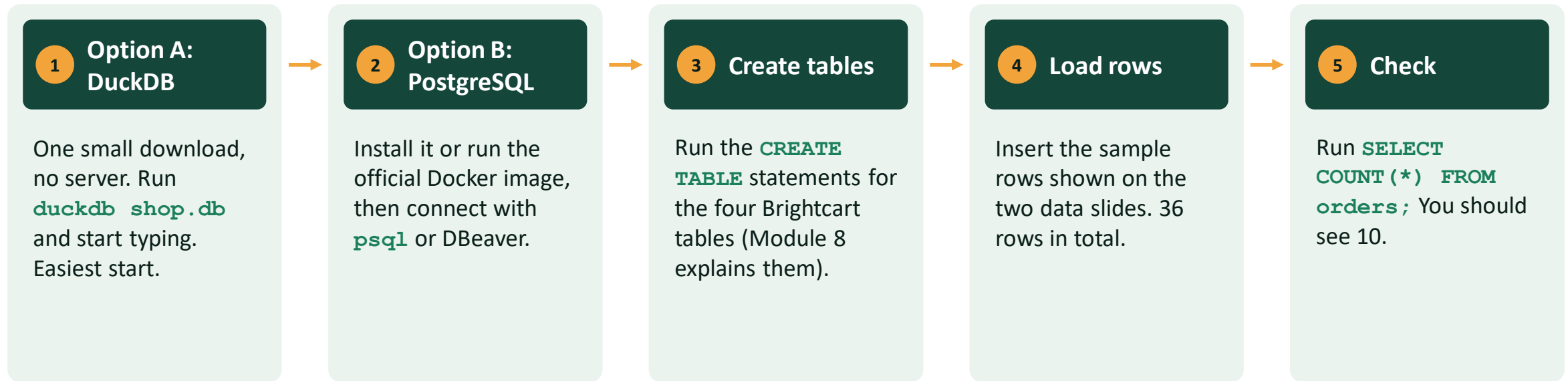
One language, many dialects

Roughly 90% of what you learn transfers unchanged. This deck teaches standard SQL in PostgreSQL style and flags differences as they appear.

Engine	Type	Where you meet it	Worth knowing
PostgreSQL	OLTP, row store	Application databases, small warehouses	Closest to the SQL standard. Best engine to learn on.
MySQL	OLTP, row store	Web applications	No FULL OUTER JOIN . Uses backticks for identifiers.
SQL Server	OLTP, row store	Enterprise systems	TOP instead of LIMIT . Dialect is called T-SQL.
Snowflake	OLAP, columnar	Cloud data warehouse	Uppercases unquoted names. QUALIFY , VARIANT for JSON.
BigQuery	OLAP, columnar	Google Cloud warehouse	Billed by bytes scanned on demand. STRUCT and ARRAY are first class.
Spark SQL / Databricks	OLAP, distributed	Lakehouses over files	Tables are Parquet or Delta files. MERGE needs Delta.

Set up a practice lab in ten minutes

Pick one option. All three run every example in this deck with little or no change.



★ PRO TIP

Keep the data tiny on purpose

With ten orders you can verify any result by hand. Learn correctness on small data first; Module 13 covers what changes at scale.

Your first queries

```
1  -- 1. SQL can compute without any table
2  SELECT 2 + 2 AS answer;
3
4  -- 2. Read three columns from a table
5  SELECT customer_id, full_name, country
6  FROM customers;
```

1

2

3

4

1

Two dashes start a **comment**. The database ignores the rest of the line.

2

SELECT returns a table, even here: one row, one column named **answer**.

3

List the columns you want, separated by commas.

4

FROM names the table. The **semicolon** ends the statement.

RESULT · 6 rows

customer_id	full_name	country
1	Ada Moreno	ES
2	Ben Carter	UK
3	Chloe Dubois	FR
4	Dev Patel	UK
5	Elena Rossi	IT
6	Farid Khan	UK



RULE OF THUMB

Keywords are case-insensitive. This deck writes them in CAPITALS and names in lowercase so structure is visible at a glance.

Five errors you will meet in week one

Error messages are precise once you know the vocabulary. Read the first line, find the position, fix one thing.

The database says	What it means	Usual fix
<code>syntax error at or near "FROM"</code>	The parser got lost just before this word.	Look left: a trailing comma or missing keyword.
<code>relation "order" does not exist</code>	No table or view has that name.	Check spelling, plural, and schema (<code>sales.orders</code>).
<code>column "name" does not exist</code>	The table exists, the column does not.	Inspect the table. Strings need single quotes.
<code>column must appear in the GROUP BY clause</code>	You mixed raw and aggregated columns.	Group by it or wrap it in an aggregate (Module 4).
<code>column reference "customer_id" is ambiguous</code>	Two joined tables both have that column.	Prefix it with the table alias (Module 5).



TRAP

Single quotes and double quotes are not interchangeable

'UK' is a text value. "UK" is an identifier, meaning a column or table named UK. Mixing them causes the third error above.

Module 1 recap

- ✓ A table has typed columns and rows of the same shape; a primary key identifies each row.
- ✓ Relationships are expressed by matching values, such as `orders.customer_id = customers.customer_id`.
- ✓ SQL is declarative: you state the result, the optimizer chooses the method.
- ✓ OLTP systems record transactions; OLAP warehouses analyse history. Pipelines connect them.
- ✓ Statements fall into DQL, DDL, DML, TCL and DCL.
- ✓ Dialects differ at the edges; the core you learn here is portable.

 NEXT UP

Module 2: reading data with SELECT, WHERE, ORDER BY and LIMIT



MODULE 02

SELECT basics

Choose columns, filter rows, sort and limit: the four moves behind every query you will ever write.

BY THE END YOU CAN

- 1 Select and rename columns, and compute new ones
- 2 Filter rows with comparison and logical operators
- 3 Handle NULL correctly with three-valued logic
- 4 Sort and limit results deterministically
- 5 Explain the order in which a query really runs

Anatomy of a SELECT statement

Five clauses, always written in this order. Only the first is required.

```
SELECT full_name, city
```

Which **columns** to return. Can also be calculations.

```
FROM customers
```

Which **table** to read.

```
WHERE country = 'UK'
```

Which **rows** to keep. Text values go in single quotes.

```
ORDER BY full_name
```

How to **sort** the output. Without it, row order is not guaranteed.

```
LIMIT 10;
```

How **many rows** to return. The semicolon ends the statement.

MENTAL MODEL

Read it as a sentence

"Give me the name and city, from customers, where the country is UK, sorted by name, first ten only."

Choosing and renaming columns

```
1 SELECT
2     customer_id,
3     full_name AS customer,
4     country AS country_code
5 FROM customers;
```

1

2

3

- 1 One column per line makes changes easy to review in version control.
- 2 **AS** gives the output column an **alias**. The table itself is unchanged.
- 3 Aliases use **snake_case**. Avoid spaces and capitals so nobody ever needs quotes.

RESULT · 6 rows

customer_id	customer	country_code
1	Ada Moreno	ES
2	Ben Carter	UK
3	Chloe Dubois	FR
4	Dev Patel	UK
5	Elena Rossi	IT
6	Farid Khan	UK

DATA ENGINEERING LENS

Renaming is real work in pipelines: staging models exist largely to turn source names like **CUST_NM** into names people understand.

Why pipelines avoid `SELECT *`

✗ FRAGILE

```
1 SELECT *
2 FROM orders;
```

- A column added upstream silently appears downstream.
- A column **reordered** upstream breaks any `INSERT` that relies on position.
- Columnar warehouses read and bill for every column, used or not.

📏 RULE OF THUMB

`SELECT *` is fine for exploring at the prompt. In any query that is saved, scheduled or shared, name your columns.

✓ ROBUST

```
1 SELECT
2     order_id,
3     customer_id,
4     order_date,
5     amount
6 FROM orders;
```

- The query is a **contract**: it states exactly what it needs.
- Schema changes fail loudly here, not three tables later.
- Only four columns are scanned.

Computing new columns

```
1 SELECT
2     product_name,
3     list_price,
4     ROUND(list_price * 1.20, 2) AS price_inc_vat,
5     ROUND(list_price * 0.90, 2) AS sale_price,
6     list_price >= 50           AS is_premium
7 FROM products;
```

1 Arithmetic works row by row: `+` `-` `*` `/`. `ROUND(x, 2)` keeps two decimals.

2 A comparison is itself a value: `true` or `false`. Useful for flags.

⚡ Every computed column should get an alias, otherwise the engine invents a name.

RESULT · 6 rows

product_name	list_price	price_inc_vat	sale_price	is_premium
Laptop Stand	45.00	54.00	40.50	false
USB-C Hub	30.00	36.00	27.00	false
Mech Keyboard	90.00	108.00	81.00	true
Wireless Mouse	25.00	30.00	22.50	false
Monitor 27in	220.00	264.00	198.00	true
Webcam HD	60.00	72.00	54.00	true

DISTINCT removes duplicate rows

```
1  -- which countries do we sell to?
2  SELECT DISTINCT country
3  FROM customers
4  ORDER BY country;
5
6  -- DISTINCT applies to the whole row
7  SELECT DISTINCT country, segment
8  FROM customers
9  ORDER BY country, segment;
```

1

2

1 Six customers, four distinct countries: ES, FR, IT, UK.

2 With two columns, a row is a duplicate only if **both** values repeat. UK appears twice because it has two segments.

RESULT OF THE SECOND QUERY · 5 rows

country	segment
ES	consumer
FR	consumer
IT	business
UK	business
UK	consumer

⚠ TRAP

DISTINCT is not a bug fix

If duplicates surprise you, find out why they exist (usually a join). Hiding them with **DISTINCT** hides the cause and costs a sort.

WHERE keeps only the rows you want

```
1 SELECT order_id, order_date, status, amount
2 FROM orders
3 WHERE amount >= 100
4     AND status = 'completed';
```

- 1 The condition is tested for **each row**. Rows where it is true are kept.
- 2 **AND** requires both conditions. Text comparison is case-sensitive: **completed** is not **Completed**.

RESULT · 4 rows

order_id	order_date	status	amount
1001	2025-03-01	completed	135.00
1002	2025-03-01	completed	220.00
1005	2025-03-12	completed	310.00
1007	2025-04-03	completed	265.00

DATA ENGINEERING LENS

Filter early

Every row removed by **WHERE** is a row that later joins, sorts and aggregations never have to touch. It is the cheapest optimisation there is.

Operators you can use in WHERE

Operator	Meaning	Example	Note
<code>=</code> <code><></code> <code>!=</code>	Equal, not equal	<code>status <> 'cancelled'</code>	<code><></code> is standard; <code>!=</code> works almost everywhere.
<code><</code> <code><=</code> <code>></code> <code>>=</code>	Comparisons	<code>order_date >= DATE '2025-04-01'</code>	Works on numbers, text and dates.
<code>AND</code> <code>OR</code> <code>NOT</code>	Combine conditions	<code>country = 'UK' AND segment = 'business'</code>	<code>AND</code> binds tighter than <code>OR</code> . Use brackets.
<code>IN (...)</code>	Matches any in a list	<code>country IN ('UK', 'FR')</code>	Shorthand for several <code>OR</code> s.
<code>BETWEEN a AND b</code>	Inside a range	<code>amount BETWEEN 50 AND 100</code>	Inclusive at both ends.
<code>LIKE</code>	Text pattern	<code>email LIKE '%@mail.com'</code>	<code>%</code> any characters, <code>_</code> exactly one.
<code>IS NULL</code>	Value is missing	<code>city IS NULL</code>	Never write <code>= NULL</code> .

AND is evaluated before OR

Goal: orders that are completed or shipped, and worth more than 100.

✗ WRONG: 8 ROWS

```
1 SELECT order_id, status, amount
2 FROM orders
3 WHERE status = 'completed'
4     OR status = 'shipped'
5     AND amount > 100;
```

RESULT · 8 rows

order_id	status	amount
1001	completed	135.00
1002	completed	220.00
1003	completed	55.00
1005	completed	310.00
... 4 more rows		

Read by the engine as: completed, **or** (shipped and over 100). Every completed order gets through, even the one for 25.

✓ RIGHT: 4 ROWS

```
1 SELECT order_id, status, amount
2 FROM orders
3 WHERE (status = 'completed'
4     OR status = 'shipped')
5     AND amount > 100;
```

RESULT · 4 rows

order_id	status	amount
1001	completed	135.00
1002	completed	220.00
1005	completed	310.00
1007	completed	265.00

Rule: whenever **AND** and **OR** meet, add brackets. No error will warn you. **status IN (...)** is cleaner still.

IN and BETWEEN

```
1 SELECT order_id, order_date, status, amount
2 FROM orders
3 WHERE status IN ('completed', 'shipped')
4     AND order_date BETWEEN DATE '2025-03-01'
5                           AND DATE '2025-03-31'
6     AND amount NOT BETWEEN 100 AND 250;
```

RESULT · 3 rows

order_id	order_date	status	amount
1003	2025-03-05	completed	55.00
1005	2025-03-12	completed	310.00
1006	2025-03-15	completed	25.00

- 1 **IN** tests membership in a list. **NOT IN** is its opposite (it has a NULL trap, see Module 6).
- 2 **BETWEEN** includes both end points: 1 March and 31 March are both in.
- 3 **NOT BETWEEN** keeps values strictly outside 100 to 250.

⚠ TRAP

BETWEEN and timestamps

BETWEEN '2025-03-01' **AND** '2025-03-31' on a timestamp column stops at midnight and loses the whole of the 31st. Use **>=** '2025-03-01' **AND** **<** '2025-04-01'.

LIKE matches text patterns

```
1 SELECT full_name, email
2 FROM customers
3 WHERE full_name LIKE '%a%'           -- contains a lowercase a
4 AND email LIKE '____@%';           -- exactly 3 chars before @
```

1

2

RESULT · 3 rows

full_name	email
Ada Moreno	ada@mail.com
Ben Carter	ben@mail.com
Dev Patel	dev@mail.com

1 % matches any run of characters, including none.

2 _ matches exactly one character. Three of them means a three-letter mailbox name.

⚡ Rows with a **NULL** email are dropped: a pattern cannot match an unknown value.

DIALECT NOTE

LIKE is case-sensitive in PostgreSQL and Snowflake but not in MySQL or SQL Server by default. PostgreSQL and Snowflake offer **ILIKE** for case-insensitive matching.

NULL means "unknown"



NULL is not zero, not an empty string and not false. It is the absence of a value.

Any comparison with an unknown gives an unknown answer. SQL therefore has three truth values: TRUE, FALSE and NULL. A **WHERE** clause keeps a row only when the result is TRUE.

COMPARISON

NULL = NULL is NULL

Two unknowns are not known to be equal, so the answer is unknown, not true.

ARITHMETIC

5 + NULL is NULL

One unknown input makes the whole expression unknown.

FILTERS

Unknown is treated as "no"

Rows whose condition evaluates to NULL are silently dropped.

NULL silently drops rows from filters

Goal: every customer who is not in Leeds. Dev Patel has no city on file. Is Dev in Leeds?

✗ LOSES DEV

```
1 SELECT full_name, city
2 FROM customers
3 WHERE city <> 'Leeds';
```

RESULT · 3 rows

full_name	city
Ada Moreno	Madrid
Chloe Dubois	Lyon
Elena Rossi	Milan

For Dev, `NULL <> 'Leeds'` is NULL, so the row is dropped. No error, no warning.

✓ KEEPS DEV

```
1 SELECT full_name, city
2 FROM customers
3 WHERE city <> 'Leeds'
4 OR city IS NULL;
```

RESULT · 4 rows

full_name	city
Ada Moreno	Madrid
Chloe Dubois	Lyon
Dev Patel	NULL
Elena Rossi	Milan

Test for NULL explicitly. Shorthand: `city IS DISTINCT FROM 'Leeds'`. Before any filter on a nullable column, decide what should happen to the missing values.

Replacing NULLs: COALESCE and NULLIF

```
1 SELECT
2     full_name,
3     COALESCE(city, 'Unknown')      AS city_clean,      1
4     COALESCE(email, 'no email')    AS email_clean,
5     NULLIF(segment, 'consumer')    AS only_business    2
6 FROM customers;
```

RESULT · 6 rows

full_name	city_clean	email_clean	only_business
Ada Moreno	Madrid	ada@mail.com	NULL
Ben Carter	Leeds	ben@mail.com	business
Chloe Dubois	Lyon	chloe@mail.com	NULL
Dev Patel	Unknown	dev@mail.com	NULL
Elena Rossi	Milan	no email	business
Farid Khan	Leeds	farid@mail.com	NULL

- 1 **COALESCE**(a, b, ...) returns the **first argument** that is not NULL.
- 2 **NULLIF**(a, b) returns NULL when a equals b, otherwise a. Its classic use: **x / NULLIF(y, 0)** avoids division by zero.

★ PRO TIP

Replace NULLs at the edge (in the final report), not in the middle of a pipeline. NULL carries information: "we do not know".

ORDER BY sorts the result

```
1 SELECT full_name, country, city, signup_date
2 FROM customers
3 ORDER BY country ASC,
4          signup_date DESC;
```

1

2

RESULT · 6 rows

full_name	country	city	signup_date
Ada Moreno	ES	Madrid	2025-01-15
Chloe Dubois	FR	Lyon	2025-02-20
Elena Rossi	IT	Milan	2025-04-02
Farid Khan	UK	Leeds	2025-04-18
Dev Patel	UK	NULL	2025-03-11
Ben Carter	UK	Leeds	2025-02-03

1

Sort by country A to Z. **ASC** is the default and can be omitted.

2

Ties are broken by the next key: within UK, newest signup first.



To control where missing values go, add **NULLS FIRST** or **NULLS LAST**. Engines disagree on the default.

⚠ TRAP

No ORDER BY, no order

A table is an unordered set. Without **ORDER BY** the engine may return rows in any order, and that order can change between runs.

LIMIT returns the first N rows

```
1  -- the three largest orders
2  SELECT order_id, customer_id, amount
3  FROM orders
4  ORDER BY amount DESC, order_id
5  LIMIT 3;
```

1

2

- 1 Sort first, then cut. `order_id` is a **tiebreaker** so the result is the same on every run.
- 2 **LIMIT 3 OFFSET 3** would skip three rows and return the next three.

RESULT · 3 rows

order_id	customer_id	amount
1005	2	310.00
1007	5	265.00
1002	2	220.00

**DIALECT NOTE**

SQL Server writes **SELECT TOP 3 . . .**. The SQL standard and Oracle write **FETCH FIRST 3 ROWS ONLY**. PostgreSQL, MySQL, Snowflake, BigQuery and DuckDB use **LIMIT**.

The order a query really runs in

You write SELECT first, but the engine evaluates it fifth. This one fact explains most beginner errors.

YOU WRITE

SELECT

FROM ... JOIN

WHERE

GROUP BY

HAVING

ORDER BY

LIMIT

THE ENGINE RUNS

1 FROM + JOIN

Build the full set of candidate rows.

2 WHERE

Filter rows. Aliases and aggregates do not exist yet.

3 GROUP BY

Collapse rows into groups.

4 HAVING

Filter groups, using aggregates.

5 SELECT

Compute expressions, window functions and aliases.

6 DISTINCT

Remove duplicate output rows.

7 ORDER BY

Sort. Aliases from SELECT are visible here.

8 LIMIT

Keep the first N rows.

Why an alias fails in WHERE

WHERE runs at step 2. The alias is created at step 5. At filter time it does not exist yet.

✗ ERROR

```
1 SELECT
2     order_id,
3     amount * 0.20 AS vat
4 FROM orders
5 WHERE vat > 50;
```

PostgreSQL replies: **column "vat" does not exist**. The engine is right: at step 2 there is no such column.

✓ WORKS

```
1 SELECT
2     order_id,
3     amount * 0.20 AS vat
4 FROM orders
5 WHERE amount * 0.20 > 50;
```

RESULT · 2 rows

order_id	vat
1005	62.00
1007	53.00

Repeat the expression, or compute it in a CTE first (Module 6). A few engines relax this rule; portable SQL does not rely on it.

Exercise: recent big orders

YOUR TURN

List the **three most recent completed orders worth more than 50**. Show the order id, the date, the amount, and the amount including 20% VAT rounded to two decimals.

TABLES YOU NEED

orders

Hint 1

Two conditions joined with **AND**.
Sort before you limit.

Hint 2

Multiply by **1.20** inside
ROUND (... , 2) and give it an alias.

EXPECTED OUTPUT

order_id	order_date	amount	amount_inc_vat
1010	2025-04-20	75.00	90.00
1009	2025-04-12	90.00	108.00
1007	2025-04-03	265.00	318.00

Solution on the next slide. Try first.

Solution: recent big orders

```
1 SELECT
2     order_id,
3     order_date,
4     amount,
5     ROUND(amount * 1.20, 2) AS amount_inc_vat
6 FROM orders
7 WHERE status = 'completed'
8     AND amount > 50
9 ORDER BY order_date DESC
10 LIMIT 3;
```

- 1 Filter first: only completed orders above 50 survive.
- 2 Newest first. If two orders could share a date, add `order_id` as a tiebreaker.
- 3 Cut to three rows only after sorting.

RESULT · 3 rows

order_id	order_date	amount	amount_inc_vat
1010	2025-04-20	75.00	90.00
1009	2025-04-12	90.00	108.00
1007	2025-04-03	265.00	318.00

Module 2 recap

- ✓ Clauses are written **SELECT**, **FROM**, **WHERE**, **ORDER BY**, **LIMIT**, but executed starting from **FROM**.
- ✓ Name your columns; keep **SELECT *** for exploration only.
- ✓ Mix **AND** and **OR** only with brackets.
- ✓ **BETWEEN** is inclusive; prefer **>=** and **<** for timestamps.
- ✓ NULL means unknown. Use **IS NULL**, never **= NULL**, and decide what filters should do with missing values.
- ✓ Without **ORDER BY** there is no order. Add a tiebreaker for deterministic results.

 NEXT UP

Module 3: data types and the functions that work on them



MODULE 03

Data types and functions

Every column has a type. Types decide what is valid, how values compare, and which functions apply.

BY THE END YOU CAN

- 1 Choose the right type for numbers, text, dates and flags
- 2 Avoid the classic money and integer-division bugs
- 3 Clean and split text with string functions
- 4 Truncate, extract and shift dates
- 5 Convert between types and branch with CASE

The data types you will use every day

Pick the narrowest type that honestly describes the data. The database then rejects bad values for you.

Family	Types	Use for	Watch out for
Whole numbers	INTEGER, BIGINT	Ids, counts, quantities	INTEGER tops out near 2.1 billion. Use BIGINT for ids in large tables.
Exact decimals	NUMERIC(12,2)	Money, rates, anything audited	12 digits in total, 2 after the point. Never store money as a float.
Approximate	REAL, DOUBLE PRECISION	Scientific measurements, ML features	Binary floating point: 0.1 + 0.2 is not exactly 0.3.
Text	VARCHAR(n), TEXT	Names, codes, free text	Trailing spaces and letter case make "equal" values unequal.
Boolean	BOOLEAN	Flags such as is_active	Three states: true, false and NULL.
Dates and times	DATE, TIMESTAMP, TIMESTAMPTZ	Business dates, event times	A timestamp without a time zone is ambiguous. Prefer TIMESTAMPTZ.
Semi-structured	JSON / JSONB, VARIANT, ARRAY	API payloads, event properties	Flexible to load, slow and risky to query raw. Extract typed columns.

Never store money in a float

Floating-point types store binary approximations. Most decimal fractions cannot be represented exactly.

✗ FLOAT: APPROXIMATE

```
1 SELECT
2     0.1::float8 + 0.2::float8 AS total,
3     0.1::float8 + 0.2::float8
4     = 0.3::float8           AS is_equal;
```

RESULT IN POSTGRESQL

total	is_equal
0.30000000000000004	false

Tiny errors accumulate over millions of rows, and equality checks between "identical" totals fail.

✓ NUMERIC: EXACT

```
1 SELECT
2     0.1::numeric + 0.2::numeric AS total,
3     0.1::numeric + 0.2::numeric
4     = 0.3::numeric           AS is_equal;
```

RESULT IN POSTGRESQL

total	is_equal
0.3	true

NUMERIC (also called **DECIMAL**) stores base-10 digits exactly. When finance reconciles your warehouse total with the ledger, the two must agree to the penny.

Integer division throws away the remainder

```
1 SELECT
2     7 / 2                AS int_div,
3     7 / 2.0              AS dec_div,
4     7 % 2                AS remainder,
5     ROUND(7 / 2.0, 1)    AS rounded;
```

- 1 Integer divided by integer gives an **integer**. The .5 is discarded, not rounded.
- 2 Make either side a decimal and you get decimal division.
- 3 % is the remainder (modulo). Handy for bucketing and sampling.

RESULT IN POSTGRESQL

int_div	dec_div	remainder	rounded
3	3.5000000000000000	1	3.5

⚠ TRAP

Ratios that are always zero

`completed_orders / total_orders` with two integer counts returns 0 for every row below 100%. Write `completed_orders * 1.0 / total_orders`.

The same division in four engines

This is one of the few places where engines disagree about the answer, not only the spelling.

Engine	SELECT 7 / 2	To force the other behaviour
PostgreSQL, SQL Server	3	7 / 2.0 or CAST(7 AS NUMERIC) / 2
MySQL	3.5000	7 DIV 2
Snowflake	3.500000	TRUNC(7 / 2)
BigQuery, DuckDB	3.5	DIV(7, 2) (DuckDB: 7 // 2)

★ PRO TIP

When you migrate a pipeline between engines, test arithmetic on integers first. It fails silently and only the numbers change.

String functions: measure, change case, split

```
1 SELECT
2     full_name,
3     UPPER(full_name)           AS shouting,
4     LENGTH(full_name)         AS n_chars,
5     SPLIT_PART(full_name, ' ', 1) AS first_name,
6     SPLIT_PART(email, '@', 2)   AS domain
7 FROM customers
8 WHERE customer_id <= 3;
```

- 1 **UPPER** and **LOWER** change case. Use them to compare text reliably.
- 2 **SPLIT_PART(text, delimiter, n)** cuts on the delimiter and returns piece number n, counting from 1.
- 3 Everything after the **@** is piece 2: the email domain.

RESULT · 3 rows

full_name	shouting	n_chars	first_name	domain
Ada Moreno	ADA MORENO	10	Ada	mail.com
Ben Carter	BEN CARTER	10	Ben	mail.com
Chloe Dubois	CHLOE DUBOIS	12	Chloe	mail.com

String functions: clean and glue

```
1 SELECT
2     TRIM('  Ada Moreno ')           AS trimmed,
3     LOWER('ADA@Mail.COM')          AS normalised,
4     REPLACE('2025/03/01', '/', '-') AS fixed,
5     'ID-' || 42                     AS glued,
6     'x' || NULL                     AS null_wins;
```

RESULT · 1 row

trimmed	normalised	fixed	glued	null_wins
Ada Moreno	ada@mail.com	2025-03-01	ID-42	NULL

- 1 **TRIM** removes leading and trailing spaces, the most common cause of joins that "should" match.
- 2 **||** concatenates. Numbers are converted to text.
- 3 Concatenating NULL gives NULL. Wrap parts in **COALESCE(x, '')**, or use **CONCAT**, which skips NULLs in PostgreSQL.

⚙️ DATA ENGINEERING LENS

Normalise keys before joining

LOWER(TRIM(email)) in the staging layer saves you from "Ada@Mail.com " and "ada@mail.com" being two different customers.

Three ways to store a moment in time

Most date bugs come from not knowing which of these three a column really holds.

DATE

A calendar day

2025-03-01

No time, no zone. Right for business dates: order date, invoice date, birthday.

TIMESTAMP

A wall-clock reading

2025-03-01 09:00:00

No zone attached. Nine o'clock where? London and Tokyo would disagree by nine hours.

TIMESTAMPZ

A real instant

2025-03-01 09:00:00+00

An unambiguous point in time, stored in UTC and shown in the session time zone.

DATA ENGINEERING LENS

Store UTC, convert at the edge

Keep event times as UTC instants throughout the pipeline. Convert to local time only in the final reporting layer, where "which day did this happen" has a business answer.

Date functions: extract and truncate

```
1 SELECT
2     order_id,
3     order_date,
4     EXTRACT(year FROM order_date) AS yr,
5     EXTRACT(dow FROM order_date) AS dow,
6     DATE_TRUNC('month', order_date)::date AS month_start,
7     order_date + 7 AS due_date
8 FROM orders
9 WHERE order_id IN (1001, 1005, 1009);
```

1

2

3

4

RESULT · 3 rows

order_id	order_date	yr	dow	month_start	due_date
1001	2025-03-01	2025	6	2025-03-01	2025-03-08
1005	2025-03-12	2025	3	2025-03-01	2025-03-19
1009	2025-04-12	2025	6	2025-04-01	2025-04-19

- 1 **EXTRACT** pulls one part out of a date: year, month, day, hour.
- 2 Day of week. In PostgreSQL 0 is Sunday, so 6 is Saturday.
- 3 **DATE_TRUNC** rounds **down** to the start of a period. It is how you group by month or week.
- 4 Adding an integer to a **DATE** adds days.

Date arithmetic: differences and intervals

```
1 SELECT
2     full_name,
3     signup_date,
4     DATE '2025-05-01' - signup_date      AS days_since,
5     (signup_date + INTERVAL '1 month')::date AS one_month_on,
6     signup_date >= DATE '2025-03-01'    AS is_recent
7 FROM customers
8 WHERE customer_id <= 4;
```

RESULT · 4 rows

full_name	signup_date	days_since	one_month_on	is_recent
Ada Moreno	2025-01-15	106	2025-02-15	false
Ben Carter	2025-02-03	87	2025-03-03	false
Chloe Dubois	2025-02-20	70	2025-03-20	false
Dev Patel	2025-03-11	51	2025-04-11	true

- 1 Subtracting two dates gives a whole number of **days**.
- 2 An **INTERVAL** is a duration. Adding one to a date returns a timestamp, so we cast back to **date**.
- 3 Dates compare chronologically. Always write literals in ISO order: year, month, day.

★ PRO TIP

In pipelines, pass the run date in as a parameter instead of calling **CURRENT_DATE**. A re-run next week must produce the same result.

Date functions across dialects

Date handling is the least portable corner of SQL. The same two jobs in three engines: round down to the month, and count days between dates.

POSTGRESQL · SNOWFLAKE

```
-- month start
DATE_TRUNC('month', order_date)

-- days between
-- PostgreSQL:
end_date - start_date
-- Snowflake:
DATEDIFF('day', start_date,
         end_date)
```

Snowflake shares **DATE_TRUNC** with PostgreSQL but uses **DATEDIFF** for differences.

BIGQUERY

```
-- month start
DATE_TRUNC(order_date, MONTH)

-- days between
DATE_DIFF(end_date,
          start_date, DAY)
```

Arguments are reversed: the date comes first, and the **later** date comes first in **DATE_DIFF**.

SQL SERVER

```
-- month start (2022+)
DATETRUNC(month, order_date)

-- days between
DATEDIFF(day, start_date,
         end_date)
```

The unit is a bare keyword, not a string. Older versions lack **DATETRUNC**.

CAST converts between types

```
1 SELECT
2     CAST('2025-03-01' AS DATE)      AS as_date,
3     '42'::INTEGER + 1                AS as_int,
4     CAST(19.99 AS INTEGER)           AS as_whole,
5     CAST(1001 AS VARCHAR) || '-A'   AS as_text;
```

- 1 `CAST(value AS type)` is the standard form and works everywhere.
- 2 `value::type` is PostgreSQL shorthand, also accepted by Snowflake and DuckDB.
- 3 PostgreSQL **rounds** to 20. SQL Server truncates to 19. Use `ROUND`, `FLOOR` or `CEIL` to say what you mean.

RESULT · 1 row

as_date	as_int	as_whole	as_text
2025-03-01	43	20	1001-A

DATA ENGINEERING LENS

One bad value fails the whole load

`CAST('n/a' AS INTEGER)` raises an error and stops the query. Safe variants return NULL instead: `TRY_CAST` (Snowflake, SQL Server, DuckDB) and `SAFE_CAST` (BigQuery). PostgreSQL has none; clean with `CASE` first.

CASE: if / else inside a query

```
1 SELECT
2     order_id,
3     amount,
4     CASE
5         WHEN amount >= 250 THEN 'large'
6         WHEN amount >= 100 THEN 'medium'
7         ELSE 'small'
8     END AS order_size
9 FROM orders
10 ORDER BY amount DESC
11 LIMIT 5;
```

- 1 Conditions are tested **top to bottom**. The first one that is true wins.
- 2 An order of 310 never reaches this line, so there is no need to write `amount < 250`.
- 3 Without `ELSE`, unmatched rows get NULL. Always decide the default on purpose.
- 4 `CASE` is an expression: it returns one value and can be used anywhere a column can.

RESULT · 5 rows

order_id	amount	order_size
1005	310.00	large
1007	265.00	large
1002	220.00	medium
1001	135.00	medium
1004	90.00	small

Reading JSON: a first look

```
1 SELECT
2     event_id,
3     payload ->> 'type'                AS event_type,
4     (payload ->> 'user_id')::INTEGER AS user_id,
5     payload -> 'props' ->> 'page'     AS page
6 FROM raw_events;
```

- 1 `->>` extracts a field **as text**.
- 2 Text is not a number. Cast it before you do arithmetic or joins.
- 3 `->` keeps the result as JSON so you can drill further. A missing key gives NULL, not an error.

RESULT · 3 rows

event_id	event_type	user_id	page
1	page_view	1	/home
2	add_to_cart	2	NULL
3	purchase	1	NULL

DATA ENGINEERING LENS

Raw events arrive as one JSON column. The first job of a pipeline is turning them into typed columns like these. Module 14 goes deeper.

Exercise: a clean contact sheet

YOUR TURN

For every customer return: the **first name in capitals**, the **email domain** (or **unknown** when there is no email), the **first day of the signup month**, and a cohort label: **early** if they signed up before 1 March 2025, otherwise **recent**.

TABLES YOU NEED

customers

Hint 1

SPLIT_PART twice: once on a space, once on @. Wrap the second in **COALESCE**.

Hint 2

DATE_TRUNC for the month, **CASE** for the cohort.

EXPECTED OUTPUT

first_name	email_domain	signup_month	cohort
ADA	mail.com	2025-01-01	early
BEN	mail.com	2025-02-01	early
CHLOE	mail.com	2025-02-01	early
DEV	mail.com	2025-03-01	recent
ELENA	unknown	2025-04-01	recent
FARID	mail.com	2025-04-01	recent

Solution on the next slide. Try first.

Solution: a clean contact sheet

```
1  SELECT
2      UPPER(SPLIT_PART(full_name, ' ', 1))           AS first_name,
3      COALESCE(SPLIT_PART(email, '@', 2), 'unknown') AS email_domain,
4      DATE_TRUNC('month', signup_date)::date         AS signup_month,
5      CASE
6          WHEN signup_date < DATE '2025-03-01' THEN 'early'
7          ELSE 'recent'
8      END                                             AS cohort
9  FROM customers
10 ORDER BY customer_id;
```

- 1 Functions nest: split first, then uppercase the piece.
- 2 A NULL email makes **SPLIT_PART** return NULL, which **COALESCE** replaces.
- 3 The date literal is typed with **DATE** so the comparison is date to date, not date to text.

★ PRO TIP

Aligning the **AS** aliases in a column makes a long select list scannable. Formatting is part of correctness when someone else reviews it.

Module 3 recap

- ✓ Use **NUMERIC** for money; floats are approximations.
- ✓ Integer / integer discards the remainder in PostgreSQL and SQL Server.
- ✓ **TRIM**, **LOWER**, **REPLACE** and **SPLIT_PART** do most text cleaning.
- ✓ **DATE_TRUNC** rounds down to a period; date minus date gives days.
- ✓ Prefer **TIMESTAMPTZ** and keep everything in UTC until the reporting layer.
- ✓ **CAST** fails on bad input; use the safe variant your engine offers.
- ✓ **CASE** tests top to bottom and should always have an **ELSE**.
- ✓ JSON fields come out as text: cast them.

 NEXT UP

Module 4: summarising many rows with aggregates and GROUP BY



MODULE 04

Aggregation

Turn thousands of rows into the handful of numbers a business actually reads.

BY THE END YOU CAN

- 1 Summarise a table with COUNT, SUM, AVG, MIN and MAX
- 2 Predict how aggregates treat NULL
- 3 Group rows and compute one result per group
- 4 Filter groups with HAVING
- 5 Build several metrics in one pass with conditional aggregation

Aggregates collapse many rows into one

```
1 SELECT
2     COUNT (*)           AS n_orders,
3     SUM (amount)        AS revenue,
4     ROUND (AVG (amount), 2) AS avg_order,
5     MIN (amount)        AS smallest,
6     MAX (amount)        AS largest
7 FROM orders;
```

- 1 **COUNT (*)** counts rows. Ten orders go in, **one** summary row comes out.
- 2 **SUM** adds a numeric column; **AVG** is the sum divided by the count.
- 3 **MIN** and **MAX** also work on dates and text: first order date, last name alphabetically.

RESULT · 1 row

n_orders	revenue	avg_order	smallest	largest
10	1325.00	132.50	25.00	310.00

MENTAL MODEL

A funnel, not a filter

A filter removes rows but keeps their shape. An aggregate consumes all the rows and returns a single value per function.

Three ways to count

```
1 SELECT
2     COUNT(*)                AS all_rows,      1
3     COUNT(email)           AS with_email,     2
4     COUNT(city)            AS with_city,
5     COUNT(DISTINCT country) AS countries,     3
6     COUNT(DISTINCT city)    AS cities
7 FROM customers;
```

- 1 `COUNT (*)` counts rows, whatever they contain.
- 2 `COUNT(column)` counts rows where that column is **not NULL**. Elena has no email, so 5.
- 3 `COUNT(DISTINCT column)` counts different values. NULL is not a value, so it is not counted.

RESULT · 1 row

all_rows	with_email	with_city	countries	cities
6	5	5	4	4

DATA ENGINEERING LENS

Your first data profile

Comparing `COUNT (*)` with `COUNT(col)` and `COUNT(DISTINCT col)` tells you how complete and how unique a column is. Run it on every new source table.

Aggregates ignore NULL

```
1 SELECT
2     COUNT (*)           AS n_rows,
3     COUNT (x)           AS n_values,
4     SUM (x)              AS total,
5     AVG (x)              AS avg_skip_nulls,
6     AVG (COALESCE (x, 0)) AS avg_null_as_zero
7 FROM (VALUES (10), (20), (NULL)) AS t(x);
```

1
2
3

RESULT · 1 row

n_rows	n_values	total	avg_skip_nulls	avg_null_as_zero
3	2	30	15.00	10.00

- 1 **AVG** divides by the number of **non-NULL** values: $30 / 2 = 15$.
- 2 Treating the missing value as zero gives $30 / 3 = 10$. Neither is "correct"; it depends what NULL means in your data.
- 3 **VALUES** builds a small inline table, perfect for experiments like this one.

 TRAP

SUM over zero rows (or over only NULLs) returns NULL, not 0.
Wrap it: **COALESCE (SUM (x) , 0)**.

GROUP BY: split, apply, combine

1 SPLIT ROWS BY KEY

orders

order_id	customer_id	amount
1001	1	135.00
1003	1	55.00
1009	1	90.00
1002	2	220.00
1005	2	310.00
1008	2	60.00
1004	3	90.00
1010	3	75.00

2 APPLY THE AGGREGATE PER GROUP

`customer_id = 1`

`135 + 55 + 90`

3 rows

`customer_id = 2`

`220 + 310 + 60`

3 rows

`customer_id = 3`

`90 + 75`

2 rows

3 COMBINE: ONE ROW PER GROUP

result

customer_id	n_orders	total
1	3	280.00
2	3	590.00
3	2	165.00

MENTAL MODEL

One output row per distinct value of the key

The engine sorts or hashes rows into buckets by `customer_id`, runs the aggregates inside each bucket, and returns one row per bucket.

GROUP BY in SQL

```
1 SELECT
2     customer_id,
3     COUNT(*)      AS n_orders,
4     SUM(amount)    AS total_spent,
5     MAX(order_date) AS last_order
6 FROM orders
7 GROUP BY customer_id
8 ORDER BY total_spent DESC;
```

RESULT · 5 rows

customer_id	n_orders	total_spent	last_order
2	3	590.00	2025-04-10
1	3	280.00	2025-04-12
5	1	265.00	2025-04-03
3	2	165.00	2025-04-20
4	1	25.00	2025-03-15

- 1 The grouping column appears in **SELECT** so you can tell the rows apart.
- 2 Every other column must be inside an aggregate.
- 3 One group, and so one output row, per distinct **customer_id**.
- 4 **ORDER BY** runs after grouping, so it can use the alias.

DATA ENGINEERING LENS

Customer 6 is missing: no orders, no group. Reports that need the zero rows start from **customers** and use a **LEFT JOIN** (Module 5).

Grouping by several columns and expressions

```
1 SELECT
2     DATE_TRUNC('month', order_date)::date AS month,
3     status,
4     COUNT(*) AS n_orders,
5     SUM(amount) AS revenue
6 FROM orders
7 GROUP BY 1, 2
8 ORDER BY 1, 2;
```

1

2

RESULT · 4 rows

month	status	n_orders	revenue
2025-03-01	cancelled	1	90.00
2025-03-01	completed	5	745.00
2025-04-01	completed	3	430.00
2025-04-01	shipped	1	60.00

- 1 You can group by an expression. Here every date is first rounded down to its month.
 - 2 **GROUP BY 1, 2** means "the first and second select columns". Convenient, but re-check it whenever you reorder the select list.
-  A group is a unique **combination**: March + cancelled, March + completed, and so on.

RULE OF THUMB

The grain of the output

Whatever you put in **GROUP BY** defines what one row means. Here: one row per month per status.

WHERE filters rows, HAVING filters groups

```
1 SELECT
2     customer_id,
3     SUM(amount) AS total
4 FROM orders
5 WHERE status = 'completed'      -- before grouping 1
6 GROUP BY customer_id
7 HAVING SUM(amount) > 200        -- after grouping 2
8 ORDER BY total DESC;
```

RESULT · 3 rows

customer_id	total
2	530.00
1	280.00
5	265.00

- 1 **WHERE** sees individual orders. It cannot use **SUM**, because no groups exist yet.
 - 2 **HAVING** sees one row per customer and can test aggregates.
- ⚡ Customers 3 and 4 completed orders worth 75 and 25. Their groups are built, then discarded.

★ PRO TIP

A condition on a plain column belongs in **WHERE**, even if **HAVING** would accept it. Fewer rows reach the expensive grouping step.

Several metrics in one pass

```
1 SELECT
2     customer_id,
3     COUNT(*)                               AS all_orders,
4     COUNT(*) FILTER (WHERE status = 'completed') AS completed,
5     SUM(CASE WHEN status = 'cancelled'
6         THEN amount ELSE 0 END)           AS cancelled_value
7 FROM orders
8 GROUP BY customer_id
9 ORDER BY customer_id;
```

RESULT · 5 rows

customer_id	all_orders	completed	cancelled_value
1	3	3	0.00
2	3	2	0.00
3	2	1	90.00
4	1	1	0.00
5	1	1	0.00

- 1 **FILTER** (**WHERE** ...) restricts **one** aggregate without touching the others. PostgreSQL, DuckDB, SQLite.
- 2 The portable form: **CASE** returns the amount for matching rows and 0 for the rest, then **SUM** adds them.

DATA ENGINEERING LENS

Conditional aggregation replaces many queries

Instead of scanning the table once per metric and joining the results, scan once and compute every metric side by side. This is also how you pivot rows into columns.

Every selected column must be grouped

After grouping there is one row per customer. Which of that customer's several statuses should the engine show?

✗ ERROR

```
1 SELECT
2   customer_id,
3   status,
4   SUM(amount) AS total
5 FROM orders
6 GROUP BY customer_id;
```

PostgreSQL: column `"orders.status"` must appear in the **GROUP BY** clause or be used in an aggregate function.

✓ TWO HONEST FIXES

```
1 -- a) make status part of the grain
2 GROUP BY customer_id, status
3
4 -- b) or reduce it to one value
5 SELECT customer_id,
6         MAX(status) AS a_status,
7         SUM(amount) AS total
```

Either group by it (more rows) or aggregate it (one value). The choice is a business decision.

🌐 DIALECT NOTE

Old MySQL configurations accept the broken query and return an arbitrary status. That is worse than an error: wrong data with no warning.

Subtotals with ROLLUP

```
1 SELECT
2     COALESCE(status, 'ALL STATUSES') AS status,
3     COUNT(*) AS n_orders,
4     SUM(amount) AS revenue
5 FROM orders
6 GROUP BY ROLLUP (status)
7 ORDER BY revenue;
```

2

1

RESULT · 4 rows

status	n_orders	revenue
shipped	1	60.00
cancelled	1	90.00
completed	8	1175.00
ALL STATUSES	10	1325.00

- 1 **ROLLUP** adds an extra row that aggregates across all statuses: the grand total.
 - 2 In the total row the grouping column is NULL, so we label it.
- ⚡ **ROLLUP** (a, b) gives a-and-b rows, a subtotals and a grand total. **CUBE** (a, b) gives every combination. **GROUPING SETS** lets you list exactly the ones you want.

DATA ENGINEERING LENS

One scan produces detail and totals together, so the numbers on a dashboard cannot disagree with each other.

Aggregating text: STRING_AGG

```
1 SELECT
2     category,
3     COUNT(*) AS n_products,
4     STRING_AGG(product_name, ', '
5                 ORDER BY product_name) AS products
6 FROM products
7 GROUP BY category
8 ORDER BY category;
```

1

2

RESULT · 3 rows

category	n_products	products
accessories	2	Laptop Stand, USB-C Hub
displays	1	Monitor 27in
peripherals	3	Mech Keyboard, Webcam HD, Wireless Mouse

- 1 `STRING_AGG(value, separator)` glues the values of a group into one string.
- 2 An `ORDER BY` inside the aggregate fixes the order of the pieces.



DIALECT NOTE

Snowflake and SQL Server: `LISTAGG` or `STRING_AGG ... WITHIN GROUP (ORDER BY ...)`. MySQL: `GROUP_CONCAT`. `ARRAY_AGG` collects into an array instead.

Exercise: revenue by status

YOUR TURN

For each order status, return the **number of orders**, the **total amount** and the **average amount** rounded to two decimals. Keep only statuses whose total is **above 80**, largest total first.

TABLES YOU NEED

orders

Hint 1

Group by **status**. Three aggregates in the select list.

Hint 2

The "above 80" test is on a total, so it cannot go in **WHERE**.

EXPECTED OUTPUT

status	n_orders	total	avg_amount
completed	8	1175.00	146.88
cancelled	1	90.00	90.00

Solution on the next slide. Try first.

Solution: revenue by status

```
1 SELECT
2     status,
3     COUNT(*)           AS n_orders,
4     SUM(amount)        AS total,
5     ROUND(AVG(amount), 2) AS avg_amount
6 FROM orders
7 GROUP BY status
8 HAVING SUM(amount) > 80
9 ORDER BY total DESC;
```

3

1

2

- 1 Three statuses exist, so three groups are formed.
- 2 The shipped group totals 60 and is removed here, after aggregation.
- 3 $1175 / 8 = 146.875$, shown as 146.88.

RESULT · 2 rows

status	n_orders	total	avg_amount
completed	8	1175.00	146.88
cancelled	1	90.00	90.00

Module 4 recap

- ✓ Aggregates turn many rows into one value: **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**.
- ✓ **COUNT (*)** counts rows; **COUNT (col)** skips NULLs; **COUNT (DISTINCT col)** counts unique values.
- ✓ All aggregates except **COUNT (*)** ignore NULL. Decide whether that is what you want.
- ✓ **GROUP BY** defines the grain: one output row per combination of grouping values.
- ✓ **WHERE** filters rows before grouping; **HAVING** filters groups after.
- ✓ **FILTER** or **SUM (CASE ...)** computes several conditional metrics in one scan.

 NEXT UP**Module 5: combining tables with joins**



MODULE 05

Joins

Real answers live across several tables. Joins bring them together, and are where most wrong numbers are born.

BY THE END YOU CAN

- 1 Explain primary and foreign keys
- 2 Choose between INNER, LEFT, RIGHT and FULL joins
- 3 Find missing data with anti-joins
- 4 Detect and prevent join fan-out
- 5 Place filters correctly in ON versus WHERE

Why data is split across tables

Storing each fact once keeps it consistent. The price is that you must join to read it back together.

✗ ONE WIDE TABLE

orders_wide

order_id	full_name	city	amount
1001	Ada Moreno	Madrid	135.00
1003	Ada Moreno	Madrid	55.00
1009	Ada Moreno	Madrid	90.00
1002	Ben Carter	Leeds	220.00

- Ada's city is stored **three times**. If she moves, three rows must change.
- Miss one and the table contradicts itself.
- A customer with no orders cannot be stored at all.

✓ TWO LINKED TABLES

customers

customer_id	full_name	city
1	Ada Moreno	Madrid
2	Ben Carter	Leeds

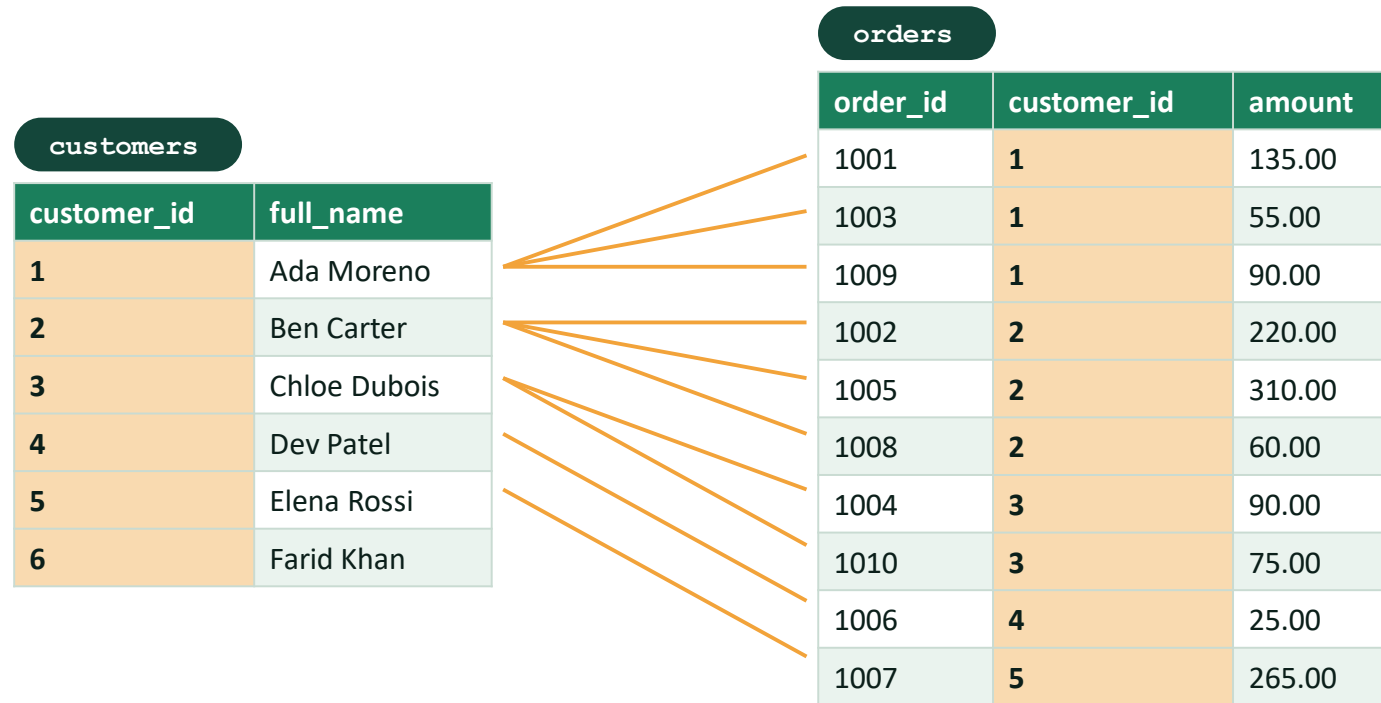
orders

order_id	customer_id	amount
1001	1	135.00
1003	1	55.00
1009	1	90.00
1002	2	220.00

- Each fact lives in **one place**. Orders carry only the customer's id.

Keys connect the rows

A join pairs each row on one side with the rows on the other side that share its key value.



Primary key (PK). `customers.customer_id` is unique: each value identifies one customer.

Foreign key (FK). `orders.customer_id` refers to that primary key. It may repeat: Ada has three orders, so three lines leave her row.

No line leaves Farid. He has no orders. Whether he appears in the result depends on the join type you choose.

INNER JOIN keeps matching pairs

```
1 SELECT
2     o.order_id,
3     o.order_date,
4     c.full_name,
5     o.amount
6 FROM orders AS o
7 INNER JOIN customers AS c
8     ON c.customer_id = o.customer_id
9 WHERE o.order_date >= DATE '2025-04-01'
10 ORDER BY o.order_id;
```

- 1 **AS** `o` gives the table a short **alias**. Prefix every column with it so nobody has to guess where it comes from.
- 2 **INNER JOIN** (or simply **JOIN**) returns a row only when both sides match.
- 3 **ON** states the matching rule: the foreign key equals the primary key.
- 4 Filters apply to the joined rows.

RESULT · 4 rows

order_id	order_date	full_name	amount
1007	2025-04-03	Elena Rossi	265.00
1008	2025-04-10	Ben Carter	60.00
1009	2025-04-12	Ada Moreno	90.00
1010	2025-04-20	Chloe Dubois	75.00

LEFT JOIN keeps every row of the left table

```
1 SELECT
2     c.customer_id,
3     c.full_name,
4     COUNT(o.order_id)           AS n_orders,
5     COALESCE(SUM(o.amount), 0) AS total_spent
6 FROM customers AS c
7 LEFT JOIN orders AS o
8     ON o.customer_id = c.customer_id
9 GROUP BY c.customer_id, c.full_name
10 ORDER BY c.customer_id;
```

- 1 The table after **FROM** is the **left** table. All six customers are kept.
- 2 Where no order matches, the order columns are filled with NULL.
- 3 Count a column from the right table, not *****. **COUNT(*)** would report 1 for Farid: his single NULL-filled row.
- 4 **SUM** over only NULLs is NULL, so we turn it into 0.

RESULT · 6 rows

customer_id	full_name	n_orders	total_spent
1	Ada Moreno	3	280.00
2	Ben Carter	3	590.00
3	Chloe Dubois	2	165.00
4	Dev Patel	1	25.00
5	Elena Rossi	1	265.00
6	Farid Khan	0	0.00

Four join types, one picture

Join table A (ids 1, 2, 3) to table B (ids 2, 3, 4) on id. Only the treatment of unmatched rows changes.



RULE OF THUMB

In practice you need two

INNER when a row without a match is meaningless, **LEFT** when the left table is the thing you are reporting on. **RIGHT** is a **LEFT** written backwards; **FULL** is for reconciliation.

FULL OUTER JOIN for reconciliation

```
1 SELECT
2     COALESCE(s.order_id, t.order_id) AS order_id,
3     s.order_id IS NOT NULL        AS in_source,
4     t.order_id IS NOT NULL        AS in_target
5 FROM (SELECT DISTINCT order_id
6       FROM raw_orders) AS s
7 FULL OUTER JOIN orders AS t
8     ON t.order_id = s.order_id
9 WHERE s.order_id IS NULL
10      OR t.order_id IS NULL
11 ORDER BY 1;
```

DATA ENGINEERING LENS

The reconciliation query

"What is in the source but not the warehouse, and the reverse?" is asked after every migration and every incident. This pattern answers both at once.

- 1 Keep unmatched rows from **both** sides: ids only in the source and ids only in the target.
- 2 Keep only the disagreements. Matching ids are not interesting.
- 3 One side is always NULL here, so **COALESCE** picks whichever id exists.

RESULT · 7 rows

order_id	in_source	in_target
1005	false	true
1006	false	true
1007	false	true
1008	false	true
1009	false	true
1010	false	true
1012	true	false

Anti-join: rows with no match

Which customers have never ordered? "Rows in A with no match in B" also finds unloaded rows, orphaned keys and unsold products.

LEFT JOIN + IS NULL

```
1 SELECT c.customer_id, c.full_name
2 FROM customers AS c
3 LEFT JOIN orders AS o
4     ON o.customer_id = c.customer_id
5 WHERE o.order_id IS NULL;
```

RESULT · 1 row

customer_id	full_name
6	Farid Khan

Keep everyone, then keep only the rows where the join found nothing.

NOT EXISTS

```
1 SELECT c.customer_id, c.full_name
2 FROM customers AS c
3 WHERE NOT EXISTS (
4     SELECT 1
5     FROM orders AS o
6     WHERE o.customer_id = c.customer_id
7 );
```

RESULT · 1 row

customer_id	full_name
6	Farid Khan

Reads like the question. Safe with NULLs, and optimizers handle it well.

Semi-join: EXISTS filters without duplicating

```
1 SELECT c.customer_id, c.full_name
2 FROM customers AS c
3 WHERE EXISTS (
4     SELECT 1
5     FROM orders AS o
6     WHERE o.customer_id = c.customer_id
7         AND o.amount >= 200
8 )
9 ORDER BY c.customer_id;
```

RESULT · 2 rows

customer_id	full_name
2	Ben Carter
5	Elena Rossi

- 1 **EXISTS** is true as soon as the inner query finds **one** row. It never adds columns or rows.
- 2 The inner query refers to the outer row: "an order belonging to this customer".
- 3 **SELECT 1** is a convention. The selected value is irrelevant; only existence matters.

⚠ TRAP

Why not a plain JOIN?

Ben has two orders of 200 or more. An **INNER JOIN** would list him twice, and you would be tempted to hide it with **DISTINCT**. **EXISTS** asks a yes/no question, so he appears once.

Joining more than two tables

```
1  SELECT
2      o.order_id,
3      c.full_name,
4      p.product_name,
5      oi.quantity,
6      oi.quantity * oi.unit_price AS line_total
7  FROM orders AS o
8  JOIN customers AS c ON c.customer_id = o.customer_id
9  JOIN order_items AS oi ON oi.order_id = o.order_id
10 JOIN products AS p ON p.product_id = oi.product_id
11 WHERE o.order_id IN (1001, 1008)
12 ORDER BY o.order_id, p.product_name;
```

1

2

3

- 1 Joins chain. Each one adds columns to the rows built so far.
- 2 This join changes the **grain**: one row per order becomes one row per order line.
- 3 Each **ON** links the new table to one already in the query.

RESULT · 3 rows

order_id	full_name	product_name	quantity	line_total
1001	Ada Moreno	Laptop Stand	1	45.00
1001	Ada Moreno	Mech Keyboard	1	90.00
1008	Ben Carter	USB-C Hub	2	60.00

Join fan-out silently inflates totals

True revenue is 1,325.00. A one-to-many join repeats the "one" side, so its columns are summed more than once. No error is raised.

✗ WRONG: 2,090.00

```
1 SELECT SUM(o.amount) AS revenue
2 FROM orders AS o
3 JOIN order_items AS oi
4     ON oi.order_id = o.order_id;
```

RESULT · 1 row

revenue
2090.00

Four orders have two lines each, so their **amount** is counted **twice**.

✓ RIGHT: 1,325.00

```
1 SELECT
2     SUM(oi.quantity * oi.unit_price)
3     AS revenue
4 FROM orders AS o
5 JOIN order_items AS oi
6     ON oi.order_id = o.order_id;
```

RESULT · 1 row

revenue
1325.00

Sum a measure that lives at the **line** grain, or aggregate items to one row per order before joining.

How to catch fan-out before it ships

```
1 SELECT
2     (SELECT COUNT(*) FROM orders)      AS rows_before,
3     COUNT(*)                          AS rows_after,
4     COUNT(DISTINCT o.order_id)         AS distinct_orders
5 FROM orders AS o
6 JOIN order_items AS oi
7     ON oi.order_id = o.order_id;
```

RESULT · 1 row

rows_before	rows_after	distinct_orders
10	14	10

- 1 Ten orders went in.
- 2 Fourteen rows came out. The join multiplied rows.
- 3 Still ten distinct orders, so some appear more than once.

★ PRO TIP**Count rows before and after every join**

If a join was meant to add columns but the row count grew, the right-hand key is not unique. Check with `GROUP BY key HAVING COUNT(*) > 1`.

A WHERE filter can undo a LEFT JOIN

✗ BECOMES AN INNER JOIN

```
1 SELECT c.full_name, o.order_id
2 FROM customers AS c
3 LEFT JOIN orders AS o
4   ON o.customer_id = c.customer_id
5 WHERE o.status = 'shipped';
```

RESULT · 1 row

full_name	order_id
Ben Carter	1008

Goal: every customer, with any shipped order beside them. But where nothing matched, **o.status** is NULL, and **WHERE** removes the row.

✓ KEEPS ALL CUSTOMERS

```
1 SELECT c.full_name, o.order_id
2 FROM customers AS c
3 LEFT JOIN orders AS o
4   ON o.customer_id = c.customer_id
5   AND o.status = 'shipped'
6 ORDER BY c.customer_id;
```

RESULT · 6 rows

full_name	order_id
Ada Moreno	NULL
Ben Carter	1008
Chloe Dubois	NULL
Dev Patel	NULL
... 2 more rows	

In **ON**, the condition limits which orders can match. Unmatched customers still survive.

Self join: a table joined to itself

```
1 SELECT
2     e.emp_name AS employee,
3     e.title,
4     m.emp_name AS manager
5 FROM employees AS e
6 LEFT JOIN employees AS m
7     ON m.emp_id = e.manager_id
8 ORDER BY e.emp_id;
```

1

3

2

MENTAL MODEL

Self joins answer "compare a row with another row of the same table": manager and report, this month and last month, an event and the one before it.

- 1 The same table is used twice under two aliases: **e** plays the employee, **m** the manager.
- 2 An employee's **manager_id** points at another row's **emp_id**.
- 3 **LEFT** keeps Grace, who has no manager.

RESULT · 6 rows

employee	title	manager
Grace	Head of Data	<i>NULL</i>
Hugo	Analytics Lead	Grace
Ines	Engineering Lead	Grace
Jamal	Data Engineer	Ines
Kira	Data Engineer	Ines
Liam	Analyst	Hugo

CROSS JOIN builds every combination

```
1 SELECT
2     c.full_name,
3     m.month,
4     COALESCE(SUM(o.amount), 0) AS revenue
5 FROM customers AS c
6 CROSS JOIN (VALUES (DATE '2025-03-01'),
7                  (DATE '2025-04-01')) AS m(month)
8 LEFT JOIN orders AS o
9     ON o.customer_id = c.customer_id
10    AND DATE_TRUNC('month', o.order_date)::date = m.month
11 WHERE c.customer_id >= 4
12 GROUP BY c.full_name, m.month
13 ORDER BY c.full_name, m.month;
```

DATA ENGINEERING LENS

The spine pattern. Dashboards need a row for every period, including quiet ones. Build the grid first, then left join the facts.

- 1 **CROSS JOIN** has no **ON**. It pairs every customer with every month: a complete grid.
- 2 Orders are then attached where they exist.
- 3 Empty grid cells become explicit zeros instead of missing rows.

RESULT · 6 rows

full_name	month	revenue
Dev Patel	2025-03-01	25.00
Dev Patel	2025-04-01	0.00
Elena Rossi	2025-03-01	0.00
Elena Rossi	2025-04-01	265.00
Farid Khan	2025-03-01	0.00
Farid Khan	2025-04-01	0.00

Joins on ranges, not only equality

```
1 WITH bands (lo, hi, band) AS (  
2     VALUES (0, 100, 'small'),  
3             (100, 250, 'medium'),  
4             (250, 99999, 'large')  
5 )  
6 SELECT o.order_id, o.amount, b.band  
7 FROM orders AS o  
8 JOIN bands AS b  
9     ON o.amount >= b.lo  
10    AND o.amount < b.hi  
11 ORDER BY o.amount DESC  
12 LIMIT 5;
```

DATA ENGINEERING LENS

Range joins power tax brackets, shipping tiers and, most importantly, "which version of the customer was valid on the order date" (Module 10).

- 1 **WITH** names a temporary result (a CTE, see Module 6). Here it is a small lookup table.
- 2 The **ON** condition can be any true/false test. Each order matches the one band whose range contains it.
- 3 Half-open ranges ($\geq$ low, $<$ high) guarantee no gaps and no overlaps.

RESULT · 5 rows

order_id	amount	band
1005	310.00	large
1007	265.00	large
1002	220.00	medium
1001	135.00	medium
1004	90.00	small

Exercise: revenue by product category

 YOUR TURN

For **completed** orders only, return each product **category** with the number of **units sold** and the **revenue** (quantity times unit price). Highest revenue first.

TABLES YOU NEED

orders

order_items

products

 EXPECTED OUTPUT

category	units	revenue
displays	3	660.00
peripherals	7	305.00
accessories	5	210.00

Solution on the next slide. Try first.

Hint 1

Start from **order_items**: it has the grain you need. Join outwards to the other two tables.

Hint 2

The status filter lives on **orders**. The grouping column lives on **products**.

Solution: revenue by product category

```
1 SELECT
2     p.category,
3     SUM(oi.quantity)           AS units,
4     SUM(oi.quantity * oi.unit_price) AS revenue
5 FROM order_items AS oi
6 JOIN orders   AS o ON o.order_id = oi.order_id
7 JOIN products AS p ON p.product_id = oi.product_id
8 WHERE o.status = 'completed'
9 GROUP BY p.category
10 ORDER BY revenue DESC;
```

- 1 The line table is the base, so both joins are many-to-one. No fan-out is possible.
- 2 Revenue is computed from line values. The three figures add up to 1,175.00, the completed total.
- 3 Filtering on **orders** drops the lines of cancelled and shipped orders.

★ PRO TIP

Always sanity-check a new aggregate against a number you already trust. Here: $660 + 305 + 210 = 1,175$.

Module 5 recap

- ✓ A foreign key holds the primary-key value of the row it refers to.
- ✓ **INNER JOIN** keeps matches only; **LEFT JOIN** keeps every left row and fills gaps with NULL.
- ✓ **FULL OUTER JOIN** shows what each side is missing: ideal for reconciliation.
- ✓ Use **NOT EXISTS** or **LEFT JOIN ... IS NULL** to find rows without a match.
- ✓ A one-to-many join repeats the "one" side. Count rows before and after.
- ✓ With **LEFT JOIN**, filters on the right table belong in **ON**, not **WHERE**.
- ✓ **CROSS JOIN** builds a complete grid for reports that must show zeros.
- ✓ Always alias tables and prefix columns.

 NEXT UP**Module 6: set operations, subqueries and CTEs**



MODULE 06

Set operations, subqueries and CTEs

Stack results, nest queries inside queries, and break a hard problem into named, readable steps.

BY THE END YOU CAN

- 1 Combine result sets with UNION, INTERSECT and EXCEPT
- 2 Use a query as a value, a list or a table
- 3 Avoid the NOT IN trap with NULLs
- 4 Structure long queries as a chain of CTEs
- 5 Walk hierarchies and generate rows with recursion

UNION ALL stacks, UNION also deduplicates

UNION ALL · KEEP EVERYTHING

```
1 SELECT country FROM customers
2 WHERE segment = 'consumer'
3 UNION ALL
4 SELECT country FROM customers
5 WHERE segment = 'business';
```

RESULT · 6 rows

country
ES
FR
UK
UK
UK
IT

Stacks the rows of two queries. Six customers in, six rows out. Both queries need the same number of columns, with compatible types.

UNION · REMOVE DUPLICATES

```
1 SELECT country FROM customers
2 WHERE segment = 'consumer'
3 UNION
4 SELECT country FROM customers
5 WHERE segment = 'business'
6 ORDER BY country;
```

RESULT · 4 rows

country
ES
FR
IT
UK

Sorts or hashes the whole result to drop repeats. Costly on large data. Column names come from the first query.

Default to UNION ALL



In pipelines, reach for UNION ALL. Use UNION only when you can explain why duplicates exist.

Combining monthly files, regional tables or event types is appending, not deduplicating. If the same row can truly arrive twice, handle that explicitly with a key (Module 7), not as a side effect.

SPEED

No hidden sort

UNION must compare every row with every other. On a billion rows that dominates the query.

CORRECTNESS

Real duplicates survive

Two customers who each paid 20.00 on the same day are two rows. UNION would merge them.

CLARITY

Add a source column

`SELECT 'eu' AS region, ...` on each branch tells you where every row came from.

INTERSECT and EXCEPT

Customers who ordered in March, compared with customers who ordered in April.

INTERSECT · IN BOTH

```
1 SELECT customer_id FROM orders
2 WHERE order_date < DATE '2025-04-01'
3 INTERSECT
4 SELECT customer_id FROM orders
5 WHERE order_date >= DATE '2025-04-01'
6 ORDER BY customer_id;
```

RESULT · 3 rows

customer_id
1
2
3

Rows returned by both queries: customers active in both months.

EXCEPT · IN FIRST, NOT SECOND

```
1 SELECT customer_id FROM orders
2 WHERE order_date < DATE '2025-04-01'
3 EXCEPT
4 SELECT customer_id FROM orders
5 WHERE order_date >= DATE '2025-04-01'
6 ORDER BY customer_id;
```

RESULT · 1 row

customer_id
4

March customers who did not return in April. Order matters. **EXCEPT** is also a **table diff**: new version **EXCEPT** old version lists every changed row (Module 12). Oracle spells it **MINUS**.

A subquery that returns one value

```
1 SELECT order_id, customer_id, amount
2 FROM orders
3 WHERE amount > (
4     SELECT AVG(amount)
5     FROM orders
6 )
7 ORDER BY amount DESC;
```

- 1 The inner query runs first and returns a single number: the average order, 132.50.
- 2 The outer query then uses it like a constant. A subquery that returns one row and one column is called **scalar**.

RESULT · 4 rows

order_id	customer_id	amount
1005	2	310.00
1007	5	265.00
1002	2	220.00
1001	1	135.00



MENTAL MODEL

A query inside brackets is a value

Depending on its shape it can stand in for a single value, a list (after **IN**) or a whole table (after **FROM**).

A subquery that returns a list

```
1 SELECT customer_id, full_name
2 FROM customers
3 WHERE customer_id IN (
4     SELECT o.customer_id
5     FROM orders AS o
6     JOIN order_items AS oi
7     ON oi.order_id = o.order_id
8     WHERE oi.product_id = 105      -- Monitor 27in
9 )
10 ORDER BY customer_id;
```

- 1 The inner query returns one column: the ids of customers who bought a monitor.
- 2 **IN** keeps customers whose id is in that list. Duplicates in the list do no harm: Ben bought two monitors and still appears once.

RESULT · 2 rows

customer_id	full_name
2	Ben Carter
5	Elena Rossi

★ PRO TIP

IN (subquery) and **EXISTS** are both semi-joins. Prefer **EXISTS** when the match needs more than one column.

Correlated subqueries run per outer row

```
1 SELECT
2     c.full_name,
3     (SELECT MAX(o.order_date)
4      FROM orders AS o
5      WHERE o.customer_id = c.customer_id) AS last_order
6 FROM customers AS c
7 ORDER BY c.customer_id;
```

RESULT · 6 rows

full_name	last_order
Ada Moreno	2025-04-12
Ben Carter	2025-04-10
Chloe Dubois	2025-04-20
Dev Patel	2025-03-15
Elena Rossi	2025-04-03
Farid Khan	NULL

- 1 The inner query mentions `c.customer_id` from the **outer** query. It is re-evaluated for each customer.
- 2 For Farid no order exists, `MAX` of nothing is NULL, and the row is kept.

DATA ENGINEERING LENS

Readable, but watch the cost

Conceptually this is a loop: one lookup per customer. Optimizers often rewrite it as a join, but not always. On large tables prefer a **LEFT JOIN** to a pre-aggregated subquery.

NOT IN returns nothing if the list has a NULL

✗ ZERO ROWS

```
1 SELECT full_name, city
2 FROM customers
3 WHERE city NOT IN (
4     SELECT city FROM customers
5     WHERE country = 'UK'
6 );
```

Goal: customers outside the cities of our UK customers. But Dev, a UK customer, has no city, so the list is ('Leeds', NULL, 'Leeds'). Is Madrid different from an unknown value? Unknown. No row is ever TRUE and the result is **empty**, with no error.

✓ CORRECT

```
1 SELECT c.full_name, c.city
2 FROM customers AS c
3 WHERE NOT EXISTS (
4     SELECT 1 FROM customers AS u
5     WHERE u.country = 'UK'
6     AND u.city = c.city
7 );
```

RESULT · 4 rows

full_name	city
Ada Moreno	Madrid
Chloe Dubois	Lyon
Dev Patel	NULL
Elena Rossi	Milan

NOT EXISTS asks "is there a matching row?". A NULL never matches, so it cannot poison the answer.

A subquery as a table

```
1 SELECT
2     ROUND(AVG(total), 2) AS avg_customer_value,
3     MAX(total)          AS best_customer_value
4 FROM (
5     SELECT customer_id, SUM(amount) AS total
6     FROM orders
7     GROUP BY customer_id
8 ) AS per_customer;
```

- 1 Step one: total per customer (five rows).
- 2 Step two: average those five totals. An aggregate of an aggregate needs two levels.
- 3 A subquery in **FROM** is a **derived table** and must have an alias.

RESULT · 1 row

avg_customer_value	best_customer_value
265.00	590.00

⚠ TRAP

Readability collapses when derived tables nest three deep: you must read from the inside out. The next slide fixes that.

CTEs: name each step, read top to bottom

```
1 WITH completed AS (  
2     SELECT customer_id, amount  
3     FROM orders  
4     WHERE status = 'completed'  
5 ),  
6 per_customer AS (  
7     SELECT customer_id,  
8         COUNT(*) AS n_orders,  
9         SUM(amount) AS total  
10    FROM completed  
11    GROUP BY customer_id  
12 )  
13 SELECT c.full_name, p.n_orders, p.total  
14 FROM per_customer AS p  
15 JOIN customers AS c ON c.customer_id = p.customer_id  
16 WHERE p.total > 100  
17 ORDER BY p.total DESC;
```

1 **WITH name AS (query)** defines a **common table expression**: a named, temporary result.

2 Later CTEs can read earlier ones, like variables.

3 The final **SELECT** uses them as if they were tables.

RESULT · 3 rows

full_name	n_orders	total
Ben Carter	2	530.00
Ada Moreno	3	280.00
Elena Rossi	1	265.00

DATA ENGINEERING LENS

This shape (import, filter, aggregate, join) is exactly how production transformation models are written.

Think of a CTE chain as a small pipeline

Each CTE does one job and hands a clean table to the next. You can run any step alone to debug it.



★ PRO TIP

Debugging trick

Replace the final statement with **SELECT * FROM per_customer** (or any step) to inspect its output. A CTE chain is a pipeline you can pause anywhere.

Recursive CTEs walk a hierarchy

```
1 WITH RECURSIVE chain AS (  
2     -- anchor: start at the top  
3     SELECT emp_id, emp_name, 1 AS depth  
4     FROM employees  
5     WHERE manager_id IS NULL  
6  
7     UNION ALL  
8  
9     -- recursive step: find direct reports  
10    SELECT e.emp_id, e.emp_name, c.depth + 1  
11    FROM employees AS e  
12    JOIN chain AS c ON e.manager_id = c.emp_id  
13 )  
14 SELECT emp_name, depth  
15 FROM chain  
16 ORDER BY depth, emp_name;
```

- 1 The **anchor** runs once and seeds the result with Grace.
- 2 The **recursive step** joins the table to the rows found so far, adding the next level down.
- 3 It repeats until a pass adds no new rows. A cycle in the data would loop forever: add a guard such as **WHERE c.depth < 20**.

RESULT · 6 rows

emp_name	depth
Grace	1
Hugo	2
Ines	2
Jamal	3
Kira	3
Liam	3

Generating rows: a date spine

```
1 WITH days AS (  
2     SELECT d::date AS day  
3     FROM generate_series(DATE '2025-03-01', DATE '2025-03-07',  
4                          INTERVAL '1 day') AS t(d)  
5 )  
6 SELECT  
7     days.day,  
8     COUNT(o.order_id) AS n_orders,  
9     COALESCE(SUM(o.amount), 0) AS revenue  
10 FROM days  
11 LEFT JOIN orders AS o ON o.order_date = days.day  
12 GROUP BY days.day  
13 ORDER BY days.day;
```

DIALECT NOTE

BigQuery: `GENERATE_DATE_ARRAY`. Snowflake: `GENERATOR`. Anywhere: a date dimension table.

- 1 `generate_series` produces one row per step: here, seven consecutive days.
- 2 Left joining orders onto the spine keeps the days with no sales.
- 3 Count a column from `orders`, so empty days show 0, not 1.

RESULT · 7 rows

day	n_orders	revenue
2025-03-01	2	355.00
2025-03-02	0	0.00
2025-03-03	0	0.00
2025-03-04	0	0.00
2025-03-05	1	55.00
2025-03-06	0	0.00
2025-03-07	1	90.00

Subquery, CTE, view or temp table?

Four ways to give a query a name. They differ in how long the name lives and whether the result is stored.

Tool	Lives for	Stores data?	Reach for it when
Subquery	One place in one query	No	The logic is tiny and used once: a scalar comparison or an IN list.
CTE	One query	No (usually inlined)	A query has several logical steps. The default for readable SQL.
View	Until dropped	No, runs on every read	Many queries or users need the same logic, always up to date.
Temporary table	The session	Yes	An expensive intermediate result is reused several times in one job.
Table (CTAS)	Until dropped	Yes	Results must persist between runs: the output of a pipeline step.

 DATA ENGINEERING LENS

A CTE referenced three times may be computed three times. If it is expensive, materialise it as a temp table or its own model.

Exercise: above-average customers

 YOUR TURN

Return the **name and total spend** of every customer whose total (all statuses) is **greater than the average customer total**. Largest first.

TABLES YOU NEED

orders

customers

Hint 1

First build a CTE with one row per customer and their total.

Hint 2

Use that CTE twice: once as the table, once inside a scalar subquery for the average.

 EXPECTED OUTPUT

full_name	total
Ben Carter	590.00
Ada Moreno	280.00

Solution on the next slide. Try first.

Solution: above-average customers

```
1 WITH per_customer AS (  
2     SELECT customer_id, SUM(amount) AS total  
3     FROM orders  
4     GROUP BY customer_id  
5 )  
6 SELECT c.full_name, p.total  
7 FROM per_customer AS p  
8 JOIN customers AS c  
9     ON c.customer_id = p.customer_id  
10 WHERE p.total > (SELECT AVG(total) FROM per_customer)  
11 ORDER BY p.total DESC;
```

- 1 Five customers have orders: totals 280, 590, 165, 25 and 265.
- 2 Their average is 265.00. Elena, at exactly 265, is not **greater than** it.
- 3 The CTE is read twice. That is fine here; on big data you might materialise it.

⚠ TRAP

This is the average over customers who ordered. Including Farid (total 0) would lower it to 220.83. State which population you mean.

Module 6 recap

- ✓ **UNION ALL** appends; **UNION** also removes duplicates and costs a sort.
- ✓ **INTERSECT** returns common rows; **EXCEPT** returns rows only in the first query.
- ✓ A subquery can act as a value, a list or a table.
- ✓ Correlated subqueries are evaluated per outer row.
- ✓ Never use **NOT IN** with a list that could contain NULL. Use **NOT EXISTS**.
- ✓ CTEs turn nested logic into named steps you can read and debug in order.
- ✓ Recursive CTEs handle hierarchies; **generate_series** builds date spines.

**NEXT UP****Module 7: window functions, the data engineer's power tool**



MODULE 07

Window functions

Compute across related rows without collapsing them. Deduplication, ranking, running totals and change detection all live here.

BY THE END YOU CAN

- 1 Explain how OVER differs from GROUP BY
- 2 Rank and number rows within groups
- 3 Deduplicate a table to its latest version
- 4 Build running totals and moving averages
- 5 Compare each row with the previous one using LAG

GROUP BY collapses rows, a window keeps them

GROUP BY · ONE ROW PER GROUP

```
1 SELECT
2     customer_id,
3     SUM(amount) AS customer_total
4 FROM orders
5 WHERE customer_id IN (1, 3)
6 GROUP BY customer_id
7 ORDER BY customer_id;
```

RESULT · 2 rows

customer_id	customer_total
1	280.00
3	165.00

Two rows. The individual orders are gone: you cannot show an order next to its customer total.

WINDOW · EVERY ROW KEPT

```
1 SELECT order_id, customer_id, amount,
2     SUM(amount) OVER (
3         PARTITION BY customer_id
4     ) AS customer_total
5 FROM orders WHERE customer_id IN (1, 3)
6 ORDER BY customer_id, order_id;
```

RESULT · 5 rows

order_id	customer_id	amount	customer_total
1001	1	135.00	280.00
1003	1	55.00	280.00
1009	1	90.00	280.00
1004	3	90.00	165.00
1010	3	75.00	165.00

What a window function is

 **A window function looks at a set of rows related to the current row, and writes its answer next to that row.**

The rows it looks at are "the window". Nothing is removed and nothing is merged: the output has exactly as many rows as the input, plus a new column.

DETAIL + TOTAL

Share of total

Each order next to its customer's total, so you can divide one by the other.

ORDER WITHIN A GROUP

Rank and number

First order, latest version, top three per category.

NEIGHBOURS

Previous and next

Change since the last row, time since the last event.

Anatomy of the OVER clause

Three optional parts inside the brackets decide which rows the function can see.

```
SUM(amount)
```

The function. Any aggregate, or a window-only function such as `ROW_NUMBER` or `LAG`.

```
OVER (
```

Marks it as a window function. Without `OVER` it would be an ordinary aggregate.

```
PARTITION BY customer_id
```

Split rows into independent groups. Omit it and the whole result is one partition.

```
ORDER BY order_date
```

Sort rows inside each partition. Required for ranking, running totals and `LAG`.

```
ROWS BETWEEN UNBOUNDED PRECEDING  
AND CURRENT ROW
```

The frame: which of the sorted rows, relative to the current one, to include.

```
) AS running_total
```

Close and alias. The result is a new column on every row.

ROW_NUMBER, RANK and DENSE_RANK

```
1 SELECT
2     order_id,
3     amount,
4     ROW_NUMBER() OVER (ORDER BY amount DESC,
5                          order_id) AS row_num,
6     RANK() OVER (ORDER BY amount DESC) AS rnk,
7     DENSE_RANK() OVER (ORDER BY amount DESC) AS dense_rnk
8 FROM orders
9 ORDER BY amount DESC, order_id
10 LIMIT 7;
```

RESULT · 7 rows

order_id	amount	row_num	rnk	dense_rnk
1005	310.00	1	1	1
1007	265.00	2	2	2
1002	220.00	3	3	3
1001	135.00	4	4	4
1004	90.00	5	5	5
1009	90.00	6	5	5
1010	75.00	7	7	6

MENTAL MODEL

Three answers to "what about ties?"

Orders 1004 and 1009 both cost 90. **ROW_NUMBER** always gives unique numbers (5, 6), so it needs a tiebreaker to be repeatable. **RANK** gives both 5 and then **skips** to 7. **DENSE_RANK** gives both 5 and continues with 6.

Deduplication: keep the latest row per key

```
1 WITH ranked AS (  
2     SELECT *,  
3         ROW_NUMBER() OVER (  
4             PARTITION BY order_id  
5             ORDER BY updated_at DESC, loaded_at DESC  
6         ) AS rn  
7     FROM raw_orders  
8 )  
9 SELECT order_id, status, amount, updated_at  
10 FROM ranked  
11 WHERE rn = 1  
12 ORDER BY order_id;
```

DATA ENGINEERING LENS

The most used pattern in data engineering. Sources resend, retries duplicate, updates arrive as new rows.

- 1 Restart the numbering for each `order_id`: the business key.
- 2 Newest version first. The second sort key breaks ties between identical rows loaded twice.
- 3 Row number 1 is the latest version. Nine raw rows become five.

RESULT · 5 rows

order_id	status	amount	updated_at
1001	completed	135.00	2025-03-02 14:30
1002	completed	220.00	2025-03-01 11:15
1003	completed	55.00	2025-03-06 16:45
1004	cancelled	90.00	2025-03-08 09:20
1012	completed	40.00	2025-03-09 13:00

What that query started from

The raw landing table keeps every version it ever received. Highlighted: the rows that deduplication keeps.

raw_orders

order_id	customer_id	status	amount	updated_at	loaded_at
1001	1	pending	135.00	2025-03-01 09:00	2025-03-01 10:00
1001	1	completed	135.00	2025-03-02 14:30	2025-03-02 15:00
1002	2	completed	220.00	2025-03-01 11:15	2025-03-01 12:00
1002	2	completed	220.00	2025-03-01 11:15	2025-03-02 12:00
1003	1	pending	55.00	2025-03-05 08:00	2025-03-05 09:00
1003	1	completed	55.00	2025-03-06 16:45	2025-03-06 17:00
1004	3	pending	90.00	2025-03-07 10:00	2025-03-07 11:00
1004	3	cancelled	90.00	2025-03-08 09:20	2025-03-08 10:00
1012	9	completed	40.00	2025-03-09 13:00	2025-03-09 14:00

1001, 1003, 1004 changed status. Two versions each; the later **updated_at** wins.

1002 was loaded twice with identical content. **loaded_at** breaks the tie.

1012 points at customer 9, who does not exist. Deduplication keeps it; Module 12 catches it.

Deduplication shortcuts by dialect

The CTE with `rn = 1` works everywhere. Some engines offer a shorter spelling of the same idea.

SNOWFLAKE · BIGQUERY · DATABRICKS

```
SELECT *
FROM raw_orders
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY order_id
    ORDER BY updated_at DESC
) = 1;
```

QUALIFY filters on a window function, as **HAVING** filters on an aggregate. No CTE needed.

POSTGRESQL

```
SELECT DISTINCT ON (order_id) *
FROM raw_orders
ORDER BY order_id,
         updated_at DESC;
```

DISTINCT ON keeps the first row per key according to the **ORDER BY**.

EVERY ENGINE

```
SELECT *
FROM (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY order_id
        ORDER BY updated_at DESC
    ) AS rn
    FROM raw_orders
) AS t
WHERE rn = 1;
```

Window functions are computed in **SELECT**, after **WHERE**, so the filter needs an outer query.

Top N per group

```
1 WITH ranked AS (  
2     SELECT  
3         customer_id, order_id, amount,  
4         ROW_NUMBER() OVER (  
5             PARTITION BY customer_id  
6             ORDER BY amount DESC, order_id  
7         ) AS rn  
8     FROM orders  
9 )  
10 SELECT customer_id, order_id, amount, rn  
11 FROM ranked  
12 WHERE rn <= 2  
13     AND customer_id <= 3  
14 ORDER BY customer_id, rn;
```

- 1 Numbering restarts for each customer.
- 2 Largest order gets 1.
- 3 Keep the top two per customer. Use **RANK** instead if ties should all be included.

RESULT · 6 rows

customer_id	order_id	amount	rn
1	1001	135.00	1
1	1009	90.00	2
2	1005	310.00	1
2	1002	220.00	2
3	1004	90.00	1
3	1010	75.00	2

Running totals

```
1 SELECT
2     order_date,
3     order_id,
4     amount,
5     SUM(amount) OVER (
6         ORDER BY order_date, order_id
7         ROWS BETWEEN UNBOUNDED PRECEDING
8             AND CURRENT ROW
9     ) AS running_revenue
10 FROM orders
11 WHERE status = 'completed'
12 ORDER BY order_date, order_id;
```

- 1 With **ORDER BY** inside **OVER**, the sum accumulates along the sort order.
 - 2 The frame says: from the first row up to and including this one.
- ⚡ The last value, 1,175.00, equals the completed total. A built-in check.

RESULT · 8 rows

order_date	order_id	amount	running_revenue
2025-03-01	1001	135.00	135.00
2025-03-01	1002	220.00	355.00
2025-03-05	1003	55.00	410.00
2025-03-12	1005	310.00	720.00
2025-03-15	1006	25.00	745.00
2025-04-03	1007	265.00	1010.00
2025-04-12	1009	90.00	1100.00
2025-04-20	1010	75.00	1175.00

The default frame groups tied rows together

Orders 1001 and 1002 share the date 1 March. Leave out the frame and they are treated as one step.

✗ DEFAULT FRAME (RANGE)

```
1 SELECT order_id, amount,  
2     SUM(amount) OVER (  
3         ORDER BY order_date  
4     ) AS running  
5 FROM orders  
6 ORDER BY order_date, order_id LIMIT 3;
```

RESULT · 3 rows

order_id	amount	running
1001	135.00	355.00
1002	220.00	355.00
1003	55.00	410.00

The default frame is **RANGE**: rows with the **same sort value** all count as current. Both show 355.

✓ EXPLICIT ROWS FRAME

```
1 SELECT order_id, amount,  
2     SUM(amount) OVER (  
3         ORDER BY order_date, order_id  
4         ROWS UNBOUNDED PRECEDING  
5     ) AS running  
6 FROM orders  
7 ORDER BY order_date, order_id LIMIT 3;
```

RESULT · 3 rows

order_id	amount	running
1001	135.00	135.00
1002	220.00	355.00
1003	55.00	410.00

ROWS counts physical rows, and the tiebreaker makes the order unique: 135, then 355.

Frames: a sliding view over sorted rows

For each row, the frame selects neighbours by position. Here: the current row and the two before it.

order_id amount

1001	135.00
1002	220.00
1003	55.00
1004	90.00
1005	310.00
1006	25.00

2 PRECEDING

1 PRECEDING

CURRENT ROW

1 FOLLOWING (outside this frame)

AVG of the frame
(55 + 90 + 310) / 3
= 151.67

FRAME VOCABULARY

UNBOUNDED PRECEDING	from the first row of the partition
n PRECEDING	n rows before the current row
CURRENT ROW	the row being computed
n FOLLOWING	n rows after the current row
UNBOUNDED FOLLOWING	to the last row of the partition

MENTAL MODEL

The frame moves with the row. Step down one row and the highlighted box slides down with it: a three-row moving window.

Moving averages

```
1 SELECT
2     order_id,
3     amount,
4     ROUND (AVG (amount) OVER (
5         ORDER BY order_date, order_id
6         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
7     ), 2) AS moving_avg_3
8 FROM orders
9 ORDER BY order_date, order_id
10 LIMIT 6;
```

DATA ENGINEERING LENS

A 7-day moving average over daily totals needs one row per day. Build the date spine first, or missing days silently shorten the window.

1 Average of this row and the two before it.

⚡ The first rows have fewer than three rows available, so they average what exists: 135, then $(135 + 220) / 2$.

⚡ The highlighted row is the one drawn on the previous slide.

RESULT · 6 rows

order_id	amount	moving_avg_3
1001	135.00	135.00
1002	220.00	177.50
1003	55.00	136.67
1004	90.00	121.67
1005	310.00	151.67
1006	25.00	141.67

LAG and LEAD: look at neighbouring rows

```
1 SELECT
2     customer_id,
3     order_date,
4     LAG(order_date) OVER w AS prev_order,
5     order_date - LAG(order_date) OVER w AS days_gap
6 FROM orders
7 WHERE customer_id <= 2
8 WINDOW w AS (PARTITION BY customer_id
9              ORDER BY order_date)
10 ORDER BY customer_id, order_date;
```

- 1 **LAG (x)** returns x from the **previous** row in the window; **LEAD (x)** from the next. A first row has no previous one, so it gets NULL.
- 2 Subtracting gives the days between consecutive orders.
- 3 A named **WINDOW** avoids repeating the same definition.

RESULT · 6 rows

customer_id	order_date	prev_order	days_gap
1	2025-03-01	NULL	NULL
1	2025-03-05	2025-03-01	4
1	2025-04-12	2025-03-05	38
2	2025-03-01	NULL	NULL
2	2025-03-12	2025-03-01	11
2	2025-04-10	2025-03-12	29

Sessionisation, step 1: measure the gaps

```
1 WITH gaps AS (  
2     SELECT user_id, event_ts,  
3            event_ts - LAG(event_ts) OVER w AS gap  
4     FROM page_events  
5     WINDOW w AS (PARTITION BY user_id ORDER BY event_ts)  
6 )  
7 SELECT * FROM gaps  
8 ORDER BY user_id, event_ts;
```

1

1 Time since the same user's previous event.



User 1 pauses for 1 hour 35 minutes before 11:00. That is where a new visit begins.

RESULT · 7 rows

user_id	event_ts	gap
1	2025-05-01 09:00	NULL
1	2025-05-01 09:10	00:10:00
1	2025-05-01 09:25	00:15:00
1	2025-05-01 11:00	01:35:00
1	2025-05-01 11:05	00:05:00
2	2025-05-01 14:00	NULL
2	2025-05-01 15:00	01:00:00

DATA ENGINEERING LENS

Events have no session id

Clickstreams record page views, not visits. Engineers derive visits by declaring "a gap of more than 30 minutes starts a new session".

Sessionisation, step 2: flag and count

```
1  -- continues from the gaps CTE
2  , flagged AS (
3      SELECT *,
4          CASE WHEN gap IS NULL
5              OR gap > INTERVAL '30 minutes'
6              THEN 1 ELSE 0 END AS new_session
7      FROM gaps
8  )
9  SELECT user_id, event_ts, new_session,
10     SUM(new_session) OVER (
11         PARTITION BY user_id ORDER BY event_ts
12     ) AS session_no
13 FROM flagged
14 ORDER BY user_id, event_ts;
```

1

2

- 1 Mark a row 1 when it starts a session: the user's first event, or one after a long gap.
 - 2 A running sum of those flags numbers the sessions: every row inherits the count of starts so far.
-  **Flag, then running-sum** turns "where does a group start?" into a group id. The same trick segments trips and status runs.

RESULT · 7 rows

user_id	event_ts	new_session	session_no
1	2025-05-01 09:00	1	1
1	2025-05-01 09:10	0	1
1	2025-05-01 09:25	0	1
1	2025-05-01 11:00	1	2
1	2025-05-01 11:05	0	2
2	2025-05-01 14:00	1	1
2	2025-05-01 15:00	1	2

LAST_VALUE needs a full frame

✗ RETURNS THE CURRENT ROW

```
1 SELECT order_id, order_date,
2     LAST_VALUE(order_id) OVER (
3         PARTITION BY customer_id
4         ORDER BY order_date
5     ) AS latest
6 FROM orders WHERE customer_id = 1
7 ORDER BY order_date;
```

RESULT · 3 rows

order_id	order_date	latest
1001	2025-03-01	1001
1003	2025-03-05	1003
1009	2025-04-12	1009

Goal: the latest order id on every row. But the default frame ends at the current row, so the "last" row it sees is itself.

✓ SEES THE WHOLE PARTITION

```
1 SELECT order_id, order_date,
2     LAST_VALUE(order_id) OVER (
3         PARTITION BY customer_id
4         ORDER BY order_date
5         ROWS BETWEEN UNBOUNDED PRECEDING
6             AND UNBOUNDED FOLLOWING
7     ) AS latest
8 FROM orders WHERE customer_id = 1
9 ORDER BY order_date;
```

RESULT · 3 rows

order_id	order_date	latest
1001	2025-03-01	1009
1003	2025-03-05	1009
1009	2025-04-12	1009

Extend the frame to the end. Or sort descending and use **FIRST_VALUE**, which has no such surprise.

Percent of total with an empty OVER ()

```
1 SELECT
2     status,
3     SUM(amount) AS revenue,
4     ROUND(100.0 * SUM(amount)
5           / SUM(SUM(amount)) OVER (), 2) AS pct_of_total
6 FROM orders
7 GROUP BY status
8 ORDER BY revenue DESC;
```

RESULT · 3 rows

status	revenue	pct_of_total
completed	1175.00	88.68
cancelled	90.00	6.79
shipped	60.00	4.53

- 1 **OVER ()** with nothing inside means "all rows of the result". The inner **SUM(amount)** is the group total; the outer window sum adds those up.
- 2 Window functions run **after GROUP BY**, so they can work on aggregated rows.
- 3 **100.0** forces decimal arithmetic.

★ PRO TIP

Percentages should add up to 100. Check: 88.68 + 6.79 + 4.53.

The window function catalogue

Three families. Any ordinary aggregate also becomes a window function when followed by OVER.

Family	Function	Returns	Typical pipeline use
Ranking	ROW_NUMBER()	Unique sequence 1, 2, 3 in the given order	Deduplication, top N, surrogate ordering
Ranking	RANK(), DENSE_RANK()	Position with ties; with or without gaps	Leaderboards, "top 3 prices"
Ranking	NTILE(n), PERCENT_RANK()	Bucket number 1 to n; relative position 0 to 1	Quartiles, percentile bands
Offset	LAG(x, n), LEAD(x, n)	Value n rows before or after	Change detection, time between events
Offset	FIRST_VALUE(x), LAST_VALUE(x)	Value at the frame's first or last row	First touch, latest known status
Aggregate	SUM, AVG, COUNT, MIN, MAX	Aggregate over the frame	Running totals, moving averages, share of total

Gaps and islands, step 1: the trick

```
1 SELECT
2     login_date,
3     ROW_NUMBER() OVER (ORDER BY login_date) AS rn,
4     login_date - CAST(ROW_NUMBER() OVER (
5         ORDER BY login_date) AS INTEGER) AS island_key
6 FROM logins
7 WHERE user_id = 1
8 ORDER BY login_date;
```

DATA ENGINEERING LENS

Uptime periods, subscription spans and consecutive-day streaks are all islands. The gaps between them are outages and churn.

- 1 Number the dates 1, 2, 3 and so on.
 - 2 Subtract that number from the date. On **consecutive** days both grow by one, so the difference stays constant.
- ⚡ When a day is skipped the date jumps but the number does not, and the key changes. Each distinct key is one streak (an "island").

RESULT · 6 rows

login_date	rn	island_key
2025-05-01	1	2025-04-30
2025-05-02	2	2025-04-30
2025-05-03	3	2025-04-30
2025-05-05	4	2025-05-01
2025-05-06	5	2025-05-01
2025-05-09	6	2025-05-03

Gaps and islands, step 2: group the streaks

```
1 WITH numbered AS (  
2     SELECT  
3         user_id,  
4         login_date,  
5         login_date - CAST(ROW_NUMBER() OVER (  
6             PARTITION BY user_id  
7             ORDER BY login_date) AS INTEGER) AS island_key  
8     FROM logins  
9 )  
10 SELECT  
11     user_id,  
12     MIN(login_date) AS streak_start,  
13     MAX(login_date) AS streak_end,  
14     COUNT(*) AS days  
15 FROM numbered  
16 GROUP BY user_id, island_key  
17 ORDER BY streak_start;
```

1 Group by the constant key: one row per streak.

2 The first and last date of each island.

⚡ Six logins become three streaks of 3, 2 and 1 days.

RESULT · 3 rows

user_id	streak_start	streak_end	days
1	2025-05-01	2025-05-03	3
1	2025-05-05	2025-05-06	2
1	2025-05-09	2025-05-09	1

Exercise: time to first order

 YOUR TURN

For each customer who has ordered, find their **first order** and the number of **days between signup and that order**. Return name, order id, order date and the day count, quickest first.

TABLES YOU NEED

orders

customers

Hint 1

Number each customer's orders by date with **ROW_NUMBER** in a CTE, then keep number 1.

Hint 2

Join to **customers** for the name and signup date. Date minus date gives days.

 EXPECTED OUTPUT

full_name	order_id	order_date	days_to_first_order
Elena Rossi	1007	2025-04-03	1
Dev Patel	1006	2025-03-15	4
Chloe Dubois	1004	2025-03-07	15
Ben Carter	1002	2025-03-01	26
Ada Moreno	1001	2025-03-01	45

Solution on the next slide. Try first.

Solution: time to first order

```
1 WITH first_orders AS (  
2     SELECT  
3         order_id, customer_id, order_date,  
4         ROW_NUMBER() OVER (  
5             PARTITION BY customer_id  
6             ORDER BY order_date, order_id  
7         ) AS rn  
8     FROM orders  
9 )  
10 SELECT  
11     c.full_name, f.order_id, f.order_date,  
12     f.order_date - c.signup_date AS days_to_first_order  
13 FROM first_orders AS f  
14 JOIN customers AS c  
15     ON c.customer_id = f.customer_id  
16 WHERE f.rn = 1  
17 ORDER BY days_to_first_order;
```

- 1 Earliest order gets 1. `order_id` breaks ties on the same day.
- 2 One row per customer survives, so the join cannot fan out.
- 3 Elena ordered the day after signing up; Ada waited 45 days.

Module 7 recap

- ✓ A window function adds a column; it never removes or merges rows.
- ✓ **PARTITION BY** splits, **ORDER BY** sorts, the frame selects neighbours.
- ✓ **ROW_NUMBER ... WHERE rn = 1** deduplicates to the latest row per key.
- ✓ Always add a tiebreaker to the window **ORDER BY**.
- ✓ Write the frame explicitly: **ROWS**, not the default **RANGE**.
- ✓ **LAG** and **LEAD** compare a row with its neighbours.
- ✓ Flag-then-running-sum builds sessions; date minus row number finds streaks.
- ✓ Windows run after **GROUP BY** and before the final **ORDER BY**.

 NEXT UP**Module 8: defining tables with DDL**



MODULE 08

DDL: defining tables

Until now you have read tables someone else made. Data engineers make them, and the choices last for years.

BY THE END YOU CAN

- 1 Write CREATE TABLE with sensible types
- 2 Protect data with constraints
- 3 Change and remove tables safely
- 4 Choose between tables, views and materialised views
- 5 Write DDL that can be re-run without failing

CREATE TABLE, line by line

```
1 CREATE TABLE IF NOT EXISTS orders (  
2     order_id      BIGINT      PRIMARY KEY,  
3     customer_id   BIGINT      NOT NULL  
4                   REFERENCES customers (customer_id),  
5     order_date    DATE        NOT NULL,  
6     status        VARCHAR(20) NOT NULL DEFAULT 'pending',  
7     amount        NUMERIC(12,2) CHECK (amount >= 0),  
8     loaded_at     TIMESTAMPTZ NOT NULL DEFAULT now()  
9 );
```

- 1 **IF NOT EXISTS** makes the statement safe to run twice.
- 2 Each line is: column name, data type, then optional rules. **PRIMARY KEY** means unique and never NULL.
- 3 A foreign key: the value must exist in **customers**.
- 4 **DEFAULT** supplies a value when an insert leaves the column out.
- 5 An audit column recording when the row arrived. Add one to every pipeline table.

Constraints: rules the database enforces

PRIMARY KEY

Identity

Unique and not NULL. One per table. May span several columns (a composite key).

FOREIGN KEY

Relationships

The value must exist in the referenced table. Prevents orphan rows.

NOT NULL

Completeness

The column must always have a value. The cheapest and most valuable constraint.

UNIQUE

No repeats

No two rows share the value, for example an email. NULLs are usually allowed.

CHECK

Validity

Any true/false rule on the row: `amount >= 0`, or a status within a fixed list.

DEFAULT

Fallback value

Not strictly a constraint: fills in a value when none is given.

DATA ENGINEERING LENS

Warehouses often do not enforce keys. Snowflake, BigQuery and Redshift accept **PRIMARY KEY** and **FOREIGN KEY** as documentation only. There you must test uniqueness yourself (Module 12).

What happens when a rule is broken

The statement fails and nothing is written. Each message names the constraint, which tells you which rule was violated.

You try to	PostgreSQL answers
Insert order_id 1001 a second time	duplicate key value violates unique constraint "orders_pkey"
Insert an order for customer 99	insert or update on table "orders" violates foreign key constraint "orders_customer_id_fkey"
Insert an order with no date	null value in column "order_date" violates not-null constraint
Insert an amount of -5	new row for relation "orders" violates check constraint "orders_amount_check"
Delete customer 1, who has orders	update or delete on table "customers" violates foreign key constraint on table "orders"

★ PRO TIP

A batch **INSERT** of a million rows fails entirely if one row breaks a constraint. Validate in staging first, then load clean rows.

Schemas and naming conventions

A schema is a folder for tables. Consistent names let anyone guess where a table lives and what one row means.

1

Three-part names

`database.schema.table`,
for example
`shop.marts.dim_date`.
Qualify names in pipeline
code.

2

Schemas as layers

`raw` as loaded, `staging`
cleaned one-to-one, `marts`
modelled for use. Permissions
follow the layers.

3

Prefixes that say what it is

`stg_` staging, `int_`
intermediate, `dim_`
dimension, `fct_` fact.

4

Boring, predictable columns

`snake_case`. Keys end in
`_id`, dates in `_date`,
timestamps in `_at`, flags start
with `is_`.

DIALECT NOTE

Quoted identifiers are a trap

Unquoted names are folded to lowercase in PostgreSQL and UPPERCASE in Snowflake. Create a column as `"OrderID"` in quotes and every query must quote it, with exact case, forever.

ALTER TABLE changes an existing table

```
1  -- add a column: existing rows receive the default
2  ALTER TABLE orders ADD COLUMN currency CHAR(3) DEFAULT 'GBP';
3
4  -- rename a column: every query using the old name breaks
5  ALTER TABLE orders RENAME COLUMN amount TO order_amount;
6
7  -- change a type: only safe when every value converts
8  ALTER TABLE orders ALTER COLUMN status TYPE VARCHAR(30);
9
10 -- remove a column: the data is gone
11 ALTER TABLE orders DROP COLUMN currency;
```

- 1 Adding a nullable or defaulted column is the one change that breaks nobody.
- 2 A rename is two changes at once: a removal and an addition.
- 3 Widening (20 to 30 characters) is safe. Narrowing can fail or truncate.

⚙️ DATA ENGINEERING LENS

On big tables some **ALTERS** rewrite every row and lock the table. Check how your engine implements a change before running it in production hours.

DELETE, TRUNCATE or DROP?

Three ways to remove data. They differ in what disappears, how fast, and whether you can change your mind.

Statement	Removes	Table still exists?	Pick rows?	Undo with ROLLBACK?	Speed
<code>DELETE FROM t WHERE ...</code>	The rows you choose	Yes	Yes	Yes	Slow: row by row, fully logged
<code>TRUNCATE TABLE t</code>	Every row	Yes	No	In PostgreSQL and SQL Server. Not in MySQL or Oracle.	Fast: releases storage in one step
<code>DROP TABLE t</code>	Rows and the table itself	No	No	In PostgreSQL. Elsewhere usually final.	Instant

 TRAP

Know your safety net before you need it

Snowflake offers `UNDROP TABLE` and Time Travel; BigQuery keeps deleted tables for a limited window. Find out what your platform retains before the day you need it.

CREATE TABLE AS: build a table from a query

```
1 CREATE TABLE customer_totals AS
2 SELECT
3     customer_id,
4     COUNT(*) AS n_orders,
5     SUM(amount) AS total_spent
6 FROM orders
7 WHERE status = 'completed'
8 GROUP BY customer_id;
9
10 SELECT * FROM customer_totals
11 ORDER BY customer_id;
```

1

2

1 "CTAS" creates the table and fills it in one statement. Column names and types come from the query.

2 The result is a real, stored table: a snapshot of the query at the moment it ran.

RESULT · 5 rows

customer_id	n_orders	total_spent
1	3	280.00
2	2	530.00
3	1	75.00
4	1	25.00
5	1	265.00

DATA ENGINEERING LENS

Most transformation steps are a CTAS

A pipeline step is usually "run this SELECT and save the result as a table". Tools such as dbt generate the **CREATE TABLE AS** for you.

Views and materialised views

Both give a query a permanent name. The difference is whether the answer is stored.

VIEW · A SAVED QUERY

```
1 CREATE OR REPLACE VIEW v_completed AS
2 SELECT order_id, customer_id,
3        order_date, amount
4 FROM orders
5 WHERE status = 'completed';
```

- Stores **no data**. The query runs every time someone reads the view.
- Always current. Costs the full query on each read.
- Use for: renaming, light filtering, hiding columns.

MATERIALIZED VIEW · A STORED ANSWER

```
1 CREATE MATERIALIZED VIEW mv_daily AS
2 SELECT order_date,
3        SUM(amount) AS revenue
4 FROM orders
5 GROUP BY order_date;
6
7 REFRESH MATERIALIZED VIEW mv_daily;
```

- Stores the **result**. Reads are fast.
- Stale until refreshed (manually in PostgreSQL, automatically in some warehouses).
- Use for: expensive aggregates read many times.

Temporary tables live for one session

```
1 CREATE TEMP TABLE tmp_big_spenders AS
2 SELECT customer_id, SUM(amount) AS total
3 FROM orders
4 GROUP BY customer_id
5 HAVING SUM(amount) > 200;
6
7 -- reuse it as often as you like in this session
8 SELECT COUNT(*) FROM tmp_big_spenders;
9
10 SELECT * FROM tmp_big_spenders
11 ORDER BY total DESC;
```

1

2

- 1 **TEMP** tables are private to your connection and vanish when it closes.
- 2 Computed once, read many times. Unlike a CTE, the result is physically stored.

RESULT · 3 rows

customer_id	total
2	590.00
1	280.00
5	265.00

★ PRO TIP

Use a temp table when an expensive intermediate result feeds several later statements in the same job.

Write DDL that survives a re-run

Pipelines fail halfway and get restarted. Setup statements must be **idempotent**: running them once or five times leaves the same result.

✗ FAILS THE SECOND TIME

```
1 CREATE TABLE stg_orders (  
2     order_id BIGINT,  
3     amount    NUMERIC(12,2)  
4 );  
5  
6 DROP TABLE old_orders;
```

Second run: **relation "stg_orders" already exists**, then **table "old_orders" does not exist**. The job stops on a harmless condition.

✓ SAFE TO REPEAT

```
1 CREATE TABLE IF NOT EXISTS stg_orders (  
2     order_id BIGINT,  
3     amount    NUMERIC(12,2)  
4 );  
5  
6 DROP TABLE IF EXISTS old_orders;  
7  
8 CREATE OR REPLACE VIEW v_orders AS  
9 SELECT order_id, amount FROM stg_orders;
```

Each statement describes the desired **end state**, not a one-time action. Module 11 applies the same idea to data.

Schema evolution: safe and breaking changes

Tables have consumers: dashboards, models, exports. A change is "breaking" if any of them stops working.

✓ BACKWARD COMPATIBLE

- **Add a nullable column.** Existing queries ignore it.
- **Add a column with a default.**
- **Widen a type:** `INTEGER` to `BIGINT`, `VARCHAR(20)` to `VARCHAR(50)`.
- **Add a new table or view.**
- **Relax a constraint:** drop a `NOT NULL`.

✗ BREAKING

- **Rename or drop a column.** Every reference fails.
- **Change a type** so values convert differently or not at all.
- **Change the meaning** of a column while keeping its name. The worst kind: nothing fails, numbers just shift.
- **Change the grain:** one row per order becomes one per order line.
- **Add `NOT NULL`** to a column that has NULLs.

★ PRO TIP

Expand, migrate, contract

To rename safely: add the new column, fill both for a while, move consumers across, then drop the old one. Three small safe steps instead of one breaking one.

Exercise: design a payments table

 YOUR TURN

Write the **CREATE TABLE** statement for a **payments** table that meets the requirements on the right. Make it safe to run more than once.

TABLES YOU NEED

orders

Hint 1

Money must be exact and an instant needs a time zone. Pick types accordingly.

Hint 2

A fixed list of allowed values is a **CHECK ... IN (...)**.

 REQUIREMENTS

Column	Must hold	Rule
payment_id	Whole number	Identifies the row
order_id	Whole number	Must exist in orders
paid_at	An instant	Required
method	Short text	card, paypal or bank only
amount	Exact money	Required, above zero
loaded_at	An instant	Filled in automatically

Solution on the next slide. Try first.

Solution: design a payments table

```
1 CREATE TABLE IF NOT EXISTS payments (  
2     payment_id BIGINT PRIMARY KEY,  
3     order_id BIGINT NOT NULL  
4         REFERENCES orders (order_id),  
5     paid_at TIMESTAMPTZ NOT NULL,  
6     method VARCHAR(10) NOT NULL  
7         CHECK (method IN ('card', 'paypal', 'bank')),  
8     amount NUMERIC(12,2) NOT NULL CHECK (amount > 0),  
9     loaded_at TIMESTAMPTZ NOT NULL DEFAULT now()  
10 );
```

- 1 **BIGINT** leaves room to grow. The primary key gives identity.
- 2 No payment can reference an order that does not exist.
- 3 Any other method is rejected at write time.
- 4 Two rules on one column: required, and strictly positive.

Module 8 recap

- ✓ **CREATE TABLE** lists columns as name, type, rules.
- ✓ Constraints (**PRIMARY KEY**, **FOREIGN KEY**, **NOT NULL**, **UNIQUE**, **CHECK**) reject bad data at write time.
- ✓ Cloud warehouses often do not enforce keys: test them yourself.
- ✓ **DELETE** removes chosen rows, **TRUNCATE** all rows, **DROP** the table.
- ✓ CTAS saves a query result; a view saves the query; a materialised view saves both.
- ✓ Use **IF NOT EXISTS**, **IF EXISTS** and **OR REPLACE** so DDL can be re-run.
- ✓ Adding is safe. Renaming, dropping and redefining are breaking.

 NEXT UP**Module 9: changing data with DML and transactions**



MODULE 09

DML and transactions

Insert, update, delete and merge rows, and make groups of changes succeed or fail as one.

BY THE END YOU CAN

- 1 Insert rows from values and from queries
- 2 Update and delete with precision
- 3 Wrap related changes in a transaction
- 4 Explain ACID and isolation levels
- 5 Upsert with ON CONFLICT and MERGE

INSERT adds rows

```
1 INSERT INTO products
2   (product_id, product_name, category, list_price)
3 VALUES
4   (107, 'Desk Lamp', 'accessories', 35.00),
5   (108, 'HDMI Cable', 'accessories', 12.50);
6
7 SELECT * FROM products
8 WHERE product_id >= 106
9 ORDER BY product_id;
```

1

2

1 Always list the target columns. The statement then keeps working when someone adds a column to the table.

2 One bracketed tuple per row. Several rows in one statement is much faster than one statement per row.

 Columns you leave out receive their **DEFAULT**, or **NULL**.

RESULT · 3 rows

product_id	product_name	category	list_price
106	Webcam HD	peripherals	60.00
107	Desk Lamp	accessories	35.00
108	HDMI Cable	accessories	12.50

INSERT ... SELECT loads from a query

```
1 CREATE TABLE orders_archive AS
2 SELECT * FROM orders WHERE 1 = 0;  -- same columns, no rows
3
4 INSERT INTO orders_archive
5     (order_id, customer_id, order_date, status, amount)
6 SELECT order_id, customer_id, order_date, status, amount
7 FROM orders
8 WHERE status = 'cancelled';
9
10 SELECT * FROM orders_archive;
```

1

2

- 1 A condition that is never true copies the structure and no data.
- 2 Any **SELECT** can feed an **INSERT**. Columns are matched **by position**, not by name.

DATA ENGINEERING LENS

This is how data moves between layers: read from staging, write to the target. Position matching is why both column lists are spelled out.

RESULT · 1 row

order_id	customer_id	order_date	status	amount
1004	3	2025-03-07	cancelled	90.00

UPDATE changes existing rows

```
1 UPDATE orders
2 SET status = 'completed'
3 WHERE order_id = 1008;
4
5 UPDATE products
6 SET list_price = ROUND(list_price * 1.10, 2)
7 WHERE category = 'accessories';
8
9 SELECT product_id, product_name, list_price
10 FROM products
11 WHERE category = 'accessories'
12 ORDER BY product_id;
```

- 1 **SET** assigns new values. Several columns can be set at once, separated by commas.
- 2 **WHERE** decides which rows change. A primary-key filter touches exactly one.
- 3 The new value may be computed from the old one: a 10% price rise.

RESULT · 2 rows

product_id	product_name	list_price
101	Laptop Stand	49.50
102	USB-C Hub	33.00

⚠ TRAP

Run the second **UPDATE** twice and prices rise 21%. Updates that depend on the current value are not idempotent.

UPDATE using another table

```
1 UPDATE customers AS c
2 SET   city      = s.city,
3       segment = s.segment
4 FROM   stg_customers AS s
5 WHERE  s.customer_id = c.customer_id;
6
7 SELECT customer_id, full_name, city, segment
8 FROM   customers
9 WHERE  customer_id IN (2, 6)
10 ORDER BY customer_id;
```

1 **FROM** brings in a second table to supply the new values.

2 The join condition. Without it, every customer would be updated from an arbitrary staging row.

⚡ Ben moves to Bristol and Farid becomes a business customer. Staging row 7 matches nobody and is ignored.

RESULT · 2 rows

customer_id	full_name	city	segment
2	Ben Carter	Bristol	business
6	Farid Khan	Leeds	business

DIALECT NOTE

PostgreSQL and Snowflake use **UPDATE ... FROM**. SQL Server writes **UPDATE c SET ... FROM customers c JOIN** MySQL writes **UPDATE c JOIN s ON ... SET**

DELETE removes rows

```
1 DELETE FROM orders
2 WHERE status = 'cancelled';
3
4 -- delete rows that match another table
5 DELETE FROM order_items AS oi
6 USING products AS p
7 WHERE p.product_id = oi.product_id
8      AND p.category = 'displays';
9
10 SELECT COUNT(*) AS remaining_lines
11 FROM order_items;
```

1

2

1 Same **WHERE** rules as **SELECT**. Rows for which it is true are removed.

2 **USING** joins a second table to decide what to delete: here, the three monitor lines.

RESULT · 1 row**remaining_lines**

11

DATA ENGINEERING LENS

Soft deletes

Warehouses rarely delete. They set **is_deleted = TRUE** and a **deleted_at** timestamp, so history stays auditable and downstream tables can react to the change.

A safe routine for changing data

An UPDATE or DELETE with no WHERE clause affects every row. The database will not ask whether you meant it.



★ PRO TIP

In PostgreSQL, add **RETURNING *** to an **UPDATE** or **DELETE** to see exactly which rows were changed.

Transactions: all or nothing

```
1 BEGIN;
2
3 UPDATE accounts
4 SET   balance = balance - 100
5 WHERE account_id = 1;
6
7 UPDATE accounts
8 SET   balance = balance + 100
9 WHERE account_id = 2;
10
11 COMMIT;           -- or ROLLBACK; to undo both
```

- 1 **BEGIN** opens a transaction. Changes after it are provisional and invisible to other sessions.
- 2 Money leaves one account...
- 3 ...and arrives in another. If the system fails between the two, neither happens.
- 4 **COMMIT** makes both permanent together. **ROLLBACK** discards both.

DATA ENGINEERING LENS

Pipelines use this constantly

"Delete yesterday's rows, insert the recalculated ones" must be one transaction. Otherwise a failure in between leaves the table empty for that day.

ACID: four guarantees of a transaction

A

Atomicity

All statements in the transaction take effect, or none do. There is no half-finished state.

C

Consistency

Every constraint holds before and after. A transaction cannot leave the data breaking its own rules.

I

Isolation

Concurrent transactions do not see each other's unfinished work.

D

Durability

Once committed, a change survives crashes and power loss.

WHY IT MATTERS

Why this matters beyond OLTP

Lakehouse table formats (Delta Lake, Iceberg) exist largely to bring ACID to files in object storage, so that a reader never sees a half-written batch.

Isolation levels

How much of other transactions' work can yours see while it runs? Stricter levels prevent more anomalies and allow less concurrency.

Level	Dirty read	Non-repeatable read	Phantom read	In practice
Read uncommitted	Possible	Possible	Possible	Rarely used. PostgreSQL treats it as read committed.
Read committed	Prevented	Possible	Possible	Default in PostgreSQL, SQL Server, Oracle and Snowflake.
Repeatable read	Prevented	Prevented	Possible	Default in MySQL. One consistent snapshot per transaction.
Serializable	Prevented	Prevented	Prevented	As if transactions ran one after another. May force retries.

 MENTAL MODEL

The three anomalies

Dirty read: seeing uncommitted data. **Non-repeatable read:** the same row differs when read twice. **Phantom:** the same query returns new rows the second time.

Upsert: insert, or update if it exists

```
1 INSERT INTO customers (customer_id, full_name, city, segment)
2 SELECT customer_id, full_name, city, segment
3 FROM stg_customers
4 ON CONFLICT (customer_id) DO UPDATE
5 SET city = EXCLUDED.city,
6     segment = EXCLUDED.segment;
7
8 SELECT customer_id, full_name, city, segment
9 FROM customers
10 WHERE customer_id IN (2, 6, 7)
11 ORDER BY customer_id;
```

RESULT · 3 rows

customer_id	full_name	city	segment
2	Ben Carter	Bristol	business
6	Farid Khan	Leeds	business
7	Gina Lopez	Porto	consumer

1 If the primary key already exists, do not fail: update instead.

2 **EXCLUDED** refers to the row that was about to be inserted.

⚡ Customers 2 and 6 existed and are updated. Customer 7 is new and is inserted.

DATA ENGINEERING LENS

Run it again and nothing changes: the upsert is **idempotent**. That is why it is the workhorse of incremental loads.

MERGE: the standard upsert

```
1 MERGE INTO customers AS t
2 USING stg_customers AS s
3     ON t.customer_id = s.customer_id
4 WHEN MATCHED THEN
5     UPDATE SET city = s.city,
6               segment = s.segment
7 WHEN NOT MATCHED THEN
8     INSERT (customer_id, full_name, city, segment)
9     VALUES (s.customer_id, s.full_name, s.city, s.segment);
```

RESULT · 3 rows

customer_id	full_name	city	segment
2	Ben Carter	Bristol	business
6	Farid Khan	Leeds	business
7	Gina Lopez	Porto	consumer

- 1 **USING** is the source of changes: a table or any query.
- 2 **ON** decides whether a source row matches a target row.
- 3 Match found: update the target.
- 4 No match: insert a new row.

⚠ TRAP

If two source rows match one target row, **MERGE** fails or picks one arbitrarily.
Deduplicate the source first.

Upsert across dialects

POSTGRESQL · SQLITE · DUCKDB

```
INSERT INTO t (id, val)
VALUES (1, 'a')
ON CONFLICT (id)
DO UPDATE
SET val = EXCLUDED.val;
```

Needs a unique constraint or primary key on the conflict column.

MYSQL

```
INSERT INTO t (id, val)
VALUES (1, 'a')
ON DUPLICATE KEY UPDATE
    val = VALUES(val);
```

Fires on any unique key. Newer versions prefer a row alias to `VALUES ()`.

SNOWFLAKE · BIGQUERY · SQL SERVER

```
MERGE INTO t
USING src AS s
    ON t.id = s.id
WHEN MATCHED THEN
    UPDATE SET val = s.val
WHEN NOT MATCHED THEN
    INSERT (id, val)
    VALUES (s.id, s.val);
```

Also PostgreSQL 15+, Oracle, Databricks. The portable choice for warehouses.

Bulk loading: files into tables

Row-by-row inserts pay overhead per row. Bulk commands read whole files in parallel.

POSTGRESQL

```
COPY raw_orders
FROM '/data/orders.csv'
WITH (FORMAT csv,
      HEADER true);
```

Server-side file. From a client machine, use `\copy` in psql.

SNOWFLAKE

```
COPY INTO raw_orders
FROM @landing/orders/
FILE_FORMAT = (
  TYPE = CSV
  SKIP_HEADER = 1);
```

Reads from a stage (cloud storage) and remembers which files it has already loaded.

BIGQUERY

```
LOAD DATA INTO
  shop.raw_orders
FROM FILES (
  format = 'CSV',
  skip_leading_rows = 1,
  uris = [
    'gs://lake/orders/*.csv']);
```

Loads every file matching the pattern from Cloud Storage.

DATA ENGINEERING LENS

Land files **as they are** in a raw table with a `loaded_at` column, then clean them in SQL. You can always replay from the original.

Exercise: a careful update

YOUR TURN

Mark every **shipped** order dated **before 15 April 2025** as **completed**. Do it so that you see exactly which rows changed, and so that you could still undo it.

TABLES YOU NEED

orders

Hint 1

Wrap the change in **BEGIN** and **COMMIT**.

Hint 2

PostgreSQL can hand back the changed rows with **RETURNING**.

EXPECTED OUTPUT

order_id	order_date	status
1008	2025-04-10	completed

Solution on the next slide. Try first.

Solution: a careful update

```
1  -- 1. preview: how many rows will change?
2  SELECT COUNT(*) FROM orders
3  WHERE status = 'shipped'
4      AND order_date < DATE '2025-04-15';
5
6  -- 2. change inside a transaction
7  BEGIN;
8
9  UPDATE orders
10 SET     status = 'completed'
11 WHERE  status = 'shipped'
12      AND order_date < DATE '2025-04-15'
13 RETURNING order_id, order_date, status;
14
15 -- 3. one row, as predicted
16 COMMIT;
```

- 1 The preview returns 1: only order 1008 qualifies.
- 2 The same **WHERE**, copied rather than retyped.
- 3 **RETURNING** shows the changed row before you commit.
- 4 Had it reported ten rows, you would type **ROLLBACK** instead.

Module 9 recap

- ✓ Always name the target columns in an **INSERT**.
- ✓ **INSERT ... SELECT** moves data between tables and matches columns by position.
- ✓ **UPDATE** and **DELETE** without **WHERE** affect every row. Preview with a **SELECT** first.
- ✓ A transaction makes several statements succeed or fail together.
- ✓ ACID: atomic, consistent, isolated, durable.
- ✓ **ON CONFLICT** and **MERGE** insert new keys and update existing ones in one statement.
- ✓ Deduplicate the source before a **MERGE**.
- ✓ Load files in bulk, untouched, into a raw table.

 NEXT UP**Module 10: data modelling for analytics**



MODULE 10

Data modelling

How you shape tables decides how easy, fast and trustworthy every later query will be.

BY THE END YOU CAN

- 1 Explain normalisation and when to reverse it
- 2 Tell facts from dimensions and design a star schema
- 3 State the grain of any table in one sentence
- 4 Track history with slowly changing dimensions
- 5 Describe the bronze, silver and gold layers

Normalisation in three steps

A recipe for removing repeated information from transactional tables. Each step fixes one kind of redundancy.

1NF

One value per cell

No lists inside a column and no repeating columns like `phone1`, `phone2`. Move repeats to their own rows.

Smell: `products = 'hub, mouse'`.

2NF

Depend on the whole key

With a composite key, every other column must depend on all of it.

Smell: `product_name` stored in `order_items`, whose key is `order + product`.

3NF

Depend on nothing but the key

No column should depend on another non-key column.

Smell: `customer_city` stored on every order.

MENTAL MODEL

One sentence to remember

Every non-key column must describe "the key, the whole key, and nothing but the key". The Brightcart tables are in third normal form.

Normalised for writing, dimensional for reading

Application databases and analytical warehouses optimise for opposite things, so they are modelled differently.

NORMALISED (3NF) · OLTP

- **Goal:** each fact stored once, so writes are small and safe.
- Many narrow tables linked by keys.
- A simple business question needs five or six joins.
- Stores the **current** state: an address is overwritten when it changes.
- Designed around application screens.

DIMENSIONAL · ANALYTICS

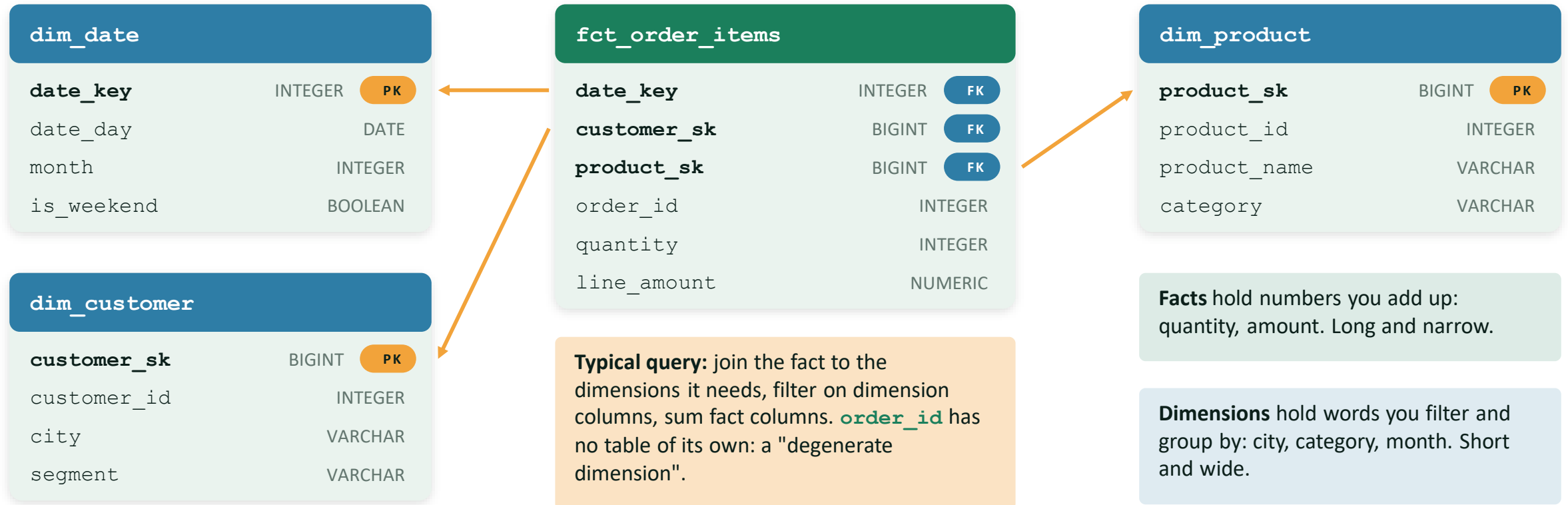
- **Goal:** questions are easy to ask and fast to answer.
- A few wide tables: facts in the middle, dimensions around.
- Most questions need one join per dimension.
- Keeps **history**: what was true when the order was placed.
- Designed around business processes: orders, payments, shipments.

DATA ENGINEERING LENS

Much of data engineering is this translation: take normalised source tables and rebuild them as facts and dimensions.

The star schema

One fact table of measurements in the centre, joined to descriptive dimension tables around it.



Grain comes first



**Before you add a single column, finish this sentence:
"One row in this table represents one ..."**

That statement is the grain. Every measure must be true at that level, and every join must respect it. Fan-out, double counting and unexplained totals are almost always grain mistakes.

FACT**fct_order_items**

One row per product line on an order.

FACT**fct_daily_revenue**

One row per calendar day.

DIMENSION**dim_customer**

One row per customer per version of their attributes.

Four kinds of fact table

Different business questions need facts recorded at different rhythms.

Type	One row is	Example	Typical measures	Rows change?
Transaction	One event at a point in time	An order line, a payment	Quantity, amount	Insert only
Periodic snapshot	One entity per period	Stock level per product per day	Balance, units on hand	One new set of rows each period
Accumulating snapshot	One process from start to end	An order: placed, paid, shipped	Dates of each milestone, durations	Updated as milestones occur
Factless	An event with no number	A student attended a class	Only the count of rows	Insert only



TRAP

Not every measure can be summed

Sales add up across days. A balance does not: summing daily stock levels over a month is meaningless. Such "semi-additive" measures are averaged or taken at period end.

Building a dimension with a surrogate key

```
1 CREATE TABLE dim_product (  
2     product_sk      BIGINT GENERATED ALWAYS AS IDENTITY  
3                     PRIMARY KEY,  
4     product_id      INTEGER NOT NULL,      -- business key  
5     product_name     VARCHAR(100) NOT NULL,  
6     category         VARCHAR(50) NOT NULL  
7 );  
8  
9 INSERT INTO dim_product (product_id, product_name, category)  
10 SELECT product_id, product_name, category  
11 FROM products;
```

- 1 A **surrogate key**: a meaningless number generated by the warehouse. Fact tables join on it.
- 2 The **business key** (or natural key) from the source system is kept as an ordinary column.
- 3 The identity column is not listed: the database fills it in.

🎯 WHY IT MATTERS

Why not join on the business key?

Source ids get reused, collide between systems, and cannot tell two historical versions of the same customer apart. Many teams use a hash of the business key, such as `MD5(source || '-' || id)`, so keys can be computed without a lookup.

Slowly changing dimensions: Type 1 and Type 2

Ben Carter moves from York to Leeds on 10 March. What should the customer dimension do?

TYPE 1 · OVERWRITE

dim_customer

customer_sk	customer_id	city
2	2	Leeds

- Update the row in place. York is gone.
- Simple, and correct for fixing typos.
- Rewrites history: orders Ben placed while living in York now report under Leeds.

TYPE 2 · ADD A VERSION

dim_customer

customer_sk	customer_id	city	valid_from	valid_to
2	2	York	2025-02-03	2025-03-10
3	2	Leeds	2025-03-10	9999-12-31

- Close the old row, insert a new one with a **new surrogate key**.
- Each version carries the period in which it was true.
- Old orders keep pointing at York; new orders at Leeds.

A Type 2 dimension, row by row

Four rows describe three customers. Ben has two versions; exactly one row per customer is current.

dim_customer

customer_sk	customer_id	city	segment	valid_from	valid_to	is_current
1	1	Madrid	consumer	2025-01-15	9999-12-31	true
2	2	York	business	2025-02-03	2025-03-10	false
3	2	Leeds	business	2025-03-10	9999-12-31	true
4	3	Lyon	consumer	2025-02-20	9999-12-31	true

valid_from is inclusive, **valid_to** is exclusive. Ben's York row ends on the day his Leeds row begins: no gap, no overlap.

9999-12-31 means "still valid". A far-future date keeps range joins simple; NULL would need special handling.

is_current is a shortcut: **WHERE is_current** gives the latest picture.

MENTAL MODEL

customer_id repeats across versions. **customer_sk** identifies one version, and is what a fact row stores.

Point-in-time join: the customer as they were

```
1 SELECT
2     o.order_id,
3     o.order_date,
4     d.customer_sk,
5     d.city AS city_at_order_time
6 FROM orders AS o
7 JOIN dim_customer AS d
8     ON d.customer_id = o.customer_id
9     AND o.order_date >= d.valid_from
10    AND o.order_date < d.valid_to
11 WHERE o.customer_id = 2
12 ORDER BY o.order_date;
```

DATA ENGINEERING LENS

Do this lookup once, when loading the fact, and store **customer_sk**. Analysts then join on a plain key.

1 Match on the business key...

2 ...and on the period in which that version was valid: a range join.

⚡ The order of 1 March finds the York version. Later orders find Leeds.

⚡ Half-open ranges guarantee each order matches exactly one version, so no fan-out.

RESULT · 3 rows

order_id	order_date	customer_sk	city_at_order_time
1002	2025-03-01	2	York
1005	2025-03-12	3	Leeds
1008	2025-04-10	3	Leeds

A date dimension, generated in SQL

```
1 CREATE TABLE dim_date AS
2 SELECT
3     d::date AS date_day,
4     EXTRACT(year FROM d)::int AS year,
5     EXTRACT(month FROM d)::int AS month,
6     EXTRACT(isodow FROM d)::int AS weekday,
7     EXTRACT(isodow FROM d) IN (6, 7) AS is_weekend
8 FROM generate_series(DATE '2025-01-01',
9                     DATE '2025-12-31',
10                    INTERVAL '1 day') AS t(d);
11
12 SELECT * FROM dim_date
13 WHERE date_day >= DATE '2025-03-01'
14 ORDER BY date_day LIMIT 3;
```

1 One row per day of the year: the grain of the table.

2 ISO weekday: Monday is 1, Sunday is 7.



Add fiscal periods, holidays and week numbers once, here, instead of recomputing them in every query.

RESULT · 3 rows

date_day	year	month	weekday	is_weekend
2025-03-01	2025	3	6	true
2025-03-02	2025	3	7	true
2025-03-03	2025	3	1	false

Medallion layers: bronze, silver, gold

A common way to organise a warehouse or lakehouse. Data becomes cleaner and more business-shaped at each step.



WHY IT MATTERS

Why keep the raw layer?

When a bug is found in a transformation, you fix the SQL and rebuild silver and gold from bronze. Without the original data, the only fix is to ask the source again, and it may have changed.

Modelling approaches at a glance

Approach	Shape	Strength	Cost	Common use
Third normal form	Many narrow, non-redundant tables	Integrity, small writes	Many joins to answer a question	Source systems, integration layer
Star schema (Kimball)	Facts surrounded by flat dimensions	Simple, fast, well understood by BI tools	Up-front design of grain and keys	Gold layer, reporting
Snowflake schema	Star with normalised dimensions	Less repetition in dimensions	More joins, harder to read	Very large dimensions
Data Vault	Hubs, links and satellites	Auditable history, many sources, parallel loads	Many tables, needs a mart on top	Enterprise integration layer
One big table	Fact with dimensions joined in	No joins at query time	Wide, repetitive, harder to change	A single dashboard or ML feature set

 RULE OF THUMB

These are layers more often than rivals: normalised or vault for integration, stars for presentation, one big table for a specific consumer.

Module 10 recap

- ✓ Normalisation stores each fact once; it suits writing, not analysis.
- ✓ Facts hold measures at a declared grain; dimensions hold descriptive context.
- ✓ State the grain first: "one row represents one ...".
- ✓ Surrogate keys shield the warehouse from source-system ids.
- ✓ Type 1 overwrites; Type 2 keeps versions with validity dates.
- ✓ A point-in-time join matches on the key and the validity range.
- ✓ A date dimension turns calendar logic into a simple join.
- ✓ Bronze keeps the raw truth, silver cleans, gold serves.

 NEXT UP

Module 11: pipeline patterns in SQL



MODULE 11

Pipeline patterns

The recurring SQL shapes that turn one-off queries into loads that run every night, survive failure and can be re-run safely.

BY THE END YOU CAN

- 1 Organise transformations into layers
- 2 Make every load idempotent
- 3 Choose between full refresh and incremental loads
- 4 Handle late data, change logs and history
- 5 Backfill without double counting

ETL and ELT

Same three letters, different order. Cheap, scalable warehouses moved the "T" inside the database, and into SQL.

ETL · TRANSFORM, THEN LOAD

- Extract from sources, transform on a **separate server**, load only the finished tables.
- The warehouse never sees raw data.
- Transformation logic lives in a proprietary tool or custom code.
- Fixing a bug means re-extracting from the source.
- Came from an era when warehouse compute was scarce.

ELT · LOAD, THEN TRANSFORM

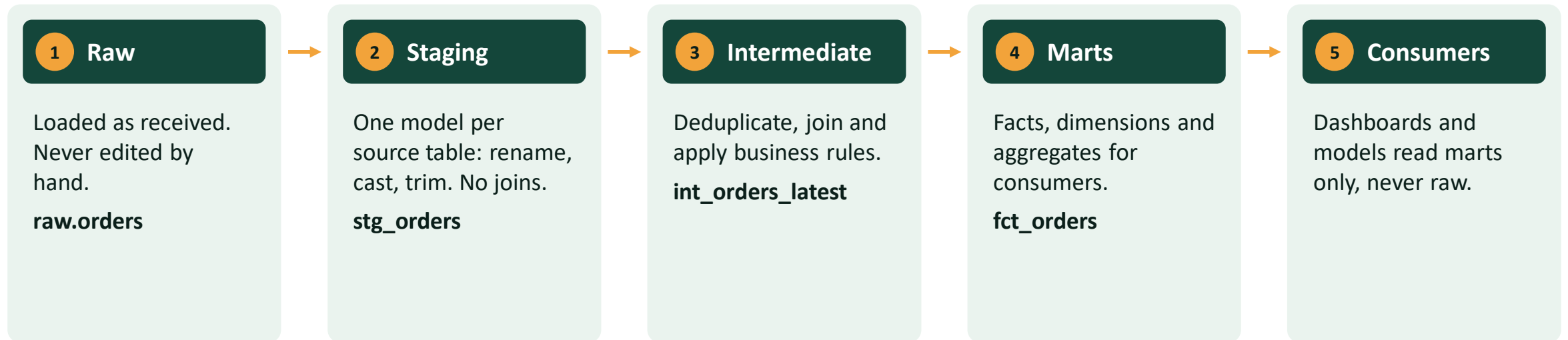
- Extract and load **raw** data first; transform **inside the warehouse** with SQL.
- Raw data is kept, so any transformation can be re-run.
- Logic is version-controlled SQL that anyone can read.
- Fixing a bug means editing a query and rebuilding.
- The default on Snowflake, BigQuery, Databricks and Redshift.

DATA ENGINEERING LENS

ELT is why SQL became the main language of data engineering: the heavy lifting moved to where SQL runs.

Transformations are organised in layers

Each layer has one job. A model reads only from the layer before it, which keeps dependencies simple.



RULE OF THUMB

Clean once, at the edge

If `status` needs trimming and lower-casing, do it in staging. Every model downstream then inherits the clean value, and the rule lives in one place.

A staging model: rename, cast, clean

```
1 CREATE OR REPLACE VIEW stg_orders AS
2 SELECT
3     CAST(order_id AS BIGINT) AS order_id,
4     CAST(customer_id AS BIGINT) AS customer_id,
5     CAST(order_date AS DATE) AS order_date,
6     LOWER(TRIM(status)) AS status,
7     CAST(amount AS NUMERIC(12,2)) AS amount,
8     updated_at,
9     loaded_at
10 FROM raw_orders;
```

- 1 A view: no storage, always reflects the raw table.
- 2 Explicit casts state the contract. If the source sends text where a number belongs, it fails here, loudly.
- 3 Normalise text once so **Completed** and **completed** cannot become two statuses.
- 4 Keep the metadata columns. Later layers need them to pick the latest version.

RULE OF THUMB

Staging models are one-to-one with source tables: same rows, better columns. No joins, no aggregation, no filtering.

Idempotency



Run it once or run it ten times: the table ends up exactly the same.

Schedulers retry. Engineers re-run after fixing a bug. Backfills replay whole months. A load that appends blindly doubles the data each time; an idempotent load cannot.

REPLACE

Overwrite the slice

Delete the rows for the period you are loading, then insert them again.

MERGE

Upsert by key

Insert new keys, update existing ones. A repeat run changes nothing.

REBUILD

Full refresh

Recreate the whole table from its inputs. Trivially idempotent.

The same load, fragile and idempotent

A daily job that summarises completed orders into daily_revenue. The scheduler retries it after a timeout.

✗ APPEND: DOUBLES ON RETRY

```
1 INSERT INTO daily_revenue
2 SELECT order_date,
3        COUNT(*), SUM(amount)
4 FROM orders
5 WHERE status = 'completed'
6 AND order_date = CURRENT_DATE - 1
7 GROUP BY order_date;
```

Two runs, two rows for the same day. And **CURRENT_DATE** means a re-run tomorrow loads a different day.

✓ REPLACE THE SLICE

```
1 BEGIN;
2 DELETE FROM daily_revenue
3 WHERE order_date = DATE '2025-03-01';
4
5 INSERT INTO daily_revenue
6 SELECT order_date,
7        COUNT(*), SUM(amount)
8 FROM orders
9 WHERE status = 'completed'
10 AND order_date = DATE '2025-03-01'
11 GROUP BY order_date;
12 COMMIT;
```

The date is a **parameter** supplied by the scheduler. Any number of runs leaves one correct row.

Full refresh or incremental?

Two ways to keep a table up to date. Use full refresh until it is too slow or too expensive: incremental logic is an optimisation, and a place for bugs to hide.

FULL REFRESH

```
1 CREATE OR REPLACE TABLE fct_orders AS
2 SELECT order_id, customer_id,
3        order_date, status, amount
4 FROM int_orders_latest;
```

- Rebuild everything on every run. (PostgreSQL has no **OR REPLACE** for tables: drop and recreate in a transaction.)
- **Simple and always correct:** deletes, late rows and logic changes are picked up automatically.
- Cost grows with total history, not with what changed.

INCREMENTAL

```
1 INSERT INTO fct_orders
2 SELECT order_id, customer_id,
3        order_date, status, amount
4 FROM int_orders_latest
5 WHERE updated_at > :last_run;
```

- Process only new or changed rows.
- **Fast and cheap** on large tables.
- You now own the hard cases: updates, deletes, late arrivals, and what happens when the logic changes.

Incremental loads with a high-water mark

```
1  INSERT INTO fct_orders
2      (order_id, customer_id, order_date,
3       status, amount, updated_at)
4  SELECT order_id, customer_id, order_date,
5         status, amount, updated_at
6  FROM stg_orders
7  WHERE updated_at > (
8      SELECT COALESCE(MAX(updated_at),
9                     TIMESTAMP '1900-01-01')
10     FROM fct_orders
11 );
```

- 1 Only rows newer than anything already loaded.
- 2 The **high-water mark** is read from the target itself, so there is no separate state to keep in sync.
- 3 On the first run the target is empty, **MAX** is NULL, and the fallback loads all history.

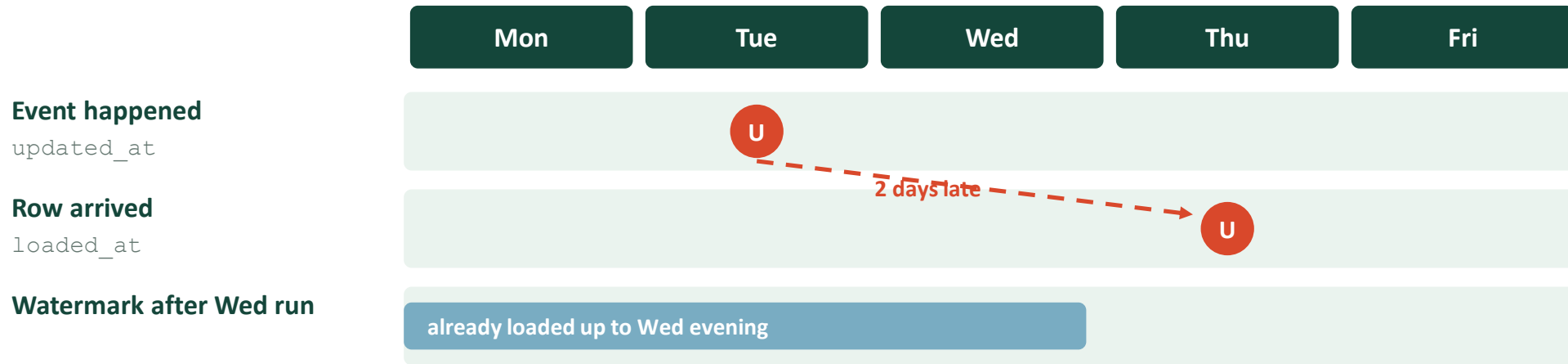
⚠ TRAP

Append-only is not enough

An order that changes from pending to completed arrives as a newer row and is inserted **again**. Appending suits immutable events. Mutable entities need a **MERGE** (two slides on).

Late-arriving data and the lookback window

An order is updated on Tuesday, but the source only delivers that change on Thursday. Will Thursday's run pick it up?



⚠️ STRICT WATERMARK

Missed for ever

Thursday asks for `updated_at` later than Wednesday evening. The late row says Tuesday, so it is skipped today and on every future run.

★ LOOKBACK WINDOW

Re-read a safety margin

Ask for `updated_at > watermark - INTERVAL '3 days'`. Recent days are processed again and a **MERGE** keeps the reprocessing free of duplicates.

Incremental MERGE: updates without duplicates

```
1  MERGE INTO fct_orders AS t
2  USING (
3      SELECT order_id, customer_id, order_date,
4              status, amount, updated_at
5      FROM int_orders_latest           -- one row per order
6      WHERE updated_at > :watermark - INTERVAL '3 days'
7  ) AS s
8      ON t.order_id = s.order_id
9  WHEN MATCHED AND s.updated_at > t.updated_at THEN
10     UPDATE SET status      = s.status,
11               amount       = s.amount,
12               updated_at   = s.updated_at
13  WHEN NOT MATCHED THEN
14     INSERT (order_id, customer_id, order_date,
15            status, amount, updated_at)
16     VALUES (s.order_id, s.customer_id, s.order_date,
17            s.status, s.amount, s.updated_at);
```

- 1 The source is deduplicated first: at most one row per key.
- 2 Watermark with a lookback. `:watermark` is a parameter.
- 3 Update only if the incoming version is **newer**, so an old replay cannot overwrite fresh data.
- 4 New orders are inserted.

Change data capture: a log of changes

CDC tools read the source database's transaction log and emit one row per change. You receive events, not the current state.

customer_changes

op	customer_id	city	segment	changed_at
I	7	Porto	consumer	2025-05-01 09:00
U	2	Bristol	business	2025-05-01 09:05
U	2	Bath	business	2025-05-01 09:40
D	4	NULL	NULL	2025-05-01 10:10

op is the operation: Insert, **U**ppdate or **D**elete.

Customer 2 changed twice in this batch. Only the last change (Bath) should reach the target.

Deletes are visible. A plain "select what changed" extract can never tell you a row was removed.

MENTAL MODEL

Two steps to apply a batch

First reduce the batch to the **latest change per key** with **ROW_NUMBER**. Then **MERGE** that into the target, treating D as a delete.

Applying a change log with MERGE

```
1  MERGE INTO customers AS t
2  USING (
3      SELECT * FROM (
4          SELECT c.*, ROW_NUMBER() OVER (
5              PARTITION BY customer_id
6              ORDER BY changed_at DESC) AS rn
7          FROM customer_changes AS c
8      ) AS ranked
9      WHERE rn = 1
10 ) AS s
11 ON t.customer_id = s.customer_id
12 WHEN MATCHED AND s.op = 'D' THEN
13     DELETE
14 WHEN MATCHED THEN
15     UPDATE SET city = s.city, segment = s.segment
16 WHEN NOT MATCHED AND s.op <> 'D' THEN
17     INSERT (customer_id, city, segment)
18     VALUES (s.customer_id, s.city, s.segment);
```

- 1 Keep the newest change per customer: the deduplication pattern from Module 7.
- 2 A delete event removes the target row (or sets `is_deleted` for a soft delete).
- 3 Any other match is an update.
- 4 Insert unseen keys, unless the last thing that happened to them was a delete.

Loading a Type 2 dimension, step 1: close

```
1 UPDATE dim_customer AS d
2 SET   valid_to   = DATE '2025-05-01',
3       is_current = FALSE
4 FROM   stg_customers AS s
5 WHERE  d.customer_id = s.customer_id
6       AND d.is_current
7       AND (d.city   IS DISTINCT FROM s.city
8            OR d.segment IS DISTINCT FROM s.segment);
```

- 1 Staging holds the latest attributes from the source.
- 2 Look only at current versions.
- 3 **IS DISTINCT FROM** is a NULL-safe "not equal". Staging says Bristol, the dimension says Leeds: a change.
- 4 End the old version on the load date. Unchanged customers are untouched.

dim_customer, CUSTOMER 2, AFTER STEP 1

customer_sk	city	valid_from	valid_to	is_current
2	York	2025-02-03	2025-03-10	false
3	Leeds	2025-03-10	2025-05-01	false

Loading a Type 2 dimension, step 2: insert

```
1 INSERT INTO dim_customer
2     (customer_sk, customer_id, city, segment,
3      valid_from, valid_to, is_current)
4 SELECT
5     (SELECT MAX(customer_sk) FROM dim_customer)
6     + ROW_NUMBER() OVER (ORDER BY s.customer_id),
7     s.customer_id, s.city, s.segment,
8     DATE '2025-05-01', DATE '9999-12-31', TRUE
9 FROM stg_customers AS s
10 LEFT JOIN dim_customer AS d
11     ON d.customer_id = s.customer_id AND d.is_current
12 WHERE d.customer_id IS NULL;
```

- 1 Find staging customers with **no current row**: brand-new customers, and the ones just closed in step 1.
 - 2 Continue the surrogate key sequence.
 - 3 The new version starts on the load date and is open-ended.
-  Run both steps in one transaction. A re-run is safe: nothing differs any more, so nothing is closed or inserted.

dim_customer, CUSTOMER 2, AFTER STEP 2

customer_sk	city	valid_from	valid_to	is_current
2	York	2025-02-03	2025-03-10	false
3	Leeds	2025-03-10	2025-05-01	false
5	Bristol	2025-05-01	9999-12-31	true

Backfills: reloading history safely

A backfill re-runs a pipeline over past dates: after a bug fix, a new column, or a source correction.

1

Parameterise by date

The job takes a logical date or a date range. It never calls `CURRENT_DATE`.

2

Replace, never append

Each run deletes or overwrites its own slice. Running March twice still gives one March.

3

Work in chunks

Reload a day or a month at a time. A failure costs one chunk, not the whole history.

4

Validate, then swap

Build into a new table, compare counts and totals with the old one, then switch names.

🔗 WHY IT MATTERS

Backfills are where idempotency pays for itself

If every daily run is "replace this day's slice", a backfill is the same job in a loop over dates. No special code, no manual cleanup.

How dbt expresses these patterns

```
models/marts/fct_orders.sql

1  {{ config(
2      materialized = 'incremental',
3      unique_key   = 'order_id'
4  ) }}
5
6  SELECT order_id, customer_id, order_date,
7         status, amount, updated_at
8  FROM {{ ref('int_orders_latest') }}
9
10 {% if is_incremental() %}
11 WHERE updated_at > (
12     SELECT MAX(updated_at) FROM {{ this }}
13 )
14 {% endif %}
```

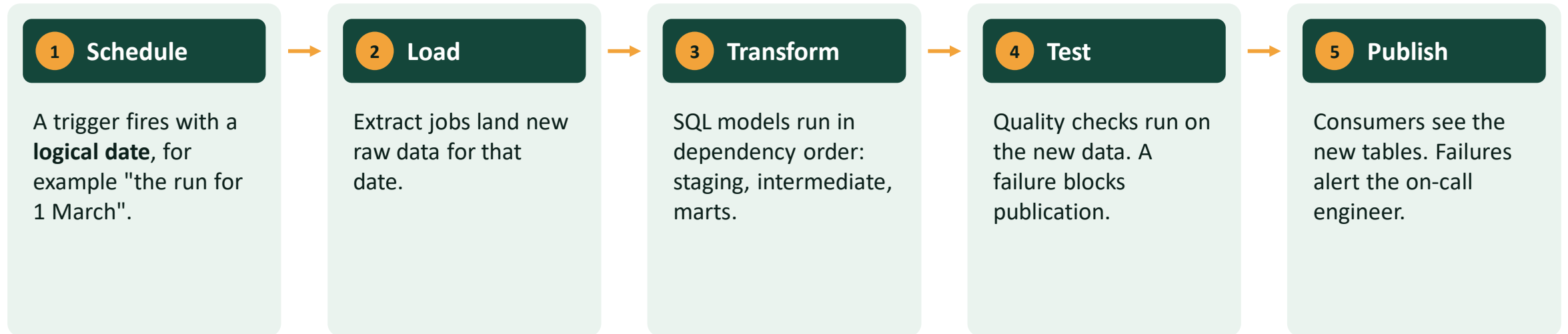
- 1 You write a **SELECT**. dbt wraps it in the **CREATE TABLE AS** or **MERGE**.
- 2 With a **unique_key**, existing rows are updated instead of duplicated.
- 3 **ref()** names another model. dbt works out the run order from these references.
- 4 This filter applies only on incremental runs. A full refresh skips it.

MENTAL MODEL

Tools change the packaging, not the ideas. Everything on this slide is a pattern you have already written by hand.

Where SQL runs: orchestration

An orchestrator such as Airflow or Dagster decides when each step runs, in what order, and what happens on failure.



DATA ENGINEERING LENS

The orchestrator passes the logical date into your SQL as a parameter. That single habit makes retries and backfills safe.

Exercise: a clean current-orders table

YOUR TURN

From `raw_orders`, produce **one row per order** holding its **latest version**, and drop any order whose customer does not exist in `customers`. Return order id, customer id, status and amount.

TABLES YOU NEED

`raw_orders``customers`

Hint 1

Latest version per key: `ROW_NUMBER` partitioned by order, newest first.

Hint 2

To keep only orders with a real customer, use `EXISTS` or an inner join.

EXPECTED OUTPUT

order_id	customer_id	status	amount
1001	1	completed	135.00
1002	2	completed	220.00
1003	1	completed	55.00
1004	3	cancelled	90.00

Solution on the next slide. Try first.

Solution: a clean current-orders table

```
1 WITH ranked AS (  
2     SELECT *,  
3         ROW_NUMBER() OVER (  
4             PARTITION BY order_id  
5             ORDER BY updated_at DESC, loaded_at DESC  
6         ) AS rn  
7     FROM raw_orders  
8 )  
9 SELECT r.order_id, r.customer_id, r.status, r.amount  
10 FROM ranked AS r  
11 WHERE r.rn = 1  
12     AND EXISTS (  
13         SELECT 1 FROM customers AS c  
14         WHERE c.customer_id = r.customer_id  
15     )  
16 ORDER BY r.order_id;
```

- 1 Nine raw rows are numbered within their order.
- 2 Five latest versions remain.
- 3 Order 1012 belongs to customer 9, who does not exist, so it is dropped. Four rows are returned.

DATA ENGINEERING LENS

In production, do not silently drop the orphan. Route it to a quarantine table and alert, so someone asks why customer 9 is missing.

Module 11 recap

- ✓ ELT loads raw data first and transforms it in the warehouse with SQL.
- ✓ Layer your models: raw, staging, intermediate, marts.
- ✓ Every load must be idempotent: replace a slice, merge by key, or rebuild.
- ✓ Pass dates in as parameters; never rely on `CURRENT_DATE`.
- ✓ Prefer full refresh until scale forces incremental.
- ✓ Use a lookback window plus `MERGE` to absorb late-arriving rows.
- ✓ Reduce a change log to the latest change per key before applying it.
- ✓ A Type 2 load closes changed versions, then inserts new ones.

 NEXT UP**Module 12: data quality**



MODULE 12

Data quality

A pipeline that runs without errors can still deliver wrong numbers. Tests are how you find out before your users do.

BY THE END YOU CAN

- 1 Name the dimensions of data quality
- 2 Write tests as queries that return failing rows
- 3 Check uniqueness, completeness and validity
- 4 Reconcile a target against its source
- 5 Detect stale data and unusual volumes

Six dimensions of data quality

Each dimension is a question you can turn into a SQL test.

1

Uniqueness

Is there exactly one row per key?

Test: group by the key, look for counts above 1.

2

Completeness

Are required values present? Did every expected row arrive?

Test: count NULLs, compare row counts.

3

Validity

Do values obey their rules: allowed lists, ranges, formats?

Test: select rows that break the rule.

4

Consistency

Do related tables agree? Does every foreign key have a parent?

Test: anti-join.

5

Timeliness

Is the data fresh enough to act on?

Test: compare the latest timestamp with now.

6

Accuracy

Does it match reality, or at least the source of record?

Test: reconcile totals.

What a data test is



**A test is a query that returns the rows that are wrong.
Zero rows means pass.**

This one convention makes every check look the same, so any query can be scheduled, counted and alerted on. When it fails, the result is already the list of rows to investigate.

WRITE

Select the violations

Not "count the good rows" but "show me the bad ones".

RUN

After every load

Before the data is published to consumers.

ACT

Fail or warn

Critical tests stop the pipeline. Minor ones raise a warning.

Uniqueness: is there one row per key?

```
1 SELECT
2     order_id,
3     COUNT(*) AS n_rows
4 FROM raw_orders
5 GROUP BY order_id
6 HAVING COUNT(*) > 1
7 ORDER BY order_id;
```

RESULT · 4 rows

order_id	n_rows
1001	2
1002	2
1003	2
1004	2

1 Group by the column that is supposed to be unique.

2 Keep only keys that occur more than once. Any row returned is a failure.



Four orders appear twice in the raw table. Expected there; a serious bug if it happened in `fct_orders`.



DATA ENGINEERING LENS

The first test on every table

Almost every model has a grain, and so a key that must be unique. A duplicate key upstream becomes fan-out in every join downstream.

Completeness and validity checks

```
1 SELECT 'customers: email is null' AS check_name,
2     COUNT(*) AS failures
3 FROM customers WHERE email IS NULL
4 UNION ALL
5 SELECT 'customers: city is null', COUNT(*)
6 FROM customers WHERE city IS NULL
7 UNION ALL
8 SELECT 'orders: unknown status', COUNT(*)
9 FROM orders
10 WHERE status NOT IN ('pending', 'completed',
11                     'shipped', 'cancelled')
12 UNION ALL
13 SELECT 'orders: negative amount', COUNT(*)
14 FROM orders WHERE amount < 0;
```

1

2

3

- 1 Each branch is one check, labelled so the output reads as a report.
 - 2 **UNION ALL** stacks the checks into one result.
 - 3 An "accepted values" test: any status outside the list is a failure.
- ★ Decide per check whether a non-zero count is an error. A missing city may be fine; a missing order id never is.

RESULT · 4 rows

check_name	failures
customers: email is null	1
customers: city is null	1
orders: unknown status	0
orders: negative amount	0

Referential integrity: finding orphans

```
1 SELECT
2     r.order_id,
3     r.customer_id,
4     r.amount
5 FROM raw_orders AS r
6 LEFT JOIN customers AS c
7     ON c.customer_id = r.customer_id
8 WHERE c.customer_id IS NULL;
```

1

2

1 An anti-join: keep orders, attach their customer where one exists.

2 Rows with no customer are orphans.



Order 1012 points at customer 9. An inner join would silently drop its 40.00 from every revenue report.

RESULT · 1 row

order_id	customer_id	amount
1012	9	40.00



DATA ENGINEERING LENS

Why orphans happen

Tables are extracted at different moments, so an order can arrive before its customer. Decide the policy: wait, quarantine, or attach an "unknown" dimension row.

Reconciliation: do two tables agree?

```
1 SELECT 'orders'      AS source,  
2       COUNT(*)      AS n_orders,  
3       SUM(amount)    AS total  
4 FROM orders  
5 UNION ALL  
6 SELECT 'order_items',  
7       COUNT(DISTINCT order_id),  
8       SUM(quantity * unit_price)  
9 FROM order_items;
```

1

2

1

Reduce each table to a few control totals: row count and sum of a key measure.

2

Compare at the same grain: distinct orders, not lines.



Both report 10 orders and 1,325.00. Header and lines reconcile.

RESULT · 2 rows

source	n_orders	total
orders	10	1325.00
order_items	10	1325.00



DATA ENGINEERING LENS

Use it at every hand-off

Source versus raw, raw versus staging, staging versus mart. When totals stop matching, the mismatch tells you which step lost or invented rows.

Freshness: is the data stale?

```
1 SELECT
2     MAX(loaded_at)           AS last_load,
3     TIMESTAMP '2025-03-10 09:00'
4     - MAX(loaded_at)       AS lag,
5     TIMESTAMP '2025-03-10 09:00'
6     - MAX(loaded_at) > INTERVAL '12 hours' AS is_stale
7 FROM raw_orders;
```

RESULT · 1 row

last_load	lag	is_stale
2025-03-09 14:00	19:00:00	true

- 1 The most recent load time is the simplest freshness signal.
- 2 In production this literal is `CURRENT_TIMESTAMP`. It is fixed here so the example is repeatable.
- 3 Compare the lag with the agreed limit. Nineteen hours against a twelve-hour limit: stale.

🎯 WHY IT MATTERS

A dashboard showing yesterday's numbers as if they were today's is worse than one that is down. Freshness checks catch the silent failure: a load that simply did not happen.

Diffing two tables with EXCEPT

```
1 SELECT customer_id, full_name, city, segment
2 FROM stg_customers
3 EXCEPT
4 SELECT customer_id, full_name, city, segment
5 FROM customers
6 ORDER BY customer_id;
```

1

RESULT · 3 rows

customer_id	full_name	city	segment
2	Ben Carter	Bristol	business
6	Farid Khan	Leeds	business
7	Gina Lopez	Porto	consumer

1 Rows in the first query that have no identical twin in the second.



All three staging rows differ from the target: Ben's city, Farid's segment, and Gina, who is new.



Swap the two queries to see the other direction. An empty result both ways means the tables are identical.

DATA ENGINEERING LENS

Refactor with confidence

Rewriting a slow model? Build the new version beside the old one and diff them. Zero rows both ways proves the rewrite changed nothing.

Volume monitoring with a window

```
1 WITH daily AS (  
2     SELECT loaded_at::date AS load_date,  
3           COUNT(*)        AS n_rows  
4     FROM raw_orders  
5     GROUP BY 1  
6 )  
7 SELECT  
8     load_date,  
9     n_rows,  
10    ROUND(AVG(n_rows) OVER (  
11        ORDER BY load_date  
12        ROWS BETWEEN 3 PRECEDING AND 1 PRECEDING  
13    ), 2) AS avg_prev_3  
14 FROM daily  
15 ORDER BY load_date;
```

1

2

- 1 How many rows arrived on each load date.
 - 2 The average of the three **previous** loads, excluding today.
-  Alert when today is far from that baseline, for example below half of it. A sudden drop usually means a partial load.

RESULT · 7 rows

load_date	n_rows	avg_prev_3
2025-03-01	2	NULL
2025-03-02	2	2.00
2025-03-05	1	2.00
2025-03-06	1	1.67
2025-03-07	1	1.33
2025-03-08	1	1.00
2025-03-09	1	1.00

Where tests live

Mechanism	Checks	Runs	Good for
Database constraints	NOT NULL, UNIQUE, CHECK, foreign keys	At write time; the write fails	OLTP and any engine that enforces them
dbt tests	unique, not_null, accepted_values, relationships, custom SQL	After each model builds	Warehouse transformations
Quality frameworks	Declarative expectations, profiling, anomaly detection	As a pipeline step	Great Expectations, Soda and similar tools
Reconciliation jobs	Counts and totals between systems	After loads, on a schedule	Source-to-target accuracy
Observability monitors	Freshness, volume and schema changes	Continuously	Catching what nobody wrote a test for

 RULE OF THUMB

A minimum for every model

Its key is unique and not NULL, its foreign keys resolve, and its row count is plausible. Those three tests catch most incidents.

Module 12 recap

- ✓ Quality has dimensions: unique, complete, valid, consistent, timely, accurate.
- ✓ Write each test as a query that returns failing rows.
- ✓ `GROUP BY key HAVING COUNT(*) > 1` finds duplicates.
- ✓ An anti-join finds foreign keys with no parent.
- ✓ Reconcile counts and totals at every hand-off between layers.
- ✓ Compare `MAX(loaded_at)` with the clock to detect stale data.
- ✓ `EXCEPT` in both directions proves two tables are identical.
- ✓ Test before publishing, and let critical failures stop the pipeline.

 NEXT UP**Module 13: performance**



MODULE 13

Performance

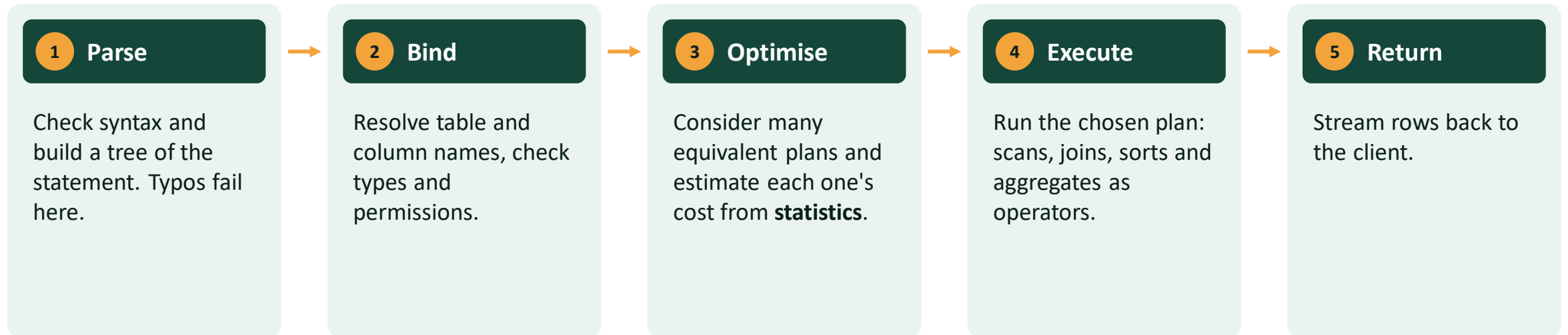
Correct first, then fast. Understand how the engine executes a query and the levers that reduce the work it does.

BY THE END YOU CAN

- 1 Read a query plan with EXPLAIN
- 2 Explain what an index is and when it helps
- 3 Write filters the engine can use efficiently
- 4 Describe columnar storage and partition pruning
- 5 Recognise the common performance anti-patterns

What happens when you press Run

Your SQL text passes through five stages. Performance work is mostly about helping stage three choose well.



MENTAL MODEL

The optimizer is a planner, not a mind reader

It can reorder joins and pick algorithms, but only within what your SQL allows and what its statistics tell it. Stale statistics or filters it cannot see through lead to bad plans.

EXPLAIN shows the plan

Illustrative PostgreSQL output on a table of one million orders. Same query, before and after adding an index.

✗ SEQUENTIAL SCAN

```
1  EXPLAIN ANALYZE
2  SELECT * FROM orders
3  WHERE customer_id = 2;
4
5  Seq Scan on orders
6    (cost=0.00..18334.00 rows=98)
7    (actual time=0.02..74.51 rows=103)
8    Filter: (customer_id = 2)
9    Rows Removed by Filter: 999897
10 Execution Time: 74.55 ms
```

Every row is read; 999,897 are thrown away to keep 103.

✓ INDEX SCAN

```
1  CREATE INDEX idx_orders_customer
2      ON orders (customer_id);
3
4  -- the same query now reports:
5  Index Scan using idx_orders_customer
6    (cost=0.42..12.15 rows=98)
7    (actual time=0.03..0.11 rows=103)
8    Index Cond: (customer_id = 2)
9  Execution Time: 0.14 ms
```

The engine jumps straight to the matching entries. Roughly 500 times faster.

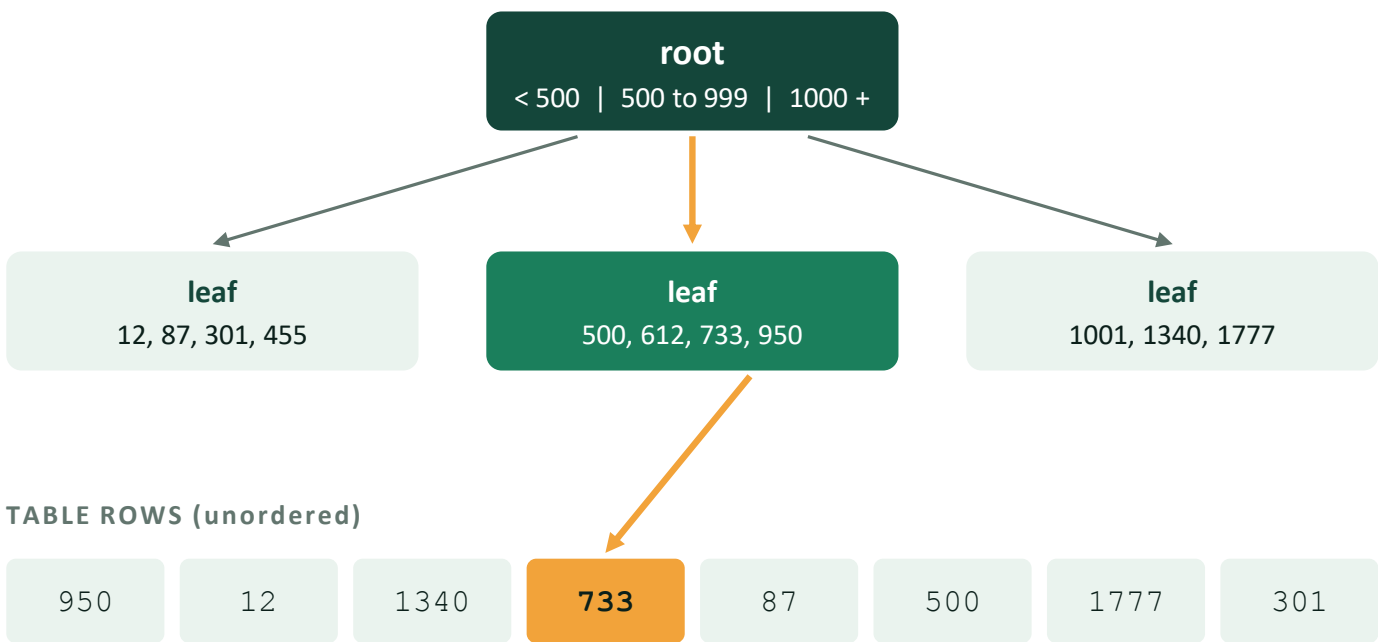
Reading one line of a plan

EXPLAIN shows estimates. **EXPLAIN ANALYZE** also runs the query and adds what really happened.

<code>Seq Scan on orders</code>	→	The operator. Here: read the whole table, row after row.
<code>cost=0.00..18334.00</code>	→	Estimated cost in arbitrary units: before the first row, and in total. Only useful for comparing plans.
<code>rows=98</code>	→	Estimated rows this step will produce, from table statistics.
<code>actual time=0.02..74.51</code>	→	Measured time in milliseconds. Present only with ANALYZE .
<code>rows=103 loops=1</code>	→	Actual rows. A large gap from the estimate means stale statistics, and usually a poor plan.
<code>Rows Removed by Filter: 999897</code>	→	Wasted work. Rows that were read and then discarded. Read plans from the most indented line outward.

How a B-tree index finds a row

An index is a sorted copy of one or more columns, organised as a shallow tree, with a pointer from each entry to its table row.

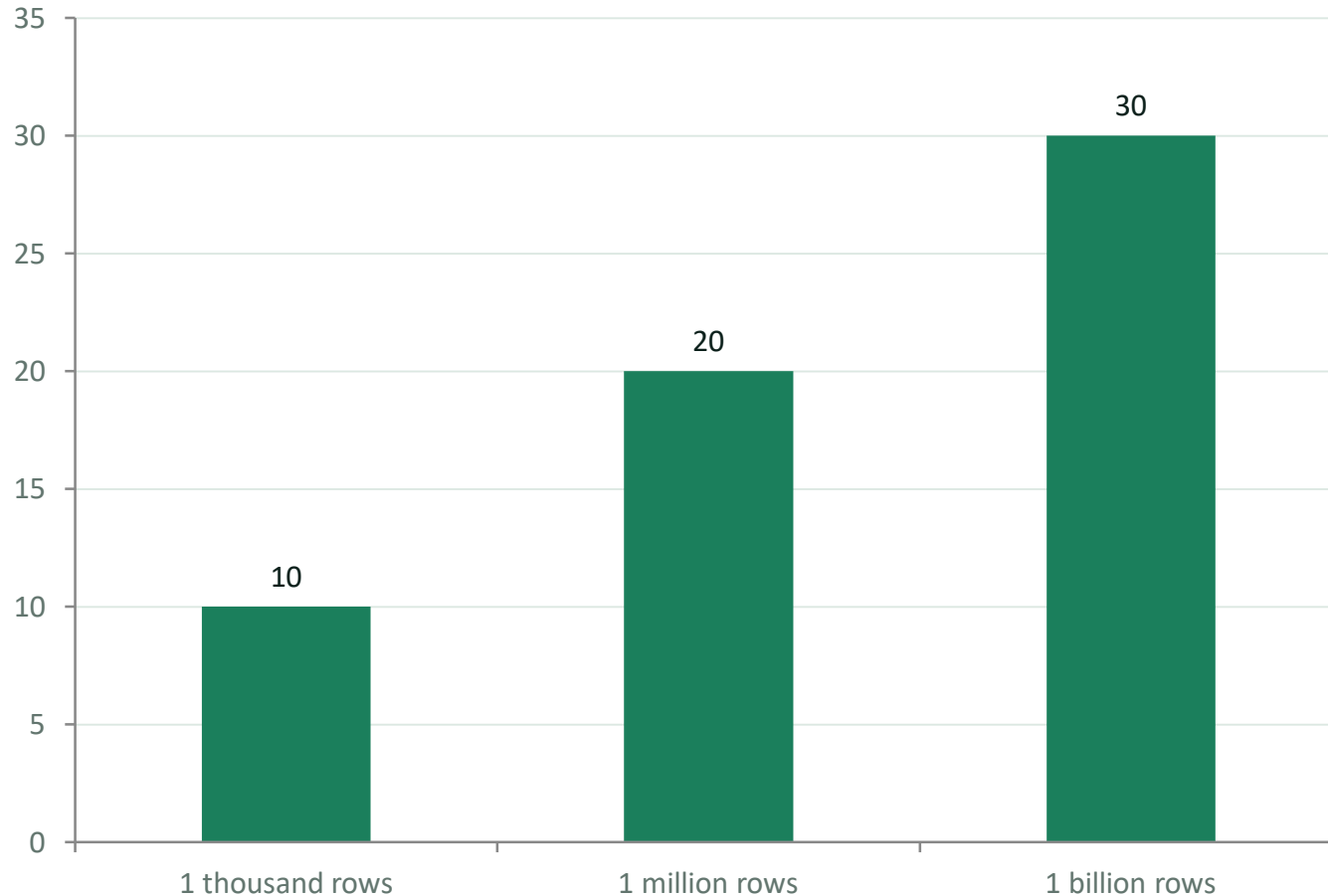


`WHERE order_id = 733`

- 1 **Root.** 733 lies between 500 and 999, so follow the middle branch.
 - 2 **Leaf.** Keys are sorted. Find 733 and read the row location stored beside it.
 - 3 **Table.** Fetch that single row directly.
- ⚡ Three small reads instead of scanning every row. A real B-tree holds hundreds of keys per node, so even a billion rows need only three or four levels.

Why sorted structures scale

Steps to find one key by halving a sorted list ($\log_2$ of the row count)



- 1 Each step halves the remaining search space, so a thousand times more data costs only **ten more steps**.
- 2 A full scan grows in proportion to the table: a thousand times more rows means a thousand times more work.
- 3 This is why a lookup by key on an indexed table stays fast as the table grows, and why the same lookup without an index does not.

MENTAL MODEL

An index trades extra storage and slower writes for reads that barely notice the table's size.

Creating indexes

```
1  -- single column: lookups by customer
2  CREATE INDEX idx_orders_customer
3      ON orders (customer_id);
4
5  -- composite: customer first, then date
6  CREATE INDEX idx_orders_customer_date
7      ON orders (customer_id, order_date);
8
9  -- unique: an index that is also a rule
10 CREATE UNIQUE INDEX idx_customers_email
11     ON customers (email);
12
13 -- partial (PostgreSQL): index only some rows
14 CREATE INDEX idx_orders_pending
15     ON orders (order_date)
16     WHERE status = 'pending';
```

- 1 Index the columns you filter and join on. Primary keys are indexed automatically.
- 2 Column order matters: see the next slide.
- 3 Enforces uniqueness and speeds lookups at once.
- 4 A small index over just the rows a hot query needs.

★ PRO TIP

Foreign-key columns are **not** indexed automatically in PostgreSQL. Index them: joins and deletes on the parent depend on it.

Composite indexes: the leading-column rule

An index on (customer_id, order_date) is sorted by customer first, then by date within each customer.

Query filter	Index used?	Why
WHERE customer_id = 2	Yes	The leading column is constrained.
WHERE customer_id = 2 AND order_date >= '2025-04-01'	Yes, fully	Equality on the first column, then a range on the second.
WHERE customer_id = 2 ORDER BY order_date	Yes, no sort	Entries for one customer are already in date order.
WHERE order_date >= '2025-04-01'	No	Dates are scattered across every customer. The index order does not help.

 MENTAL MODEL

Think of a phone book

Sorted by surname, then first name. Finding "Carter" is instant, and so is "Carter, Ben". Finding everyone called "Ben" means reading the whole book.

Write filters the engine can search

A predicate is "sargable" (search-argument-able) when the bare column is compared with a value. Wrap the column in a function and the index or partition is ignored.

✗ FORCES A FULL SCAN

```
1  -- function around the column
2  WHERE EXTRACT(year FROM order_date) = 2025
3
4  WHERE LOWER(email) = 'ada@mail.com'
5
6  -- arithmetic on the column
7  WHERE amount * 1.2 > 120
8
9  -- leading wildcard
10 WHERE full_name LIKE '%son'
```

✓ CAN USE AN INDEX

```
1  -- a range on the bare column
2  WHERE order_date >= DATE '2025-01-01'
3      AND order_date < DATE '2026-01-01'
4
5  -- store emails already lower-cased
6  WHERE email = 'ada@mail.com'
7
8  -- move the maths to the constant
9  WHERE amount > 100
10
11 -- anchor the pattern at the start
12 WHERE full_name LIKE 'Ada%'
```

When an index does not help

Indexes are not free, and not universal.

1

Every write pays

Each **INSERT**, **UPDATE** and **DELETE** must also maintain every index on the table. Ten indexes, ten extra writes.

2

Low selectivity

An index on **status** with three values points at a third of the table. Scanning is cheaper than a million jumps.

3

Small tables

A few thousand rows fit in memory. A scan beats the overhead of the tree.

4

Analytical scans

Queries that aggregate most of a table need to read it anyway. An index cannot shrink "all of it".

DATA ENGINEERING LENS

Warehouses mostly do without B-trees

Snowflake, BigQuery and Redshift have no conventional indexes. They get speed from columnar storage, partitioning and clustering, which skip large blocks of data instead of locating single rows.

Three ways to execute a join

The optimizer picks one per join based on table sizes, available indexes and sort order.

NESTED LOOP

For each row, look up matches

For every row of the outer table, probe the inner table, ideally through an index.

Best when one side is small and the other is indexed.

HASH JOIN

Build a lookup, then probe

Load the smaller table into an in-memory hash table keyed on the join column, then stream the larger one past it.

Best when joining large unsorted tables on equality. The warehouse default.

MERGE JOIN

Zip two sorted lists

Sort both sides on the key, or use existing order, then walk them together.

Best when both inputs are already sorted.

★ PRO TIP

If a plan shows a nested loop over two big tables, an index is missing or the row estimates are badly wrong.

Row stores and column stores

The same table laid out on disk in two ways. The query: `SELECT SUM(amount) FROM orders.`

ROW STORE · OLTP

1001	1	03-01	135.00
1002	2	03-01	220.00
1003	1	03-05	55.00
1004	3	03-07	90.00

Values of one row sit together. Perfect for "fetch order 1003". But to sum one column, every row must be read in full, including the three columns we do not need.

COLUMN STORE · OLAP

1001	1	03-01	135.00
1002	2	03-01	220.00
1003	1	03-05	55.00
1004	3	03-07	90.00
order_id	customer_id	order_date	amount

Values of one column sit together. The sum reads a quarter of the data and skips the rest. Similar values side by side also compress far better.

Partitioning splits a big table by a key

```
PostgreSQL

1 CREATE TABLE events (
2     event_id BIGINT,
3     user_id BIGINT,
4     event_ts TIMESTAMPTZ NOT NULL,
5     payload JSONB
6 ) PARTITION BY RANGE (event_ts);
7
8 CREATE TABLE events_2025_03
9     PARTITION OF events
10     FOR VALUES FROM ('2025-03-01')
11     TO ('2025-04-01');
```

- 1 The table is divided by a column, almost always a date. Queries still address **events** as one table.
- 2 Each partition is stored separately and covers one range: here, March 2025.



Old partitions can be dropped or archived instantly, without a slow **DELETE**.



DIALECT NOTE

BigQuery: **PARTITION BY DATE(event_ts) CLUSTER BY user_id**. Snowflake partitions automatically; you guide it with **CLUSTER BY**. Databricks: **PARTITIONED BY** or liquid clustering.

Partition pruning skips data you did not ask for

A year of events in twelve monthly partitions. How much does each query have to read?

✓ PRUNED: 1 OF 12 PARTITIONS READ

```
WHERE event_ts >= '2025-03-01'
AND event_ts < '2025-04-01'
```

The filter is a plain range on the partition column. The planner proves eleven partitions cannot contain matches and **never opens them**.

Jan

Feb

Mar

Apr

May

Jun

Jul

Aug

Sep

Oct

Nov

Dec

✗ NOT PRUNED: ALL 12 READ

```
WHERE EXTRACT(month FROM event_ts) = 3
AND EXTRACT(year FROM event_ts) = 2025
```

Same rows wanted, but the column is hidden inside a function. Many engines cannot map that back to partitions, so they **scan everything**.

Jan

Feb

Mar

Apr

May

Jun

Jul

Aug

Sep

Oct

Nov

Dec

Distributed engines: four words to know

Snowflake, BigQuery, Spark and Redshift split work across many machines. That adds a new cost: moving data between them.

1

Shuffle

To join or group, rows with the same key must meet on the same machine. Redistributing them over the network is often the slowest step.

2

Skew

If one key holds a large share of the rows (a NULL customer, one giant client), a single worker gets most of the work while the others idle.

3

Broadcast join

When one table is small, copy it to every worker instead of shuffling the big one. Fast, and the usual plan for dimension joins.

4

Spill

A sort or hash too large for memory overflows to disk and slows sharply. Reduce rows and columns before the heavy step.

DATA ENGINEERING LENS

Filter early, select few columns, and aggregate before joining. On a distributed engine, less data moved is the whole game.

Performance anti-patterns and their fixes

Anti-pattern	Why it hurts	Do this instead
<code>SELECT *</code>	Reads, moves and bills for every column	Name the columns you need
Function on a filtered column	Defeats indexes and partition pruning	Compare the bare column with a range
<code>DISTINCT</code> to hide duplicates	Adds a sort and conceals a join bug	Fix the grain of the join
<code>UNION</code> where <code>UNION ALL</code> would do	Deduplicates rows that cannot repeat	Use <code>UNION ALL</code>
Join first, aggregate later	Builds a huge intermediate result, risks fan-out	Aggregate each side to the join grain first
<code>ORDER BY</code> inside a CTE or view	Sorts rows that are re-sorted or discarded later	Sort once, in the final query
No filter on the partition column	Scans all history for a one-day question	Always bound the partition key
Row-by-row loops from code	One round trip per row	One set-based statement

In the cloud, slow queries are also expensive

Warehouses bill for the work a query does. Performance tuning and cost control are the same activity.

1

Pay per bytes scanned

BigQuery on-demand pricing charges for the data a query reads. Fewer columns and pruned partitions lower the bill directly.

2

Pay per compute time

Snowflake, Redshift and Databricks charge for the time a warehouse or cluster runs. Faster queries and idle suspension save money.

3

LIMIT may not save you

In a columnar engine, **LIMIT 10** on an unfiltered table can still scan and bill for the full columns. Filter on the partition key.

4

Materialise what repeats

If fifty dashboards recompute the same aggregate, compute it once in a table and let them read that.

★ PRO TIP

Before running a heavy query, check the estimate: BigQuery shows bytes to be scanned, and **EXPLAIN** shows the plan everywhere else.

Module 13 recap

- ✓ The optimizer chooses a plan from statistics; **EXPLAIN** shows it.
- ✓ Compare estimated with actual rows to spot bad plans.
- ✓ A B-tree index makes selective lookups fast and writes slower.
- ✓ In a composite index, the leading column must be constrained.
- ✓ Keep filtered columns bare so indexes and pruning can work.
- ✓ Columnar storage reads only the columns a query uses.
- ✓ Partitioning lets the engine skip whole slices of a table.
- ✓ Less data read and moved means faster and cheaper.

 NEXT UP

Module 14: the advanced toolkit



MODULE 14

Advanced toolkit

The remaining tools you will meet in real codebases: reshaping, nested data, functions, security and professional habits.

BY THE END YOU CAN

- 1 Pivot rows into columns
- 2 Work with arrays and nested JSON
- 3 Package logic in a function
- 4 Control access with roles and grants
- 5 Write SQL that others can review and trust

Pivoting rows into columns

```
1 SELECT
2     c.full_name,
3     COALESCE(SUM(o.amount) FILTER (
4         WHERE o.order_date < DATE '2025-04-01'), 0) AS mar,
5     COALESCE(SUM(o.amount) FILTER (
6         WHERE o.order_date >= DATE '2025-04-01'), 0) AS apr
7 FROM customers AS c
8 LEFT JOIN orders AS o
9     ON o.customer_id = c.customer_id
10 GROUP BY c.full_name
11 ORDER BY c.full_name;
```

- 1 One conditional aggregate per output column. Each sums only the rows for its month.
- 2 The left join keeps customers with no orders; **COALESCE** turns their empty sums into 0. Portable form: **SUM (CASE WHEN ...)**.

RESULT · 6 rows

full_name	mar	apr
Ada Moreno	190.00	90.00
Ben Carter	530.00	60.00
Chloe Dubois	90.00	75.00
Dev Patel	25.00	0.00
Elena Rossi	0.00	265.00
Farid Khan	0.00	0.00

Arrays: many values in one cell

ARRAY_AGG · ROWS INTO AN ARRAY

```
1 SELECT customer_id,  
2       ARRAY_AGG(order_id  
3             ORDER BY order_id) AS order_ids  
4 FROM orders  
5 WHERE customer_id <= 3  
6 GROUP BY customer_id  
7 ORDER BY customer_id;
```

RESULT · 3 rows

customer_id	order_ids
1	{1001,1003,1009}
2	{1002,1005,1008}
3	{1004,1010}

An aggregate that collects a group's values into one array instead of reducing them to a number.

UNNEST · AN ARRAY BACK INTO ROWS

```
1 SELECT u.order_id, u.pos  
2 FROM UNNEST(ARRAY[1001, 1003, 1009])  
3      WITH ORDINALITY AS u(order_id, pos)  
4 ORDER BY u.pos;
```

RESULT · 3 rows

order_id	pos
1001	1
1003	2
1009	3

UNNEST gives one row per element (**FLATTEN** in Snowflake). It is how one nested order document becomes one row per line item.

Reading nested JSON across dialects

The payload is {"user_id": 1, "props": {"page": "/home"}}. Extract the page as text in three engines.

POSTGRESQL

```
SELECT
  payload -> 'props'
    ->> 'page' AS page,
  (payload ->> 'user_id')::int
    AS user_id
FROM raw_events;
```

-> returns JSON, ->> returns text. Prefer the binary **JSONB** type, which can be indexed.

SNOWFLAKE

```
SELECT
  payload:props.page::string
    AS page,
  payload:user_id::int
    AS user_id
FROM raw_events;
```

Colon and dot paths on a **VARIANT** column, with a cast at the end.

BIGQUERY

```
SELECT
  JSON_VALUE(payload,
    '$.props.page') AS page,
  CAST(JSON_VALUE(payload,
    '$.user_id') AS INT64)
    AS user_id
FROM raw_events;
```

JSONPath strings. **JSON_VALUE** returns a scalar as text.

RULE OF THUMB

Keep the raw JSON in bronze. In silver, extract every field you rely on into a typed column, so a renamed key fails in one visible place.

User-defined functions package logic

PostgreSQL

```
1 CREATE OR REPLACE FUNCTION net_amount(  
2     gross      NUMERIC,  
3     vat_rate   NUMERIC DEFAULT 0.20  
4 )  
5 RETURNS NUMERIC  
6 LANGUAGE SQL  
7 IMMUTABLE  
8 AS $$  
9     SELECT ROUND(gross / (1 + vat_rate), 2);  
10 $$;  
11  
12 SELECT order_id, amount,  
13        net_amount(amount) AS net  
14 FROM orders  
15 ORDER BY order_id  
16 LIMIT 3;
```

- 1 A function takes parameters and returns a value. Defined once, used in any query.
- 2 **IMMUTABLE** promises the same output for the same input, which lets the engine optimise.
- 3 Called like a built-in function.

RESULT · 3 rows

order_id	amount	net
1001	135.00	112.50
1002	220.00	183.33
1003	55.00	45.83

⚠ TRAP

A business rule hidden in a function is invisible in the query that uses it. Keep functions small, and keep pipeline logic in reviewable models.

Security: roles and privileges

```
1  -- a role is a named bundle of permissions
2  CREATE ROLE analyst;
3
4  -- allow reading the modelled layer only
5  GRANT USAGE ON SCHEMA marts TO analyst;
6  GRANT SELECT ON ALL TABLES IN SCHEMA marts TO analyst;
7
8  -- make sure raw data stays closed
9  REVOKE ALL ON SCHEMA raw FROM analyst;
10
11 -- give the role to a person
12 GRANT analyst TO maria;
```

- 1 Grant privileges to roles, and roles to people. Never to individuals directly.
- 2 **USAGE** lets the role see into the schema; **SELECT** lets it read the tables.
- 3 Raw tables often hold personal data. Most users should never see them.

RULE OF THUMB

Least privilege

Every person and every pipeline gets the minimum access that lets it do its job. For sensitive columns, add masking policies or row-level security so one table can serve several audiences.

SQL injection and parameterised queries

When application code builds SQL by gluing strings together, user input can become executable SQL.

✗ VULNERABLE

```
1 name = input()    # user types: x' OR '1'='1'
2
3 sql = ("SELECT * FROM customers "
4        "WHERE full_name = '" + name + "'")
5 cur.execute(sql)
```

The database receives `WHERE full_name = 'x' OR '1'='1'`, which is true for every row. The whole table is returned. Worse inputs can delete data.

✓ SAFE

```
1 name = input()
2
3 sql = ("SELECT * FROM customers "
4        "WHERE full_name = %s")
5 cur.execute(sql, (name,))
```

The query text and the value travel **separately**. The driver passes the value as data, so it can never be read as SQL.

📌 RULE OF THUMB

Never build SQL from untrusted text. Use bind parameters for values, and an allow-list when a table or column name must vary.

Style: the same query, twice

SQL is read far more often than it is written, usually by someone debugging at an inconvenient hour.

✗ HARD TO REVIEW

```
1 select a.id,b.nm,sum(c.amt) x from
2 t1 a,t2 b,t3 c where a.id=b.id
3 and b.id=c.id and c.dt>'2025-01-01'
4 group by 1,2 order by 3 desc
```

- Aliases **a**, **b**, **c** and **x** say nothing.
- Comma joins hide which condition joins which table.
- One edit changes most of the lines in a diff.

✓ EASY TO REVIEW

```
1  SELECT
2      c.customer_id,
3      c.full_name,
4      SUM(o.amount) AS total_spent
5  FROM customers AS c
6  JOIN orders AS o
7      ON o.customer_id = c.customer_id
8  WHERE o.order_date >= DATE '2025-01-01'
9  GROUP BY c.customer_id, c.full_name
10 ORDER BY total_spent DESC;
```

- One clause per line, one column per line.
- Explicit **JOIN ... ON**, meaningful aliases.

Treat SQL as software

The practices that make application code reliable apply to pipelines without change.

1

Version control

Every model lives in Git. Changes arrive as pull requests that a colleague reviews.

2

Automated tests

Key, null and relationship tests run on every change, against a development copy of the data.

3

Separate environments

Develop in your own schema, promote to staging, then production. Never edit production tables by hand.

4

Documentation

Describe each table's grain and each column's meaning next to the code, so they change together.

★ PRO TIP

A linter such as SQLFluff enforces style automatically, so reviews can focus on logic instead of commas.

Dialect cheat sheet

Task	PostgreSQL	Snowflake	BigQuery	SQL Server
First n rows	<code>LIMIT n</code>	<code>LIMIT n</code>	<code>LIMIT n</code>	<code>SELECT TOP (n)</code>
Concatenate	<code>a b</code>	<code>a b</code>	<code>a b</code>	<code>a + b, CONCAT(a, b)</code>
Current time	<code>now()</code>	<code>CURRENT_TIMESTAMP()</code>	<code>CURRENT_TIMESTAMP()</code>	<code>SYSDATETIME()</code>
Days between	<code>d2 - d1</code>	<code>DATEDIFF('day', d1, d2)</code>	<code>DATE_DIFF(d2, d1, DAY)</code>	<code>DATEDIFF(day, d1, d2)</code>
Safe cast	none built in	<code>TRY_CAST(x AS INT)</code>	<code>SAFE_CAST(x AS INT64)</code>	<code>TRY_CAST(x AS INT)</code>
Filter a window	<code>subquery, DISTINCT ON</code>	<code>QUALIFY</code>	<code>QUALIFY</code>	<code>subquery</code>
Upsert	<code>ON CONFLICT, MERGE</code>	<code>MERGE</code>	<code>MERGE</code>	<code>MERGE</code>
Join strings	<code>STRING_AGG</code>	<code>LISTAGG</code>	<code>STRING_AGG</code>	<code>STRING_AGG</code>
JSON field	<code>col ->> 'k'</code>	<code>col:k::string</code>	<code>JSON_VALUE(col, '\$.k')</code>	<code>JSON_VALUE(col, '\$.k')</code>

Module 14 recap

- ✓ Pivot with one conditional aggregate per output column.
- ✓ `ARRAY_AGG` packs rows into an array; `UNNEST` unpacks them.
- ✓ Extract JSON fields into typed columns early.
- ✓ Functions reuse logic; keep them small and visible.
- ✓ Grant privileges to roles and follow least privilege.
- ✓ Always use bind parameters for values coming from outside.
- ✓ Format for the reviewer: one clause per line, explicit joins.
- ✓ Version, test and document SQL like any other code.

 NEXT UP**Capstone: build a tested pipeline end to end**



PUT IT ALL TOGETHER

Capstone

From a messy raw table to a tested daily revenue mart, using only what you have learned.

BY THE END YOU CAN

- 1 Deduplicate raw data to its latest state
- 2 Quarantine rows that fail integrity checks
- 3 Build a fact table at a declared grain
- 4 Load a mart idempotently
- 5 Prove the result with tests

The brief

Finance wants a table of daily completed revenue they can trust. The only input is `raw_orders`, which you have already met.

1

The input is messy

Nine rows for five orders: status changes, an exact duplicate, and one order for a customer who does not exist.

2

The output has a grain

`mart_daily_revenue`: one row per order date, with the number of completed orders and their revenue.

3

It must be re-runnable

The job will be retried and backfilled. Running it twice must not change the answer.

4

It must be proven

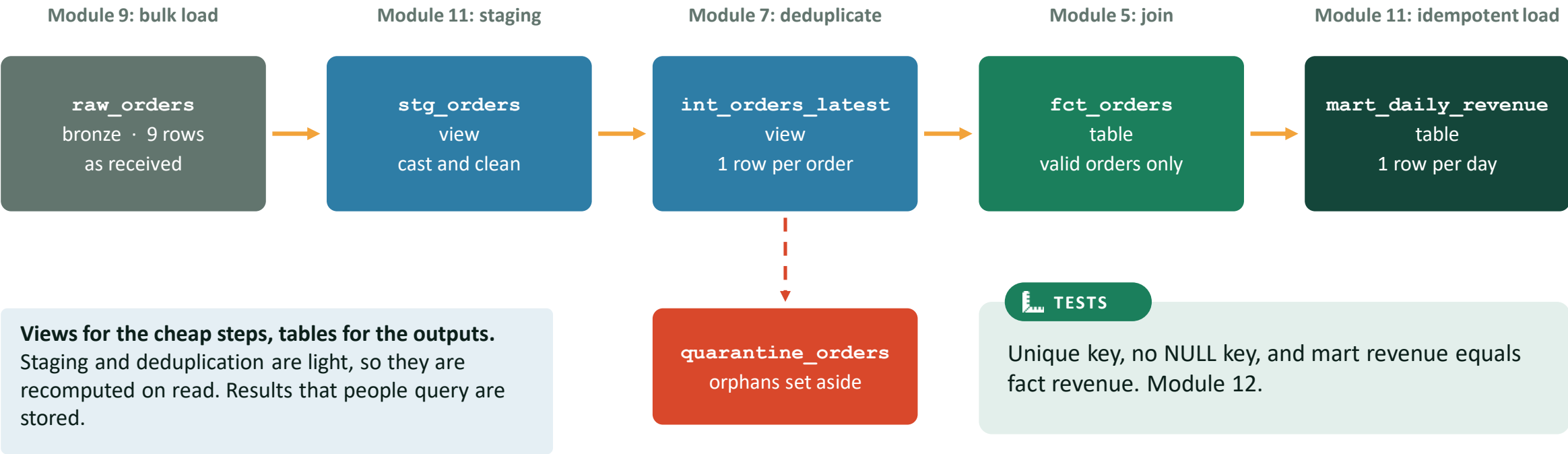
Tests confirm unique keys and that mart revenue reconciles to the fact table.

TRY IT

Before reading on, sketch the steps yourself. Which patterns from Modules 7, 11 and 12 do you need, and in what order?

The plan: four models and a test suite

Each box is one SQL statement. Arrows show which model reads which.



Step 1: latest version of every order

```
1 CREATE OR REPLACE VIEW int_orders_latest AS
2 WITH ranked AS (
3     SELECT *,
4         ROW_NUMBER() OVER (
5             PARTITION BY order_id
6             ORDER BY updated_at DESC, loaded_at DESC
7         ) AS rn
8     FROM stg_orders
9 )
10 SELECT order_id, customer_id, order_date,
11         status, amount, updated_at
12 FROM ranked
13 WHERE rn = 1;
```

- 1 Reads `stg_orders`, the staging view from Module 11.
- 2 One partition per business key, newest version first.
- 3 Nine raw rows become five.
- ⚡ Order 1012 is still here. Deduplication is not validation.

int_orders_latest · 5 rows

order_id	customer_id	status	amount
1001	1	completed	135.00
1002	2	completed	220.00
1003	1	completed	55.00
1004	3	cancelled	90.00
1012	9	completed	40.00

Step 2: quarantine orphans, build the fact

```
1 CREATE TABLE quarantine_orders AS
2 SELECT o.*
3 FROM int_orders_latest AS o
4 WHERE NOT EXISTS (
5     SELECT 1 FROM customers AS c
6     WHERE c.customer_id = o.customer_id);
7
8 CREATE TABLE fct_orders AS
9 SELECT o.order_id, o.customer_id,
10        c.country, c.segment,
11        o.order_date, o.status, o.amount
12 FROM int_orders_latest AS o
13 JOIN customers AS c
14     ON c.customer_id = o.customer_id;
```

1

2

1 Orders with no matching customer are set aside, not deleted.

2 The inner join keeps valid orders and adds customer attributes.



Grain: one row per order. Customers are unique, so the join cannot fan out.

fct_orders · 4 rows

order_id	country	segment	status	amount
1001	ES	consumer	completed	135.00
1002	UK	business	completed	220.00
1003	ES	consumer	completed	55.00
1004	FR	consumer	cancelled	90.00

Step 3: load the mart idempotently

```
1 BEGIN;
2
3 DELETE FROM mart_daily_revenue
4 WHERE order_date >= DATE '2025-03-01'
5      AND order_date < DATE '2025-03-08';
6
7 INSERT INTO mart_daily_revenue
8     (order_date, n_orders, revenue)
9 SELECT order_date, COUNT(*), SUM(amount)
10 FROM fct_orders
11 WHERE status = 'completed'
12      AND order_date >= DATE '2025-03-01'
13      AND order_date < DATE '2025-03-08'
14 GROUP BY order_date;
15
16 COMMIT;
```

1 Clear the slice this run is responsible for: the first week of March.

2 Recompute exactly that slice from the fact table.

3 Delete and insert commit together, so readers never see a half-loaded week.



The result shown was produced by running this block **twice**. Still two rows.

mart_daily_revenue AFTER TWO RUNS

order_date	n_orders	revenue
2025-03-01	2	355.00
2025-03-05	1	55.00

Step 4: prove it with tests

```
1 SELECT 'fct_orders: duplicate order_id' AS test,
2     COUNT(*) AS failures
3 FROM (SELECT order_id FROM fct_orders
4       GROUP BY order_id HAVING COUNT(*) > 1) AS d
5 UNION ALL
6 SELECT 'fct_orders: order_id is null', COUNT(*)
7 FROM fct_orders WHERE order_id IS NULL
8 UNION ALL
9 SELECT 'mart: revenue does not reconcile',
10     CASE WHEN (SELECT SUM(revenue) FROM mart_daily_revenue)
11              = (SELECT SUM(amount) FROM fct_orders
12                 WHERE status = 'completed')
13              THEN 0 ELSE 1 END
14 UNION ALL
15 SELECT 'quarantine: orders waiting', COUNT(*)
16 FROM quarantine_orders;
```

- 1 Uniqueness of the fact's key.
- 2 Reconciliation: mart total equals completed revenue in the fact.
- 3 Not a failure but a signal: one order is waiting for its customer.

RESULT · 4 rows

test	failures
fct_orders: duplicate order_id	0
fct_orders: order_id is null	0
mart: revenue does not reconcile	0
quarantine: orders waiting	1

From working to production-ready

The capstone is correct. Here is what a team would add before relying on it every night.

1

Parameterise the dates

Replace the literal week with the orchestrator's logical date, so the same SQL serves daily runs and backfills.

2

Make the fact incremental

Once `raw_orders` is large, switch `fct_orders` from a rebuild to a `MERGE` with a lookback window.

3

Alert on the quarantine

A growing quarantine table means an upstream problem. Someone should be told.

4

Add freshness and volume checks

Catch the night when no file arrived, before finance opens the dashboard.

5

Document the grain

Write "one row per order date, completed orders only" where every consumer will see it.

6

Put it under version control

Each model in its own file, reviewed, tested in a development schema, then deployed.



REFERENCE AND NEXT STEPS

Wrap-up

A pattern index, the questions interviewers actually ask, and where to go from here.

BY THE END YOU CAN

- 1 Find the right pattern for a problem quickly
- 2 Answer common data engineering SQL questions
- 3 Plan what to learn next

Pattern index: problem to SQL tool

When you need to	Reach for	Module
Keep the latest row per key	<code>ROW_NUMBER() ... WHERE rn = 1</code>	7
Find rows with no match in another table	<code>NOT EXISTS, LEFT JOIN ... IS NULL</code>	5
Show zeros for periods with no data	<code>date spine + LEFT JOIN</code>	5, 6
Compare a row with the previous one	<code>LAG() OVER (PARTITION BY ... ORDER BY ...)</code>	7
Compute several conditional totals in one scan	<code>SUM(CASE WHEN ...), FILTER (WHERE ...)</code>	4
Group consecutive rows into runs or sessions	<code>flag + running SUM, date minus ROW_NUMBER</code>	7
Insert new keys and update existing ones	<code>MERGE, INSERT ... ON CONFLICT</code>	9
Reload a period without double counting	<code>DELETE + INSERT in one transaction</code>	11
Join a fact to the right historical version	<code>range join on valid_from / valid_to</code>	10
Check two tables are identical	<code>EXCEPT in both directions</code>	12

Interview questions: querying

Short answers to practise saying aloud. Each one is developed in the module shown.

Question	A strong short answer	Module
WHERE versus HAVING?	WHERE filters rows before grouping; HAVING filters groups after aggregation and can use aggregates.	4
RANK versus DENSE_RANK versus ROW_NUMBER?	All number rows in order. ROW_NUMBER is always unique, RANK leaves gaps after ties, DENSE_RANK does not.	7
Why did my LEFT JOIN lose rows?	A WHERE condition on the right table rejects the NULLs of unmatched rows. Move it into ON .	5
Why is my total too high after a join?	Fan-out: a one-to-many join repeated the "one" side. Aggregate to the join grain first.	5
UNION versus UNION ALL?	UNION removes duplicates and pays for a sort; UNION ALL just appends.	6

Interview questions: engineering

Question	A strong short answer	Module
What does idempotent mean for a pipeline?	Re-running it leaves the same result. Achieved by replacing a slice, merging by key, or rebuilding.	11
How do you deduplicate a table?	Number rows per business key with <code>ROW_NUMBER</code> , newest first with a tiebreaker, and keep row 1.	7
What is a Type 2 slowly changing dimension?	A dimension that keeps history: each change adds a row with validity dates and a new surrogate key.	10
How would you speed up a slow query?	Read the plan. Check row estimates, filter and prune earlier, reduce columns, fix join grain, then consider indexes or clustering.	13
How do you handle late-arriving data?	Reprocess a lookback window on each run and <code>MERGE</code> by key so the reprocessing is idempotent.	11
What is the grain of a table?	What one row represents. It decides which measures are valid and which joins are safe.	10

Where to go next

You have the language. The next layer is the tooling that runs it at scale, in roughly this order.



★ PRO TIP

The fastest way to learn

Take a query that works and ask: what is its grain, what happens on a re-run, what happens with a NULL or a duplicate? Those three questions are the job.

YOU MADE IT

You can now read, write and ship SQL like a data engineer

From SELECT to a tested, idempotent pipeline, on one small dataset you can verify by hand.

1

Know the grain

Say what one row means before you join or aggregate.

2

Make it re-runnable

Replace, merge or rebuild. Never append blindly.

3

Prove it

A result without a test is a rumour.